

SoMachine Basic

Guide de la bibliothèque des fonctions génériques

04/2014

Le présent document comprend des descriptions générales et/ou des caractéristiques techniques des produits mentionnés. Il ne peut pas être utilisé pour définir ou déterminer l'adéquation ou la fiabilité de ces produits pour des applications utilisateur spécifiques. Il incombe à chaque utilisateur ou intégrateur de réaliser l'analyse de risques complète et appropriée, l'évaluation et le test des produits pour ce qui est de l'application à utiliser et de l'exécution de cette application. Ni la société Schneider Electric ni aucune de ses sociétés affiliées ou filiales ne peuvent être tenues pour responsables de la mauvaise utilisation des informations contenues dans le présent document. Si vous avez des suggestions, des améliorations ou des corrections à apporter à cette publication, veuillez nous en informer.

Aucune partie de ce document ne peut être reproduite sous quelque forme ou par quelque moyen que ce soit, électronique, mécanique ou photocopie, sans autorisation préalable de Schneider Electric.

Toutes les réglementations de sécurité pertinentes locales doivent être observées lors de l'installation et de l'utilisation de ce produit. Pour des raisons de sécurité et afin de garantir la conformité aux données système documentées, seul le fabricant est habilité à effectuer des réparations sur les composants.

Lorsque des équipements sont utilisés pour des applications présentant des exigences techniques de sécurité, suivez les instructions appropriées.

La non-utilisation du logiciel Schneider Electric ou d'un logiciel approuvé avec nos produits matériels peut entraîner des blessures, des dommages ou un fonctionnement incorrect.

Le non-respect de cette consigne peut entraîner des lésions corporelles ou des dommages matériels.

© 2014 Schneider Electric. Tous droits réservés.

Table des matières



	Consignes de sécurité	9
	A propos de ce manuel	11
Chapitre 1	Introduction	15
	Utilisation des exemples de code source	16
	Blocs opération	19
	Blocs comparaison	20
Chapitre 2	Objets langage	21
	Objets	22
	Objets bit mémoire	23
	Objets d'E/S	25
	Objets mot	28
	Objets mot double et flottants	33
	<Objets structurés	37
	Objets indexés	40
	Objets de bloc fonction	42
Chapitre 3	Procédure	45
3.1	Traitement booléen	46
	Instructions booléennes	47
	Opérateurs de chargement (LD, LDN, LDR, LDF)	50
	Opérateurs d'affectation (ST, STN, R, S)	52
	Opérateurs AND logiques (AND, ANDN, ANDR, ANDF)	54
	Opérateurs OR logiques (OR, ORN, ORR, ORF)	56
	Opérateurs OR exclusifs (XOR, XORN, XORR, XORF)	58
	Opérateur NOT (N)	61
	Instructions de comparaison	62
3.2	Traitement numérique	64
	Introduction aux instructions numériques	65
	Instructions d'affectation	66
	Affectation de chaînes de bits	67
	Affectation de mots	69
	Opérateurs arithmétiques sur des entrées	71
	Instructions logiques	75
	Instructions de décalage	77
	Instruction de conversion BCD/binaire	79
	Instructions de conversion de mot simple/double	81

3.3	Programme	82
	Instructions END	83
	Instructions NOP	84
	Instructions de saut	85
	Instructions de sous-programme	87
3.4	Virgule flottante	89
	Instructions arithmétiques sur les objets à virgule flottante	90
	Instructions trigonométriques	93
	Instructions de conversion d'angle	95
	Instructions de conversion d'entiers/nombres à virgule flottante	96
3.5	ASCII	98
	Instructions ROUND	99
	Instructions de conversion ASCII/entier	101
	Instructions de conversion entier/ASCII	103
	Instruction de conversion ASCII/valeur flottante	105
	Instructions de conversion valeur flottante/ASCII	107
3.6	Opérateurs de pile	109
	Instructions de pile (MPS, MRD, MPP)	109
3.7	Instructions sur tables d'objets	111
	Affectation de tables de mots, mots doubles et valeurs flottantes	112
	Fonctions de sommation de table	114
	Fonction de comparaison de tables	116
	Fonctions de recherche dans une table	118
	Fonctions de recherche de valeurs maximum et minimum dans des tables	120
	Nombre d'occurrences d'une valeur dans une table	122
	Fonctions de décalage circulaire dans une table	123
	Fonctions de tri de table	125
	Fonctions LKUP (interpolation de table à virgule flottante)	126
	Fonctions MEAN des valeurs d'une table de valeurs flottantes	131
Chapitre 4	Objets logiciels	133
4.1	Utilisation des blocs fonction	134
	Principes de programmation des blocs fonction	135
	Ajout d'un bloc fonction	137
	Configuration d'un bloc fonction	139

4.2	Timer (%TM)	140
	Description	141
	Configuration	142
	TON : Timer On-Delay	144
	TOF : Timer Off-Delay	145
	TP : Pulse Timer	146
	Exemple de programmation	147
4.3	LIFO/FIFO Register (%R)	148
	Description	149
	Configuration	151
	Fonction du registre LIFO	153
	Fonctionnement du registre FIFO	154
	Exemple de programmation	155
4.4	Shift Bit Register (%SBR)	157
	Description	158
	Configuration	159
	Exemple de programmation	161
4.5	Step Counter (%SC)	162
	Description	163
	Configuration	164
	Exemple de programmation	165
4.6	Counter (%C)	168
	Description	169
	Configuration	171
	Exemple de programmation	174
4.7	Bloc fonction Fast Counter (%FC)	176
	Fast Counter	176
4.8	Bloc fonction High Speed Counter (%HSC)	177
	High Speed Counter	177
4.9	Drum Register (%DR)	178
	Description	179
	Configuration	180
	Exemple de programmation	183
4.10	Bloc fonction Pulse (%PLS)	186
	Pulse	186
4.11	Bloc fonction Pulse Width Modulation (%PWM)	187
	Pulse Width Modulation	187

4.12	Message (%MSG) et échange (EXCH)	188
	Présentation	189
	Description	191
	Configuration	194
	Exemple de programmation	199
	Exemples ASCII	201
	Requêtes et exemples Modbus standard	203
Chapitre 5	Blocs fonction Schedule Blocks (%SCH)	213
	Description	214
	Programmation et configuration	216
Chapitre 6	Fonction PID	219
6.1	Modes de marche des PID	220
	Modes de fonctionnement PID Modes de fonctionnement	220
6.2	Configuration Auto-Tuning du PID	222
	Configuration d'Auto-Tuning du PID	222
6.3	Configuration standard du PID	226
	Configuration de l'adresse de mot du PID	227
	Réglage du PID avec la fonction Auto-Tuning (AT)	230
	Mode Manuel	233
	Détermination de la période d'échantillonnage (Ts)	235
6.4	Assistant PID	238
	Accès à l'Assistant PID	239
	Onglet Général	241
	Onglet Entrée	244
	Onglet PID	245
	Onglet AT	247
	Onglet Sortie	249
6.5	Programmation du PID	251
	Description	252
	Programmation et configuration	254
	Etats et codes d'erreur détectée de PID	255
Chapitre 7	Fonctions d'horloge	259
	Fonctions d'horloge	260
	Horodatage	261
	Réglage de la date et de l'heure	263
Annexes		267

Annexe A	Paramètres du PID	269
	Rôle et influence des paramètres PID	270
	Méthode de réglage des paramètres PID	272
Glossaire		275
Index		277

Consignes de sécurité



Informations importantes

AVIS

Lisez attentivement ces instructions et examinez le matériel pour vous familiariser avec l'appareil avant de tenter de l'installer, de le faire fonctionner ou d'assurer sa maintenance. Les messages spéciaux suivants que vous trouverez dans cette documentation ou sur l'appareil ont pour but de vous mettre en garde contre des risques potentiels ou d'attirer votre attention sur des informations qui clarifient ou simplifient une procédure.



La présence d'un de ces symboles sur une étiquette de sécurité Danger collée sur un équipement indique qu'un risque d'électrocution existe, susceptible d'entraîner la mort ou des blessures corporelles si les instructions ne sont pas respectées.



Ce symbole est le symbole d'alerte de sécurité. Il vous avertit d'un risque de blessures corporelles. Respectez scrupuleusement les consignes de sécurité associées à ce symbole pour éviter de vous blesser ou de mettre votre vie en danger.

DANGER

DANGER indique une situation immédiatement dangereuse qui, si elle n'est pas évitée, **entraînera** la mort ou des blessures graves.

AVERTISSEMENT

AVERTISSEMENT indique une situation potentiellement dangereuse et **susceptible d'entraîner** la mort ou des blessures graves.

ATTENTION

ATTENTION indique une situation potentiellement dangereuse et **susceptible d'entraîner** des blessures mineures ou modérées.

AVIS

AVIS indique des pratiques n'entraînant pas de risques corporels.

REMARQUE IMPORTANTE

L'installation, l'utilisation, la réparation et la maintenance des équipements électriques doivent être assurées par du personnel qualifié uniquement. Schneider Electric décline toute responsabilité quant aux conséquences de l'utilisation de ce matériel.

Une personne qualifiée est une personne disposant de compétences et de connaissances dans le domaine de la construction, du fonctionnement et de l'installation des équipements électriques, et ayant suivi une formation en sécurité leur permettant d'identifier et d'éviter les risques encourus.

A propos de ce manuel



Présentation

Objectif du document

Ce guide explique comment utiliser les blocs fonction et les instructions dans les programmes que vous créez avec le logiciel SoMachine Basic. Ces descriptions s'appliquent à tous les Logic Controller pris en charge par SoMachine Basic.

Champ d'application

Les informations présentées dans ce manuel ne concernent **que** les produits compatibles avec SoMachine Basic.

Ce document a été mis à jour suite au lancement de SoMachine Basic V1.1.

Les caractéristiques techniques des équipements décrits dans ce document sont également fournies en ligne. Pour accéder à ces informations en ligne :

Etape	Action
1	Accédez à la page d'accueil de Schneider Electric www.schneider-electric.com .
2	Dans la zone Search , saisissez la référence d'un produit ou le nom d'une gamme de produits. ● N'insérez pas d'espaces dans le numéro de modèle ou la gamme de produits. ● Pour obtenir des informations sur un ensemble de modules similaires, utilisez des astérisques (*).
3	Si vous avez saisi une référence, accédez aux résultats de recherche Product datasheets et cliquez sur la référence qui vous intéresse. Si vous avez saisi une gamme de produits, accédez aux résultats de recherche Product Ranges et cliquez sur la gamme de produits qui vous intéresse.
4	Si plusieurs références s'affichent dans les résultats de recherche Products , cliquez sur la référence qui vous intéresse.
5	Selon la taille de l'écran, vous serez peut-être amené à faire défiler la page pour consulter la fiche technique.
6	Pour enregistrer ou imprimer une fiche technique au format .pdf, cliquez sur Download XXX product datasheet .

Les caractéristiques présentées dans ce manuel devraient être identiques à celles fournies en ligne. Toutefois, en application de notre politique d'amélioration continue, nous pouvons être amenés à réviser le contenu du document afin de le rendre plus clair et plus précis. Si vous constatez une différence entre le manuel et les informations fournies en ligne, utilisez ces dernières en priorité.

Document(s) à consulter

Titre de documentation	Référence
SoMachine Basic - Guide d'utilisation	EIO0000001354 (ENG) EIO0000001355 (FRA) EIO0000001356 (GER) EIO0000001357 (SPA) EIO0000001358 (ITA) EIO0000001359 (CHS) EIO0000001366 (POR) EIO0000001367 (TUR)

Vous pouvez télécharger ces publications et autres informations techniques depuis notre site web à l'adresse : www.schneider-electric.com.

Information spécifique au produit

AVERTISSEMENT

PERTE DE CONTROLE

- Le concepteur d'un circuit de commande doit tenir compte des modes de défaillance potentiels des canaux de commande et, pour certaines fonctions de commande critiques, prévoir un moyen d'assurer la sécurité en maintenant un état sûr pendant et après la défaillance. Par exemple, l'arrêt d'urgence, l'arrêt en cas de surcourse, la coupure de courant et le redémarrage sont des fonctions de commande cruciales.
- Des canaux de commande séparés ou redondants doivent être prévus pour les fonctions de commande critiques.
- Les liaisons de communication peuvent faire partie des canaux de commande du système. Une attention particulière doit être prêtée aux implications des délais de transmission non prévus ou des pannes de la liaison.
- Respectez toutes les réglementations de prévention des accidents ainsi que les consignes de sécurité locales.¹
- Chaque implémentation de cet équipement doit être testée individuellement et entièrement pour s'assurer du fonctionnement correct avant la mise en service.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

¹ Pour plus d'informations, consultez le document NEMA ICS 1.1 (dernière édition), « Safety Guidelines for the Application, Installation, and Maintenance of Solid State Control » (Directives de sécurité pour l'application, l'installation et la maintenance de commande statique) et le document NEMA ICS 7.1 (dernière édition), « Safety Standards for Construction and Guide for Selection, Installation, and Operation of Adjustable-Speed Drive Systems » (Normes de sécurité relatives à la construction et manuel de sélection, installation et opération de variateurs de vitesse) ou son équivalent en vigueur dans votre pays.

AVERTISSEMENT

COMPORTEMENT IMPREVU DE L'EQUIPEMENT

- N'utilisez que le logiciel approuvé par Schneider Electric pour faire fonctionner cet équipement.
- Mettez à jour votre programme d'application chaque fois que vous modifiez la configuration matérielle physique.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Chapitre 1

Introduction

Présentation

Ce chapitre explique comment utiliser les exemples de code source et les blocs requis pour exécuter de multiples exemples d'opérations et instructions d'affectation fournies dans ce document.

Contenu de ce chapitre

Ce chapitre contient les sujets suivants :

Sujet	Page
Utilisation des exemples de code source	16
Blocs opération	19
Blocs comparaison	20

Utilisation des exemples de code source

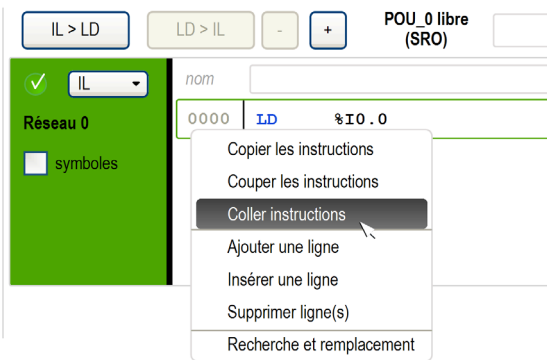
Présentation

Sauf mention contraire, les exemples de code source contenus dans ce manuel sont valides pour les deux langages de programmation LD (schéma à contacts) et IL (liste d'instructions). Un exemple complet peut néanmoins nécessiter plus d'un réseau.

Procédure de réversibilité

Le présent manuel illustre uniquement du code source IL.

Pour obtenir le code LD équivalent, procédez comme suit :

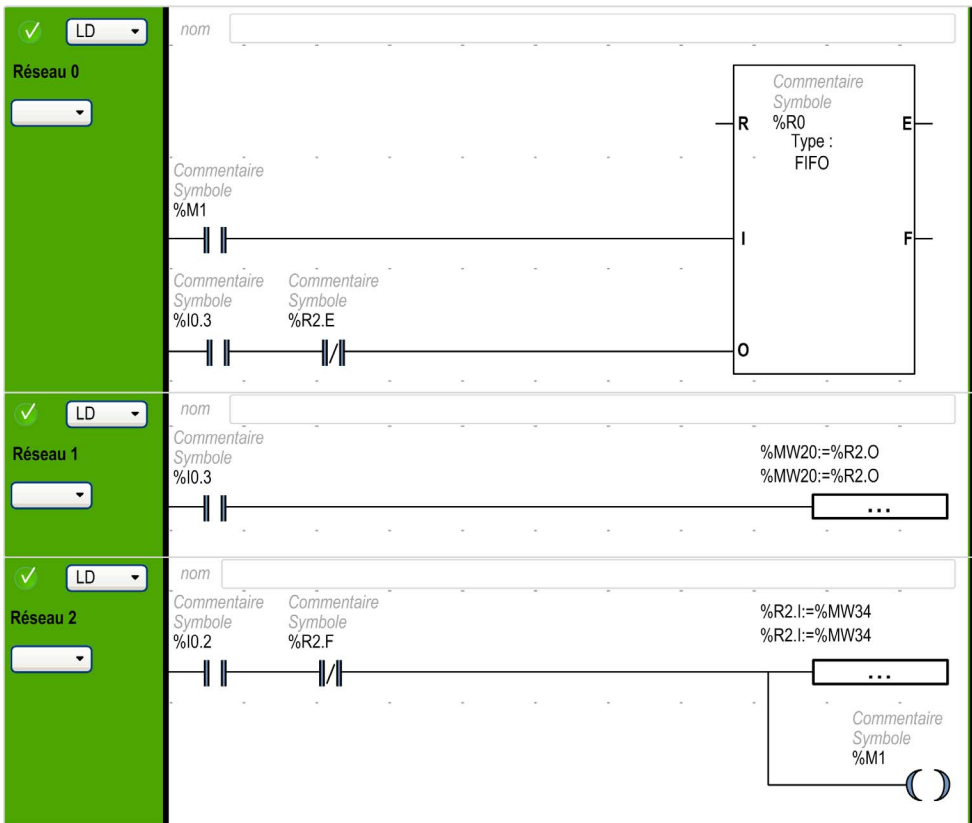
Etape	Action
1	Dans SoMachine Basic, créez un POU contenant un réseau vide.
2	Dans ce réseau, cliquez sur le bouton LD > IL pour afficher le code source en langage IL.
3	Sélectionnez le code source du premier réseau de l'exemple de programme et copiez-le dans le presse-papiers (Ctrl+C).
4	Cliquez avec le bouton droit de la souris sur la ligne numéro 0000 de la première instruction et choisissez Coller instructions pour insérer la copie du code source dans le réseau :  NOTE : n'oubliez pas de supprimer l'instruction LD de la dernière ligne du réseau, si vous avez collé les instructions en insérant la ou les lignes avant l'opérateur LD par défaut.
5	Cliquez sur le bouton IL > LD pour afficher le code source LD.
6	Répétez les étapes précédentes pour les autres réseaux de l'exemple de programme. Cliquez  sur dans la barre d'outils pour ajouter des réseaux.

Exemple

Programme en langage IL :

Réseau	Code source
0	BLK %R0 LD %M1 I LD %I0.3 ANDN %R2.E O END_BLK
1	LD %I0.3 [%MW20:=%R2.O]
2	LD %I0.2 ANDN %R2.F [%R2.I:=%MW34] ST %M1

Schéma à contacts (LD) correspondant :



Blocs opération

Insertion d'opérations et d'instructions d'affectation en langage Liste d'instructions dans des schémas à contacts

Utilisez le symbole graphique **Bloc opération** pour insérer des opérations et instructions d'affectation en langage Liste d'instructions dans des réseaux en langage Schéma à contacts.

Symbole

Expression d'opération



Pour insérer un bloc opération dans un réseau, procédez comme suit :

Etape	Action
1	Cliquez sur le bouton Bloc opération  dans la barre d'outils.
2	Cliquez dans la zone d'action (les deux dernières colonnes) du réseau pour insérer le Bloc opération .
3	Double-cliquez sur la ligne expression d'opération .
4	Saisissez une opération ou une instruction d'affectation en langage Liste d'instructions.

Obtention d'aide sur la syntaxe

Si la syntaxe de l'opération ou de l'instruction d'affectation en langage Liste d'instructions est incorrecte, la bordure du champ **expression d'opération** devient rouge. Pour obtenir de l'aide, procédez au choix comme suit :

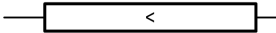
- Placez le curseur de la souris sur la ligne **expression d'opération**. ou
- Sélectionnez **Outils** → **Messages de programme**.

Blocs comparaison

Insertion d'expressions de comparaison en langage Liste d'instructions dans des schémas à contacts

Utilisez le symbole graphique **Bloc comparaison** pour insérer des expressions de comparaison en langage Liste d'instructions dans les réseaux en langage Schéma à contacts.

Symbole
Expression de comparaison



Procédez comme suit :

Etape	Action
1	Cliquez sur le bouton Bloc opération  dans la barre d'outils.
2	Cliquez n'importe où dans le réseau pour insérer le Bloc comparaison .
3	Double-cliquez sur la ligne Expression de comparaison .
4	Saisissez une opération de comparaison valide en langage Liste d'instructions.

Obtention d'aide sur la syntaxe

Si la syntaxe de l'opération de comparaison en langage Liste d'instructions est incorrecte, la bordure du champ **Expression de comparaison** devient rouge. Pour obtenir de l'aide, procédez au choix comme suit :

- Placez le curseur de la souris sur la ligne **Expression de comparaison**. ou
- Sélectionnez **Outils** → **Messages de programme**.

Chapitre 2

Objets langage

Contenu de ce chapitre

Ce chapitre contient les sujets suivants :

Sujet	Page
Objets	22
Objets bit mémoire	23
Objets d'E/S	25
Objets mot	28
Objets mot double et flottants	33
<Objets structurés	37
Objets indexés	40
Objets de bloc fonction	42

Objets

Présentation

Dans SoMachine Basic, le terme *objet* représente une zone mémoire d'un Logic Controller réservée à l'usage d'une application. Exemples d'objets :

- Variables logicielles simples (bits et mots mémoire par exemple)
- Adresses d'entrées et de sorties numériques ou analogiques
- Variables internes du contrôleur (mots et bits système par exemple)
- Fonctions système ou blocs fonction prédéfinis (temporisateurs ou compteurs par exemple)

La mémoire du contrôleur est soit pré-allouée à certains types d'objets, soit allouée automatiquement lorsqu'une application est téléchargée dans le Logic Controller.

Les objets ne peuvent être adressés dans un programme qu'après allocation de mémoire.

L'adressage des objets utilise le préfixe `%`. Par exemple, `%MW12` est l'adresse d'un mot mémoire, `%Q0.3` est l'adresse d'une sortie numérique intégrée et `%TM0` est l'adresse d'un bloc fonction Timer.

Objets bit mémoire

Introduction

Les objets bit mémoire sont des variables logicielles binaires, qui peuvent être utilisées comme des opérandes et testées par des instructions booléennes.

Exemples d'objets bit :

- Bits de mémoire
- Bits système
- Bits étape
- Bits extraits de mots

Les objets valides sont compris entre 0 et la référence maximum utilisée dans votre application (consultez le *guide de programmation* de votre Logic Controller).

Syntaxe

Respectez la syntaxe suivante pour adresser des objets bit mémoire, système et d'étape :

%	M, S ou X	i
Symbole	Type d'objet	Identificateur d'instance d'objet

Le tableau suivant décrit les éléments de la syntaxe d'adressage :

Groupe	Élément	Description
Symbole	%	Une variable logicielle doit toujours débiter par un symbole de pourcentage (%).
Type d'objet	M	Les bits mémoire stockent des valeurs intermédiaires pendant l'exécution d'un programme.
	S	Les bits système donnent des informations d'état et de régulation concernant le contrôleur.
	X	Les bits d'étape indiquent l'état des activités des étapes Grafcet.
Identificateur d'instance d'objet	i	L'identificateur de l'objet représentant leur instance séquentielle en mémoire. Le nombre maximum d'objets dépend du nombre d'objets configurés aux limites de la mémoire disponible. Pour connaître la quantité maximale de mémoire disponible, consultez le <i>guide de programmation</i> de votre Logic Controller.

Pour plus d'informations sur l'adressage des bits d'E/S, consultez la section Objets d'E/S ([voir page 25](#)).

Pour plus d'informations sur l'adressage d'un bit extrait d'un mot, consultez la section Extraction d'un bit d'un objet mot ([voir page 32](#)).

Description

Ce tableau répertorie et décrit les bits mémoire, système et d'étape utilisés comme opérandes dans les instructions booléennes :

Type	Description	Adresse ou valeur	Accès en écriture ⁽¹⁾
Valeurs immédiates	0 ou 1 (False ou True)	0 ou 1	–
Mémoire	Les bits mémoire sont des zones mémoire interne qui stockent des valeurs binaires. Remarque : les objets d'E/S inutilisés ne peuvent pas être utilisés comme des bits mémoire.	%Mi	Oui
Système	Les bits système %S0 à %S127 vous permettent de surveiller le bon fonctionnement du contrôleur ainsi que l'exécution correcte du programme de l'application, et de contrôler certaines fonctionnalités du système.	%Si	Dépend de i.
Étapes Grafcet	Les bits %Xi à %Xi sont associés aux étapes Grafcet. Le bit d'étape xi est à 1 lorsque l'étape correspondante est active et à l'état 0 lorsqu'elle est désactivée.	%Xi	Oui
(1) Ecrit par le programme ou à l'aide d'une table d'animation.			

Exemple

Ce tableau présente quelques exemples d'adressage d'objets bit :

Objet bit	Description
%M25	Bit mémoire numéro 25
%S20	Bit système numéro 20
%X4	Étape Grafcet numéro 4

Objets d'E/S

Introduction

Les objets d'E/S contiennent des bits et des mots. Chaque entrée et sortie physique est mappée à ces objets dans la mémoire interne. Les objets bit d'E/S peuvent être utilisés comme des opérandes et testés à l'aide d'instructions booléennes. Les objets mot d'E/S sont utilisables dans la plupart des instructions non booléennes, comme les fonctions et les instructions contenant des opérateurs arithmétiques.

Exemples d'objets d'E/S :

- Entrées numériques
- Sorties analogiques
- Entrées analogiques
- Sorties analogiques
- Entrées et sorties de communication

La plage d'objets autorisés va de 0 à la valeur maximale configurée et prise en charge par votre contrôleur (consultez le guide de référence du matériel et le guide de programmation de votre Logic controller).

Syntaxe

Cette figure illustre le format d'adresse des entrées/sorties :

%	I, Q, IW, QW, IWS ou QWS	y	.	z
Symbole	Type d'objet	Numéro de module	Point	Numéro de voie

Le tableau suivant décrit les composants du format d'adressage :

Composant	Elément	Valeur	Description
Icône	%	—	Une adresse interne doit toujours débiter par le symbole de pourcentage (%).
Type d'objet	I	—	Entrée numérique (objet bit)
	Q	—	Sortie numérique (objet bit)
	IW	—	Valeur d'entrée analogique (objet bit)
	QW	—	Valeur de sortie analogique (objet bit)
	IWS	—	Etat d'entrée analogique (objet bit)
	QWS	—	Etat de sortie analogique (objet bit)
(1) <i>m</i> est le nombre de modules locaux configurés (7 au maximum).			
(2) <i>n</i> est le nombre de modules distants configurés (<i>n</i> +7 au maximum). La position maximale a le numéro 14.			

Composant	Elément	Valeur	Description
Numéro de module	y	0	Voie d'E/S intégrées sur le Logic Controller.
		1 à $m^{(1)}$	Voie d'E/S sur un module d'extension connecté directement au contrôleur.
		$m+1 \dots n^{(2)}$	Voie d'E/S sur un module d'extension connecté à l'aide de modules émetteur/récepteur TM3.
Numéro de voie	z	0 à 31	Numéro de la voie d'E/S sur le Logic Controller ou le module d'extension. Nombre de voies disponibles variable selon le modèle de Logic Controller ou le type du module d'extension.
(1) m est le nombre de modules locaux configurés (7 au maximum). (2) n est le nombre de modules distants configurés (n+7 au maximum). La position maximale a le numéro 14.			

Description

Ce tableau répertorie et décrit tous les objets d'E/S utilisés comme opérandes dans des instructions :

Type	Adresse ou valeur	Accès en écriture ⁽¹⁾	Description
Bits d'entrée	%I _y .z ⁽²⁾	Non ⁽³⁾	Ces bits sont des images logiques des états électriques des E/S numériques physiques. Ils sont stockés dans la mémoire des données et mis à jour entre chaque scrutation de la logique du programme.
Bits de sortie	%Q _y .z ⁽²⁾	Oui	
Mot d'entrée	%IW _y .z ⁽²⁾	Non	Ces objets mot contiennent la valeur analogique de la voie correspondante.
Mot de sortie	%QW _y .z ⁽²⁾	Oui	
Etat du mot d'entrée	%IWS _y .z ⁽²⁾	Non	Ces objets mot contiennent l'état de la voie analogique correspondante.
Etat du mot de sortie	%QWS _y .z ⁽²⁾	Non	

(1) Ecrit par le programme ou à l'aide d'une table d'animation.

(2) _y est le numéro du module et _z celui de la voie. Pour une description de _y et _z, consultez la section Syntaxe d'adressage des E/S (*voir page 25*).

(3) Vous ne pouvez pas écrire de données dans les bits d'entrée, mais vous pouvez les forcer.

Exemples

Ce tableau présente quelques exemples d'adressage des E/S :

Objet d'E/S	Description
%I0.5	Voie d'entrée numérique numéro 5 sur le contrôleur (E/S intégrées correspondant au module numéro 0).
%Q3.4	Voie de sortie numérique numéro 4 sur le module d'extension à l'adresse 3 (E/S du module d'extension).
%IW0.1	Entrée analogique n° 1 sur le contrôleur (E/S intégrée).
%QW2.1	Sortie analogique n° 1 sur le module d'extension à l'adresse 2 (E/S du module d'extension).
%IWS0.1	Etat de l'entrée analogique numéro 1 sur le contrôleur (E/S intégrées).
%QWS1.1	Etat de la sortie analogique 1 sur le module d'extension à l'adresse 1 (E/S du module d'extension).

Objets mot

Introduction

Les objets mot adressés sous la forme de mots de 16 bits sont stockés dans la mémoire de données et peuvent contenir un entier compris entre -32 768 et 32 767 (sauf pour le bloc fonction **Fast Counter** qui est compris entre 0 et 65 535).

Exemples d'objets mot :

- Valeurs immédiates
- Mots mémoire (%MWi)
- Mots constants (%KWi)
- Mots d'échange d'E/S (%IWi, %QWi, %IWSi, %QWSi)
- Mots système (%SWi)
- Blocs fonction (données de configuration et/ou d'exécution)

Les objets valides sont compris entre 0 et la référence maximum utilisée dans votre application (consultez le Guide de programmation de votre Logic Controller).

Par exemple, si la référence maximum de votre application pour les mots mémoire est %MW9, l'espace alloué va de %MW0 à %MW9. Dans cet exemple, %MW10 n'est pas valide. Aucun accès à cette zone n'est autorisé, tant de manière interne qu'externe.

Syntaxe

Respectez la syntaxe suivante pour adresser des mots mémoire, constants et système :

%	M, K ou S	W	i
Symbole	Type d'objet	Format	Identificateur d'instance d'objet

Le tableau suivant décrit les éléments de la syntaxe d'adressage :

Groupe	Elément	Description
Symbole	%	Une adresse interne doit toujours débuter par le symbole de pourcentage (%).
Type d'objet	M	Les mots mémoire stockent des valeurs pendant l'exécution d'un programme.
	K	Les mots constants permettent de stocker des valeurs constantes ou des messages alphanumériques. Leur contenu ne peut être écrit ou modifié qu'en utilisant SoMachine Basic.
	S	Les mots système fournissent des informations d'état et de régulation concernant le contrôleur.
Format	W	Mot de 16 bits.
Identificateur d'instance d'objet	i	L'identificateur de l'objet représentant leur instance séquentielle en mémoire. Le nombre maximum d'objets dépend du nombre d'objets configurés aux limites de la mémoire disponible. Pour connaître la quantité maximale de mémoire disponible, consultez le <i>guide de programmation</i> de votre Logic Controller.

Format

Le contenu des mots ou des valeurs est stocké dans la mémoire utilisateur sous la forme d'un code binaire à 16 bits (complément de deux) selon la convention suivante :

	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0	Position du bit
$\pm$	16 384	8 192	4 096	2 048	1 024	512	256	128	64	32	16	8	4	2	1		Valeur du bit

Pour les notations binaires signées, le bit 15 est attribué, par convention, au signe de la valeur codée :

- Le bit 15 est mis à 0 : le contenu du mot est une valeur positive.
- Le bit 15 est mis à 1 : le contenu du mot est une valeur négative (les valeurs négatives sont exprimées en logique de complément de deux).

Les mots et les valeurs immédiates (consultez la section Liste des exceptions ([voir page 30](#)) pour les entiers non signés) peuvent être saisis ou récupérés au format suivant :

- Décimal
Min. : -32 768, Max. : 32 767 (1579, par exemple)
- Hexadécimal
Min. : 16#0000, Max. : 16#FFFF (par exemple, 16#A536)
Autre syntaxe : #A536
- Les règles du format ASCII sont les suivantes :
 - La fonction lit toujours l'octet de poids fort en premier.
 - Tout caractère ASCII en dehors de l'intervalle [0 - 9] ([16#30 - 16#39]) est considéré comme un caractère de fin, à l'exception du signe moins '-' (16#2D) lorsqu'il est placé comme premier caractère.
 - En cas de débordement (32 767 ou < -32 768), le bit système %S18 (erreur détectée ou débordement arithmétique) est mis à 1 et la valeur 32 767 ou -32 768 est renvoyée.
 - Si le premier caractère de l'opérande est un caractère de « fin », la valeur 0 est renvoyée et le bit %S18 est mis à 1.

Par exemple, "HELLO" :

- %MW0 :="HE"
- %MW1 :="LL"
- %MW2 :="O"

Liste des exceptions

Ce tableau répertorie les valeurs des objets qui sont des entiers non signés :

Objet	Valeur
%SW	0 à 65 535
%FC.V et %FC.P	0 à 65 535
%FC.VD et %FC.PD	0 à 4 294 967 295
%HSC.V, %HSC.P, %HSC.S0, %HSC.S1 et %HSC.C	0 à 65 535
%HSC.DV, %HSC.PD, %HSC.S0D, %HSC.S1D et %HSC.CD	0 à 4 294 967 295
%HSC.T	100 à 1 000
%PWM.P	0 à 32 767
%PWM.R	0 à 100
%PLS.P	0 à 32 767
%PLS.N	0 à 32 767
%PLS.ND	0 à 2 147 483 647

Les objets ne figurant pas dans cette liste des exceptions ont les plages de valeurs suivantes :

- Mots : -32 768 à 32 767
- Mots doubles : -2 147 483 648 à 2 147 483 647

Description

Le tableau suivant décrit les objets mot :

Mots	Description	Adresse ou valeur	Accès en écriture ⁽¹⁾
Valeurs immédiates	Il s'agit d'entiers dont le format est identique à celui des mots de 16 bits. Cela permet de leur attribuer des valeurs.	-	Non
	Base 10 (décimal)	-32 768 à 32 767	
	Base 16 (hexadécimal)	16#0000 à 16#FFFF	
Mémoire	Mots utilisés pour stocker des valeurs dans la mémoire de données au cours du fonctionnement.	%MWi	Oui
Constantes	Stockent des constantes ou des messages alphanumériques. Leur contenu ne peut être écrit ou modifié que par voie logicielle lors de la configuration.	%KWi	Non
Système	Ces mots de 16 bits comportent plusieurs fonctions : • Ils permettent l'accès aux données provenant directement du contrôleur en lisant les mots %SWi. • Ils effectuent des opérations sur l'application (ajustement des objets Schedule Blocks, par exemple).	%SWi	Dépend de i.
Blocs fonction	Ces mots correspondent aux paramètres ou aux valeurs courantes des blocs fonction.	%TM2.P, %Ci.P, etc.	Oui
(1) Ecrit par le programme ou à l'aide d'une table d'animation.			

Le nombre maximum d'objets disponibles est déterminé par le Logic Controller. Pour plus d'informations sur le nombre maximum d'objets, consultez le *guide de programmation* de votre Logic Controller.

Exemple

Ce tableau présente quelques exemples d'adressage d'objets mot :

Objet mot	Description
%MW15	Mot mémoire numéro 15
%KW26	Mot constant numéro 26
%SW30	Mot système numéro 30

Extraction d'un bit d'un objet mot

Ce tableau explique comment extraire un des 16 bits des objets mot suivants :

Objet mot	Adresse ou valeur	Accès en écriture ⁽¹⁾
Mémoire	%MWi:Xk	Oui
Système	%SWi:Xk	Dépend de i.
Constant	%KWi:Xk	Non
Valeur de l'entrée	%IWY.z:Xk ⁽²⁾	Non
Valeur de la sortie	%QWY.z:Xk ⁽²⁾	Oui
Etat de l'entrée	%IWSY.z:Xk ⁽²⁾	Non
Etat de la sortie	%QWSY.z:Xk ⁽²⁾	Oui
(1) Ecrit par le programme ou à l'aide d'une table d'animation. (2) Pour plus d'informations sur les objets mot d'E/S, consultez la section Adressage d'objets d'E/S (voir page 25). Xk Indique le numéro du bit à extraire de l'objet mot. Par exemple, %MW0.X3 (le bit extrait est stocké à la troisième position séquentielle du mot mémoire %MW0.).		

Objets mot double et flottants

Introduction

Un objet flottant est un nombre réel, c'est-à-dire un nombre avec une partie fractionnelle (par exemple : 3,4E+38, 2,3 ou 1,0).

Un mot double est constitué de 4 octets stockés dans la mémoire des données et dont la valeur (complément de deux) est comprise entre -2 147 483 648 et +2 147 483 647.

Tous les Logic Controller ne prennent pas en charge les opérations sur les objets flottants et les objets mot double.

Pour plus d'informations sur la compatibilité, consultez le *Guide de programmation* de votre Logic Controller.

Format et valeur du mot flottant

Le format flottant utilisé est celui de la norme IEEE 734-1985 (équivalente à la norme CEI 559). La longueur des mots est de 32 bits, ce qui correspond à des nombres à virgule flottante à une décimale.

Le tableau ci-dessous décrit le format d'une valeur flottante :

Bit 31	Bits {30 à 23}	Bits {22 à 0}
Signe de l'exposant	Exposant	Mantisse

La précision de représentation des nombres à virgule flottante est comprise entre 2 et 24. Il est inutile d'afficher plus de 6 chiffres après la virgule.

NOTE : La valeur "1285" est interprétée comme une valeur entière. Pour qu'elle soit prise en compte comme une valeur à virgule flottante, elle doit être écrite comme suit : 1285,0.

Plage limite des fonctions arithmétiques sur des objets flottants

Le tableau suivant décrit la plage limite des fonctions arithmétiques sur des objets flottants :

Fonction arithmétique		Plage limite et opérations invalides	
Type	Syntaxe	#QNAN (non valide)	#INF (infini)
Racine carrée d'un opérande	SQRT (x)	x < 0	x > 1,7E38
Puissance d'un entier par un réel EXPT (%MF, %MW)	EXPT (y, x) (où : $x^y = \%MW^{\%MF}$)	x < 0	y.ln(x) > 88
Logarithme en base 10	LOG (x)	x <= 0	x > 2,4E38
Logarithme népérien	LN (x)	x <= 0	x > 1,65E38
Exponentiel naturel	EXP (x)	x < 0	x > 88,0

Vérification de la validité

Le bit système %S18 est mis à 1 lorsque le résultat est hors de la plage valide.

Le mot d'état %SW17 indique la cause d'une erreur détectée dans une opération flottante.

Différents bits du mot %SW17 :

%SW17:X0	Opération incorrecte, le résultat n'est pas un nombre (1.#NAN ou -1.#NAN).
%SW17:X1	Réservé
%SW17:X2	Division par 0. Le résultat est l'infini (-1.#INF ou 1.#INF).
%SW17:X3	Résultat supérieur en valeur absolue à +3,402824e+38 : le résultat est l'infini (-1.#INF ou 1.#INF)
%SW17:X4 to X15	Réservé

Ce mot est remis à 0 par le système après un démarrage à froid et par le programme à des fins de réutilisation.

Syntaxe

L'adressage des objets flottants mémoire et constants doit respecter la syntaxe suivante :

%	M ou K	F	i
Symbole	Type d'objet	Format	Identificateur d'instance d'objet

L'adressage des objets mot double mémoire et constant doit respecter la syntaxe suivante :

%	M ou K	D	i
Symbole	Type d'objet	Format	Identificateur d'instance d'objet

Le tableau suivant décrit les éléments de la syntaxe d'adressage :

Groupe	Elément	Description
Symbole	%	Une adresse interne doit toujours débuter par le symbole de pourcentage (%).
Type d'objet	M	Les objets mémoire permettent de stocker des valeurs intermédiaires pendant l'exécution d'un programme.
	K	Les mots constants permettent de stocker des valeurs constantes ou des messages alphanumériques (uniquement pour les mots doubles).
Format	F	Objet flottant 32 bits.
	D	Objet mot double mot 32 bits.
Identificateur d'instance d'objet	i	L'identificateur représentant l'instance (position séquentielle) d'un objet en mémoire. Pour plus d'informations sur le nombre maximum d'objets, consultez le <i>guide de programmation</i> de votre Logic Controller.

Description des objets flottants et mots doubles

Le tableau suivant décrit les objets flottants et mot double :

Type d'objet	Description	Adresse	Accès en écriture
Valeurs immédiates	Entiers (mot double) ou décimaux (virgule flottante) dont le format est identique aux objets 32 bits.	-	Non
Flottant mémoire	Objets utilisés pour stocker des valeurs dans la mémoire des données lorsque le système est en cours d'exécution.	%MF <i>i</i>	Oui
Mot double mémoire		%MD <i>i</i>	Oui
Valeur constante flottante	Permet de stocker les constantes.	%KF <i>i</i>	Oui (pas par le programme)
Constante double		%KD <i>i</i>	Oui (pas par le programme)

NOTE : Le nombre maximum d'objets est déterminé par le Logic Controller. Pour plus d'informations, consultez le Guide de programmation de votre plate-forme matérielle.

Exemple

Le tableau suivant montre des exemples d'adressage d'objets flottants et mot double :

Objet	Description
%MF15	Objet flottant mémoire numéro 15
%KF26	Objet flottant constant numéro 26
%MD15	Mot double mémoire numéro 15
%KD26	Mot double constant numéro 26

Possibilité de recouvrement entre des objets

Les mots de longueur simple, de longueur double et flottants sont stockés dans une zone mémoire de l'espace des données. Ainsi, le mot flottant %MF_i et le mot double %MD_i correspondent aux mots de longueur simple %MW_i et %MW_{i+1} (le mot %MW_i contenant les bits de poids faible et le mot %MW_{i+1} les bits de poids fort du mot %MF_i).

Ce tableau illustre le recouvrement des mots mémoire flottants et doubles :

Objets flottants et doubles	Adresse impaire	Mots mémoire
%MF0 / %MD0		%MW0
	%MF1 / %MD1	%MW1
%MF2 / %MD2		%MW2
	%MF3 / %MD3	%MW3
%MF4 / %MD4		%MW4
	à	%MW5
à		à
	%MF _i / %MD _i	%MW _i
%MF _{i+1} / %MD _{i+1}		%MW _{i+1}

Le tableau suivant illustre le recouvrement des objets constants flottants et doubles :

Objets flottants et doubles	Adresse impaire	Mots mémoire
%KF0 / %KD0		%KW0
	%KF1 / %KD1	%KW1
%KF2 / %KD2		%KW2
	%KF3 / %KD3	%KW3
%KF4 / %KD4		%KW4
	à	%KW5
à		...
	%KF _i / %KD _i	%KW _i
%KF _{i+1} / %KD _{i+1}		%KW _{i+1}

Exemple :

%MF0 correspond à %MW0 et %MW1. %KF543 correspond à %KW543 et %KW544.

<Objets structurés

Introduction

Les objets structurés sont des ensembles formés par des objets adjacents. SoMachine Basic prend en charge les types d'objet structuré suivants :

- Chaînes de bits
- Tables de mots
- Tables de mots doubles
- Tables de mots flottants

Chaînes de bits

Les chaînes de bits sont composées d'une série de bits d'objets adjacents, de même type et de longueur (L) définie. Elles sont référencées par les limites des octets.

Exemple : chaîne de bits %M8 : 6

%M8	%M9	%M10	%M11	%M12	%M13

NOTE : %M8 : 6 est valide (car 8 est un multiple de 8), alors que %M10 : 16 ne l'est pas (10 n'est pas un multiple de 8).

Les chaînes de bits peuvent être utilisées avec l'instruction d'affectation (*voir page 52*).

Types de bit disponibles

Types de bit disponibles pour les chaînes de bits :

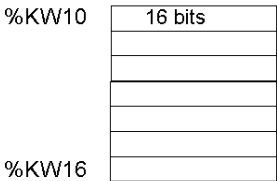
Type	Adresse	Accès en écriture
Bits d'entrée numérique	%I0.0:L ou %I1.0:L ⁽¹⁾	Non
Bits de sortie numérique	%Q0.0:L ou %Q1.0:L ⁽¹⁾	Oui
Bits système	%Si:L "i" étant un multiple de 8	En fonction de i
Bits d'étape Grafcet	%Xi:L "i" étant un multiple de 8	Oui (via le programme)
Bits mémoire	%Mi:L "i" étant un multiple de 8	Oui
(1) Seuls les bits d'E/S de 0 à 16 peuvent être lus dans une chaîne de bits. Pour les Logic Controller avec 24 ou 32 voies d'E/S, les bits supérieurs à 16 sont illisibles dans une chaîne de bits. L Représente la longueur des objets structurés (chaînes de bits, table de mots, table de mots doubles et table de mots flottants).		

Le nombre de bits est déterminé par le Logic Controller. Pour plus d'informations, consultez le Guide de programmation de votre plate-forme matérielle.

Tables de mots

Les tables de mots sont composées d'une série de mots adjacents de même type et de longueur (L) définie (255 au maximum).

Exemple : table de mots %KW10 : 7



Les tables de mots peuvent être utilisées avec l'instruction d'affectation (*voir page 52*).

Types de mot disponibles

Types de mot disponibles pour les tables de mots :

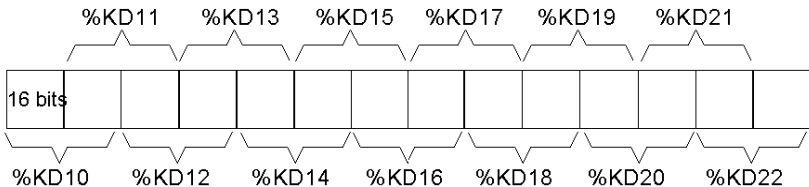
Type	Adresse	Accès en écriture
Mots mémoire	%MWi:L	Oui
Mots constants	%KWi:L	Non
Mots système	%SWi:L	En fonction de i

Le nombre de mots est déterminé par le Logic Controller. Pour plus d'informations, consultez le Guide de programmation de votre plate-forme matérielle.

Tables de mots doubles

Les tables de mots doubles sont composées d'une série de mots adjacents de même type et de longueur (L) définie (255 au maximum).

Exemple : table de mots doubles %KD10 : 7



Les tables de mots doubles peuvent être utilisées avec l'instruction d'affectation (*voir page 52*).

Types de mot double disponibles

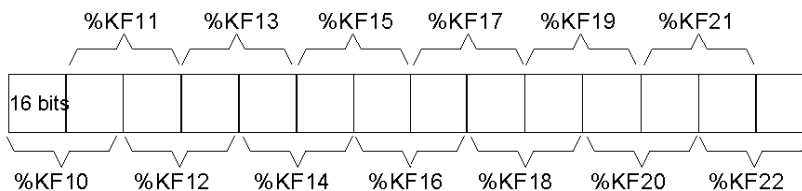
Types de mot disponibles pour les tables de mots doubles :

Type	Adresse	Accès en écriture
Mots mémoire	%MDi:L	Oui
Mots constants	%KDi:L	Non

Tables de mots flottants

Les tables de mots flottants sont composées d'une série de mots adjacents de même type et de longueur (L) définie (255 au maximum).

Exemple : table de mots flottants %KF10:7



Les tables de mots flottants peuvent être utilisées avec l'instruction d'affectation (*voir page 52*).

Types de mot flottant disponibles

Types de mot disponibles pour les tables de mots flottants :

Type	Adresse	Accès en écriture
Mots mémoire	%MFi:L	Oui
Mots constants	%KFi:L	Non

Objets indexés

Introduction

Un objet indexé est un mot simple, un mot double ou un objet à virgule flottante, dont l'adresse est indexée. Il existe deux types d'adressage d'objet :

- Adressage direct
- Adressage indexé

Adressage direct

L'adresse directe d'un objet est définie lors de l'écriture du programme.

Exemple : `%M26` est un bit mémoire dont l'adresse directe est 26.

Adressage indexé

L'adresse indexée d'un objet permet de modifier l'adresse de cet objet en lui ajoutant un index. Le contenu de l'index est ajouté à l'adresse directe de l'objet. Cet index est défini par un mot mémoire `%MWi`.

Exemple : `%MW108 [%MW2]` est un mot dont l'adresse est composée de l'adresse directe 108 et du contenu du mot `%MW2`.

Si la valeur du mot `%MW2` est 12, écrire des données dans `%MW108 [%MW2]` équivaut à écrire dans `%MW120` (108 + 12).

Objets disponibles pour l'adressage indexé

Ce tableau décrit les types d'objets disponibles pour l'adressage indexé :

Type	Adresse	Accès en écriture
Mots mémoire	<code>%MWi [MWj]</code>	Oui
Mots constants	<code>%KW i [%MWj]</code>	Non
Mots mémoire doubles	<code>%MDi [MWj]</code>	Oui
Mots doubles constants	<code>%KD i [%MWj]</code>	Non
Mots mémoire flottants	<code>%MFi [MWj]</code>	Oui
Mots flottants constants	<code>%KF i [%MWj]</code>	Non
i Identificateur représentant l'instance (position séquentielle) d'un objet dans la mémoire. Pour plus d'informations sur le nombre maximum d'objets, consultez le guide de programmation de votre Logic Controller. j Identificateur de l'objet d'index dont le contenu doit être ajouté à l'adresse directe d'un autre objet.		

Les objets indexés sont utilisables avec l'instruction d'affectation ([voir page 66](#)) et dans les instructions de comparaison ([voir page 62](#)).

Ce type d'adressage permet de scruter un ensemble d'objets de même type (tels que des mots mémoire ou constants) successivement, en modifiant le contenu de l'objet indexé dans le programme.

Bit système de débordement d'index %S20

Un débordement d'index se produit lorsque l'adresse d'un objet indexé dépasse les limites de la zone mémoire contenant ce même type d'objet. Pour résumer :

- L'adresse de l'objet plus le contenu de l'index est inférieur à 0.
- L'adresse de l'objet plus le contenu de l'index est supérieur au plus grand mot directement référencé dans l'application.

En cas de débordement d'index, le bit système %S20 est mis à 1 et l'objet reçoit une valeur d'index égale à 0.

NOTE : La surveillance des débordements vous incombe. Votre programme doit lire %S20 pour identifier le traitement possible. Vous devez ensuite confirmer qu'il est remis à 0.

%S20 (état initial = 0) :

- En cas de débordement de l'index : mise à 1 par le contrôleur.
- Acquiescement de débordement : mise manuelle à 0 dans le programme après modification de l'index.

AVERTISSEMENT

COMPORTEMENT IMPREVU DE L'EQUIPEMENT

- Ecrivez des instructions de programmation pour tester la validité des opérandes destinés à être utilisés dans des opérations mathématiques.
- Evitez d'utiliser des opérandes de différents types de données dans des opérations mathématiques.
- Surveillez toujours les bits système affectés pour indiquer des résultats mathématiques non valides.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Objets de bloc fonction

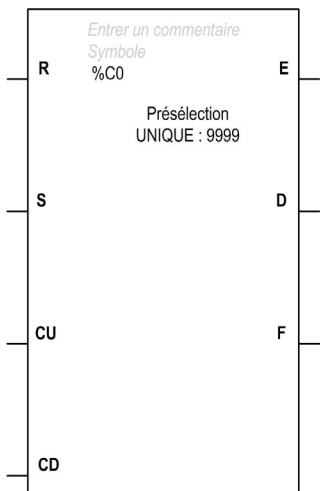
Introduction

Un bloc fonction est un objet réutilisable qui accepte une ou plusieurs valeurs d'entrée et renvoie une ou plusieurs valeurs de sortie. Il est toujours appelé par une instance (une copie d'un bloc fonction ayant son propre nom et ses propres variables). Chaque instance conserve son état (sorties et variables internes) entre deux appels.

NOTE : les blocs fonction (%FC, %HSC, %PLS et %PWM) et l'alarme d'état pilotent leurs entrées et sorties (%I0.x et %Q0.x affectés dans la configuration) directement et indépendamment du cycle du contrôleur. Les bits d'image (%I0.x et %Q0.x) ne sont pas mis à jour par le contrôleur. Donc, ces entrées et sorties ne sont pas exploitables directement dans le programme utilisateur, et une table d'animation utilisant ces E/S ne peut pas indiquer leur état.

Exemple

Cette figure montre un bloc fonction StepCounter :



Objets bit

Les objets bit correspondent aux sorties des blocs fonction. Les instructions booléennes de test peuvent accéder à ces bits :

- directement (par exemple, LD E) s'ils sont câblés au bloc en programmation (*voir page 135*) réversible ;
- en spécifiant le type de bloc (LD %Ci.E, par exemple).

Les entrées sont accessibles sous la forme d'instructions.

Objets mot

Les objets mot correspondent à des paramètres et des valeurs :

- Paramètres de configuration de bloc : le programme peut accéder à certains paramètres (paramètres de présélection, par exemple), mais pas à d'autres (base de temps, par exemple).
- Valeurs courantes : par exemple, `%Ci.V`, la valeur de comptage actuelle.

Objets mot double

Les objets mot double augmentent la capacité de calcul de votre Logic Controller lors de l'exécution de fonctions système, telles que les compteurs `%FC` et `%HSC`, ainsi que les générateurs d'impulsions (`%PLS`, `%PWM`).

Pour adresser les objets mot double 32 bits utilisés avec les blocs fonction, il suffit d'ajouter le caractère D à la syntaxe d'origine des objets mot standard.

L'exemple suivant indique comment adresser la valeur courante d'un compteur rapide au format standard et au format mot double :

- `%FCi.V` est la valeur courante du compteur rapide au format standard.
- `%FCi.VD` est la valeur courante du compteur rapide au format mot double.

Chapitre 3

Procédure

Contenu de ce chapitre

Ce chapitre contient les sous-chapitres suivants :

Sous-chapitre	Sujet	Page
3.1	Traitement booléen	46
3.2	Traitement numérique	64
3.3	Programme	82
3.4	Virgule flottante	89
3.5	ASCII	98
3.6	Opérateurs de pile	109
3.7	Instructions sur tables d'objets	111

Sous-chapitre 3.1

Traitement booléen

Objet de cette section

Cette section présente les instructions de traitement booléen.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Instructions booléennes	47
Opérateurs de chargement (LD, LDN, LDR, LDF)	50
Opérateurs d'affectation (ST, STN, R, S)	52
Opérateurs AND logiques (AND, ANDN, ANDR, ANDF)	54
Opérateurs OR logiques (OR, ORN, ORR, ORF)	56
Opérateurs OR exclusifs (XOR, XORN, XORR, XORF)	58
Opérateur NOT (N)	61
Instructions de comparaison	62

Instructions booléennes

Introduction

Les instructions booléennes s'apparentent aux éléments du langage Schéma à contacts. Elles sont résumées dans le tableau suivant :

Élément	Opérateur	Exemple d'instruction	Description
Éléments de test	L'instruction Load (LD) équivaut au premier contact ouvert connecté à un rail d'alimentation d'un schéma à contacts. Les instructions AND et OR logiques équivalent à des contacts ouverts après le premier contact connecté au rail d'alimentation d'un schéma à contacts.	LD %I0.0	Le contact est fermé lorsque le bit %I0.0 est à l'état 1.
Éléments d'action	L'instruction Store(ST) équivaut à une bobine.	ST %Q0.0	L'objet bit associé prend la valeur logique de l'accumulateur du bit (résultat de la logique précédente).

Le résultat booléen des éléments de test est appliqué aux éléments d'action, comme le montrent les instructions suivantes :

Réseau	Instruction
0	LD %I0.0 AND %I0.1 ST %Q0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Test des entrées du contrôleur

Des instructions de test booléennes peuvent être utilisées pour détecter des fronts montants ou descendants sur les entrées du contrôleur. Un front est détecté lorsque l'état d'une entrée est passé de la valeur "scrutation n-1" à la valeur "scrutation n". La détection de ce front reste effective pendant la scrutation courante.

Détection de fronts

Le tableau suivant résume les instructions et les chronogrammes de détection de fronts :

Front	Instruction IL	Schéma à contacts	Chronogramme
Front montant	LDR %I0.0		
Front descendant	LDF %I0.0		

NOTE : Les fronts montants et descendants ne s'appliquent qu'avec %I et %M.

Détection d'un front montant

L'instruction Load Rising Edge (LDR) équivaut à un contact de détection de front montant. Le front montant détecte le passage de l'entrée de 0 à 1.

Un contact de détection de transition positive permet de détecter un front montant, comme dans l'exemple suivant :

Réseau	Instruction
0	LDR %I0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Détection de front descendant

L'instruction Load Falling Edge (LDF) équivaut à un contact de détection de front descendant. Le front descendant détecte le passage de l'entrée de régulation de 1 à 0.

Un contact de détection de transition négative permet de détecter un front descendant, comme dans l'exemple suivant :

Réseau	Instruction
0	LDF %I0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Opérateurs de chargement (LD, LDN, LDR, LDF)

Introduction

Les opérateurs de chargement LD, LDN, LDR et LDF correspondent respectivement à des contacts ouvert, fermé, de front montant et de front descendant. Les instructions LDR et LDF ne sont utilisées qu'avec les entrées et les mots internes de Logic Controller.

Syntaxe

Ce tableau répertorie les types d'opérateur de chargement avec leurs équivalents en langage Schéma à contacts et les opérateurs autorisés :

Opérateurs	Equivalent en langage Schéma à contacts	Opérandes autorisés
LD		0/1 %I, %Q, %M, %S, %X, %BLK.x %IW:Xk, %QW:Xk, %IWS:Xk, %QWS:Xk, %MW :Xk, %SW:Xk, %KW:Xk
LDN		
LDR		%I, %M
LDF		

Exemples de codage

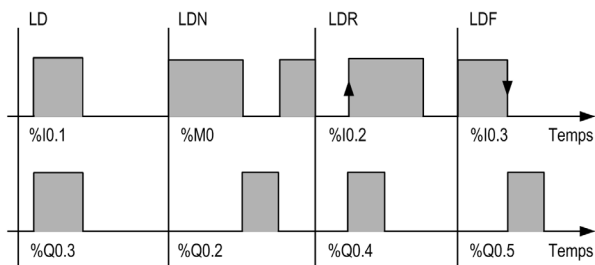
Exemples d'instructions de chargement :

Réseau	Instruction
0	LD %I0.1 ST %Q0.3
1	LDN %M0 ST %Q0.2
2	LDR %I0.1 ST %Q0.4
3	LDF %I0.3 ST %Q0.5

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Le schéma suivant montre le chronogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :



Opérateurs d'affectation (ST, STN, R, S)

Introduction

Les instructions d'affectation ST, STN, S et R correspondent respectivement aux bobines directe, inverse, d'enclenchement et de déclenchement.

Syntaxe

Ce tableau répertorie les types d'opérateur d'affectation avec leurs équivalents en langage Schéma à contacts et les opérateurs autorisés :

Opérateurs	Equivalent en langage Schéma à contacts	Opérandes autorisés
ST		%Q, %M, %S, %X, %BLK.x %QW:Xk, %MW:Xk, %SW:Xk ⁽¹⁾
STN		
S		
R		
(1) %SW:Xk est activé sur les mots système qui ne sont pas en lecture seule.		

Exemples de codage

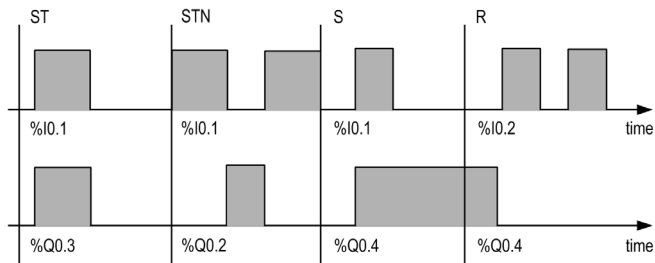
Exemples d'instructions d'affectation :

Réseau	Instruction
0	LD %I0.1 ST %Q0.3 STN %Q0.2 S %Q0.4
1	LD %I0.2 R %Q0.4

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Le schéma suivant montre le chronogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :



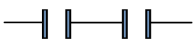
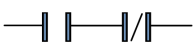
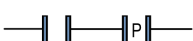
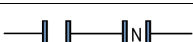
Opérateurs AND logiques (AND, ANDN, ANDR, ANDF)

Introduction

Les opérateurs AND effectuent une opération AND logique entre l'opérande (ou son inverse, son front montant ou son front descendant) et le résultat booléen de l'instruction précédente.

Syntaxe

Ce tableau répertorie les types d'opérateur AND, leur équivalent en langage Schéma à contacts et les opérandes autorisés :

Opérateurs	Equivalent en langage Schéma à contacts	Opérandes autorisés
AND		0/1 %I, %Q, %M, %S, %X, %BLK.x %IW:Xk, %QW:Xk, %IWS:Xk, %QWS:Xk, %MW:Xk, %SW:Xk, %KW:Xk
ANDN		
ANDR		%I, %M
ANDF		

Exemples de codage

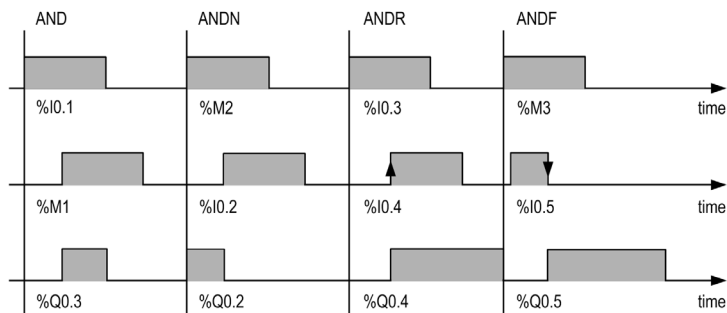
Exemples d'instructions AND logiques :

Réseau	Instruction	
0	LD	%I0.1
	AND	%M1
	ST	%Q0.3
1	LD	%M0
	ANDN	%I0.0
	ST	%Q0.2
2	LD	%I0.3
	ANDR	%I0.4
	S	%Q0.4
3	LD	%M3
	ANDF	%I0.5
	S	%Q0.5

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Le schéma suivant montre le chronogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :



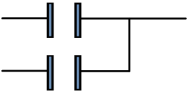
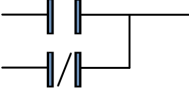
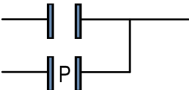
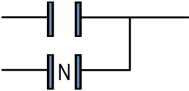
Opérateurs OR logiques (OR, ORN, ORR, ORF)

Introduction

Les opérateurs **OR** effectuent une opération **OR** logique entre l'opérande (ou son inverse, son front montant ou son front descendant) et le résultat booléen de l'instruction précédente.

Syntaxe

Ce tableau répertorie les types d'opérateur **OR**, leur équivalent en langage Schéma à contacts et les opérandes autorisés :

Opérateurs	Equivalent en langage Schéma à contacts	Opérandes autorisés
OR		0/1 %I, %Q, %M, %S, %X, %BLK.x %IW:Xk, %QW:Xk, %IWS:Xk, %QWS:Xk, %MW:Xk, %SW:Xk, %KW:Xk
ORN		
ORR		%I, %M
ORF		

Exemples de codage

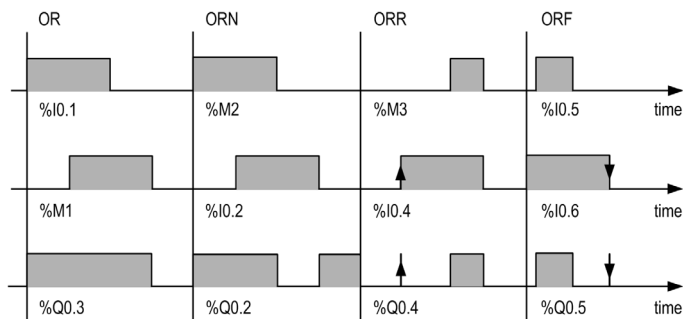
Exemples d'instructions OR logiques :

Réseau	Instruction		
0	LD	%I0.1	
	OR	%M1	
	ST	%Q0.0	
1	LD	%I0.2	
	ORN	%M2	
	ST	%Q0.1	
2	LD	%M0	
	ORR	%I0.3	
	S	%Q0.5	
3	LDF	%I0.5	
	ORF	%I0.6	
	S	%Q0.0	

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Chronogramme

Le schéma suivant montre le chronogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :



Opérateurs OR exclusifs (XOR, XORN, XORR, XORF)

Introduction

L'opérateur `XOR` exécute une opération OR exclusif entre l'opérande et le résultat booléen de l'instruction précédente.

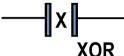
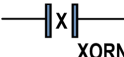
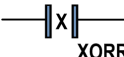
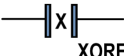
L'opérateur `XORN` exécute une opération OR exclusif entre l'inverse de l'opérande et le résultat booléen de l'instruction précédente.

L'opérateur `XORR` exécute une opération OR exclusif entre le front montant de l'opérande et le résultat booléen de l'instruction précédente.

L'opérateur `XORF` exécute une opération OR exclusif entre le front descendant de l'opérande et le résultat booléen de l'instruction précédente.

Syntaxe

Ce tableau répertorie les types d'opérateur `XOR` et les opérandes autorisés :

Opérateurs	Equivalent en langage Schéma à contacts	Opérandes autorisés
XOR	 XOR	%I, %Q, %M, %S, %X, %BLK.x %IW:Xk, %QW:Xk, %IWS:Xk, %QWS:Xk, %MW:Xk, %SW:Xk, %KW:Xk
XORN	 XORN	
XORR	 XORR	%I, %M
XORF	 XORF	

Exemples de codage

Utilisation de l'instruction XOR :

Réseau	Instruction
0	LD %I0.1 XOR %M1 ST %Q0.3

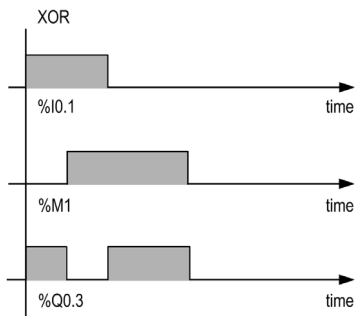
Instructions logiques équivalentes de l'opérateur XOR :

Réseau	Instruction
0	LD %I0.1 ANDN %M1 OR (%M1 ANDN %I0.1) ST %Q0.3

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Le schéma suivant montre le chronogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :

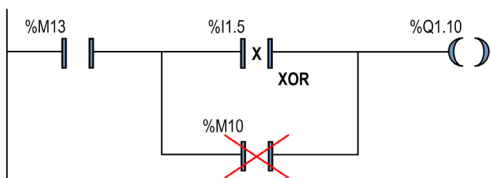


Cas particuliers

N'insérez pas de :

- contacts XOR dans la première position d'un réseau ;
- contacts XOR en parallèle avec d'autres éléments en langage Schéma à contacts (comme dans l'exemple suivant).

Comme l'illustre l'exemple suivant, l'insertion d'un élément en parallèle avec un contact XOR génère une erreur détectée de validation :



Opérateur NOT (N)

Introduction

L'opérateur NOT (N) a un opérande implicite (le résultat stocké dans l'accumulateur booléen). NOT rend négative la valeur de l'accumulateur.

Syntaxe

Ce tableau montre l'opérateur N :

Opérateur	Equivalent en langage Schéma à contacts	Opérandes autorisés
N		Sans objet.

Exemples de codage

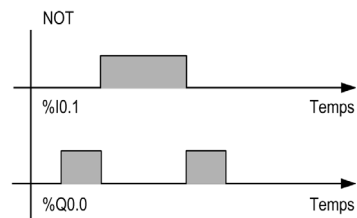
Exemple d'instruction NOT :

Réseau	Instruction
0	LD %I0.1 N ST %Q0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Chronogramme

Le schéma suivant montre le chonogramme, ainsi que l'effet sur la sortie, du code d'un ou des exemples fournis :



Instructions de comparaison

Introduction

Les opérateurs de comparaison permettent de comparer deux opérandes.

Ce tableau répertorie les différents types d'opérateurs de comparaison :

Opérateur	Fonction
>	Teste si Op1 est supérieur à Op2.
>=	Teste si Op1 est supérieur ou égal à Op2.
<	Teste si Op1 est inférieur à Op2.
<=	Teste si Op1 est inférieur ou égal à Op2.
=	Teste si Op1 est égal à Op2.
<>	Teste si Op1 est différent de Op2.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des expressions de comparaison en langage Liste d'instructions (*voir page 20*) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc comparaison**.

Syntaxe des instructions de comparaison :

Opérateur	Syntaxe
>, >=, <, <=, =, <>	LD [Op1 opérateur Op2] AND [Op1 opérateur Op2] OR [Op1 opérateur Op2]

Le tableau suivant détaille les opérandes :

Type	Op1	Op2
Mots	%MWi, %KWi, %IW, %QWi, %SWi, %BLK.x	Valeur immédiate, %MWi, %KWi, %IW, %QW, %IWSi, %QWSi, %SWi, %BLK.x, %MWi[%MWi], %KWi[%MWi]
Mots doubles	%MDi, %KDi	Valeur immédiate, %MDi, %KDi, %MDi[%MWi], %KD[%MWi]
Mots flottants	%MFi, %KFi	Valeur flottante immédiate, %MFi, %KFi, %MFi[%MWi], %KFi[%MWi]

NOTE : Les instructions de comparaison peuvent apparaître entre parenthèses.

Exemples de codage

La comparaison s'effectue entre les crochets qui suivent les instructions LD, AND et OR. Le résultat est 1 lorsque la comparaison demandée est vraie.

Exemples d'instructions de comparaison :

Réseau	Instruction
0	LD %I0.2 AND [%MW10>100] ST %Q0.3
1	LD %M0 AND [%MW20<%KW35] ST %Q0.4
2	LD %I0.2 OR [%MF30>=%MF40] ST %Q0.5

Exemple d'utilisation d'une instruction de comparaison entre parenthèses :

Réseau	Instruction
0	LD %M0 AND ([%MF20>10.0] OR %I0.0) ST %Q0.1

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Sous-chapitre 3.2

Traitement numérique

Objet de cette section

Cette section présente les instructions de traitement numérique.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Introduction aux instructions numériques	65
Instructions d'affectation	66
Affectation de chaînes de bits	67
Affectation de mots	69
Opérateurs arithmétiques sur des entrées	71
Instructions logiques	75
Instructions de décalage	77
Instruction de conversion BCD/binaire	79
Instructions de conversion de mot simple/double	81

Introduction aux instructions numériques

Présentation

En général, les instructions numériques s'appliquent aux mots de 16 bits et aux mots doubles de 32 bits. Elles apparaissent entre crochets. Si le résultat de l'opération logique précédente est vrai (accumulateur booléen = 1), l'instruction numérique est exécutée. S'il est faux (accumulateur booléen = 0), l'instruction numérique n'est pas exécutée et l'opérande reste inchangé.

Instructions d'affectation

Introduction

Les instructions d'affectation permettent de charger l'opérande 2 (Op2) dans l'opérande 1 (Op1).

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions d'affectation :

Opérateur	Syntaxe
:=	[Op1: = Op2] Op1 prend la valeur de Op2.

Les opérations d'affectation peuvent être exécutées sur :

- des chaînes de bits ;
- des mots ;
- des mots doubles ;
- un mot flottant ;
- des tables de mots ;
- des tables de mots doubles ;
- des tables de mots flottants.

Affectation de chaînes de bits

Introduction

Les opérations sont autorisées sur les chaînes de bits suivantes :

- Chaîne de bits à chaîne de bits (exemple 1)
- Chaîne de bits à mot (exemple 2) ou mot double (indexé)
- Mot ou mot double (indexé) à chaîne de bits (exemple 3)
- Valeur immédiate à chaîne de bits

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des affectations de chaînes de bits :

Opérateur	Syntaxe
:=	[Op1: = Op2] Op1 prend la valeur de Op2.

Le tableau suivant détaille les opérandes :

Type	Op1	Op2
Mot, mot double	%MWi, %QWi, %SWi %MWi[%MWi], %MDi, %MDi[%MWi] %Mi:L, %Qi:L, %Si:L, %Xi:L %Tmi.P, %Ci.P, %Ri.I, %Ri.O, %F Ci.P, %PLSi.P, %PwMi.P %Ci.PD, %FCi.PD	Valeur immédiate, %MWi, %KWi, %IW, %QWi, %IWSi, %QW Si, %SWi, %BLK.x, %MWi[%MWi], %KWi[%MWi], %MDi[%MWi], %KDi[%MWi], %Mi:L, %Qi:L, %Si:L, %Xi:L, %Ii:L %Tmi.P, %Ci.P, %Ri.I, %Ri.O, %FCi.P, %PLSi.P, %PwMi.P %Ci.PD, %FCi.PD

NOTE : L'abréviation %BLK.x (%C0.P, par exemple) est utilisée pour désigner n'importe quel mot de bloc fonction.

Structure

Exemples d'affectations de chaînes de bits :

Réseau	Instruction
0	LD 1 [%Q0.0:8:=%M64:8]
1	LD %I0.2 [%MW100:=%M0:16]
2	LDR %I0.3 [%MW104:16:=%KW0]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir [page 16](#)).

Règles d'utilisation :

- Affectation d'une chaîne de bits à un mot : les bits de la chaîne sont transférés au mot en commençant par la droite (premier bit de la chaîne vers le bit 0 du mot). Les bits de mot non concernés par le transfert (longueur ≤ 16) sont mis à 0.
- Affectation d'un mot à une chaîne de bits : les bits du mot sont transférés en partant de la droite (du bit 0 du mot vers le premier bit de la chaîne).

Affectation de mots

Introduction

Les opérations d'affectation peuvent être exécutées sur les mots et mots doubles suivants :

- Mot (indexé) à mot (exemple 2) (indexé ou non)
- Mot double (indexé) à mot double (indexé ou non)
- Valeur entière immédiate à mot (exemple 3) ou mot double (indexé ou non)
- Chaîne de bits à mot ou mot double
- Valeur flottante (indexée ou non) à valeur flottante (indexée ou non)
- Mot ou mot double à chaîne de bits
- Valeur flottante immédiate à valeur flottante (indexée ou non)

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des affectations de mots :

Opérateur	Syntaxe
:=	[Op1: = Op2] Op1 prend la valeur de Op2.

Le tableau suivant détaille les opérandes :

Type	Op1	Op2
Mot, mot double, chaîne de bits	%BLK.x, %MWi, %QWi, %SWi %MWi[MWi], %MDi, %MDi[%MWj], %Mi:L, %Qi:L, %Si:L, %Xi:L	Valeur immédiate, %MWi, %KWi, %IW, %QWi, %IWSi, QWSi, %SWi, %MWi[MWi], %KWi[MWi], %MDi, %MDi[%MWj], %KDi, %KDi[%MWj], %Mi:L, %Qi:L, %Si:L, %Xi:L, %Ii:L
Valeur flottante	%MFi, %MFi[%MWj]	Valeur flottante immédiate, %MFi, %MFi[%MWj], %KFi, %KFi[%MWj]

NOTE : L'abréviation %BLK.x (R3.I, par exemple) permet de désigner n'importe quel mot de bloc fonction. Pour les chaînes de bits %Mi:L, %Si:L et %Xi:L, l'adresse de base du premier bit de la chaîne doit être un multiple de 8 (0, 8, 16, ..., 96, etc.).

Structure

Exemples d'affectations de mots :

Réseau	Instruction
0	LD 1 [%SW112:=%MW100]
1	LD %I0.2 [%MW0 [%MW10] :=%KW0 [%MW20]]
2	LD %I0.3 [%MW10:=100]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Opérateurs arithmétiques sur des entrées

Introduction

Les opérateurs arithmétiques permettent d'effectuer des opérations arithmétiques entre deux opérandes d'entiers ou sur un opérande d'entier.

Ce tableau répertorie les différents types d'opérateurs arithmétiques :

Opérateur	Fonction
+	Addition de 2 opérandes
-	Soustraction de 2 opérandes
*	Multiplication de 2 opérandes
/	Division de 2 opérandes
REM	Reste de la division de 2 opérandes
SQRT	Racine carrée d'un opérande
INC	Incrémentation d'un opérande
DEC	Décrémentation d'un opérande
ABS	Valeur absolue d'un opérande

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions arithmétiques :

Opérateur	Syntaxe
+, -, *, / , REM	[Op1: = Op2 opérateur Op3]
INC, DEC	[opérateur Op1]
SQRT (1)	[Op1: = SQRT(Op2)]
ABS (1)	[Op1: = ABS(Op2)]

Le tableau suivant détaille les opérandes :

Type	Op1	Op2 et Op3 ⁽¹⁾
Mots	%MWi, %QWi, %SWi, %BLK.x ⁽²⁾	Valeur immédiate, %MWi, %KWi, %IWi, %QWi, %IWSi, %QWSi, %SWi, %BLK.x ⁽²⁾
Mots doubles	%MDi, %BLK.x	Immediate value, %MDi, %KDi, %BLK.x ⁽²⁾
(1) Avec cet opérateur, Op2 ne peut pas être une valeur immédiate. La fonction ABS n'est utilisable qu'avec des mots doubles (%MD et %KD) et flottants (%MF et %KF). Par conséquent, Op1 et Op2 doivent être des mots doubles ou flottants.		
(2) %BLK.x représente tous les objets de bloc.		

Structure

Exemples d'instructions arithmétiques :

Réseau	Instruction
0	LD %M0 [%MW0:=%MW10+10]
1	LD %I0.2 [%MW0:=SQRT(%MW10)]
2	LDR %I0.3 [%MW10:=32767]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Cas particuliers

Addition

- Débordement pendant l'opération sur mots
Si le résultat dépasse la capacité du mot de résultat, le bit %S18 (débordement) est mis à 1 et le résultat n'est pas significatif (voir le réseau 1 de l'exemple d'application (voir page 74)). Le programme utilisateur gère le bit %S18.

NOTE : Pour les mots doubles, les limites sont -2 147 483 648 et 2 147 483 647.

Multiplication

- Débordement pendant l'opération
Si le résultat dépasse la capacité du mot de résultat, le bit %S18 (débordement) est mis à 1 et le résultat n'est pas significatif.

Division / reste

- Division par 0
Si le diviseur est 0, la division est impossible et le bit système %S18 est mis à 1. Le résultat est alors incorrect.
- Débordement pendant l'opération
Si le quotient de la division dépasse la capacité du mot de résultat, le bit système %S18 est mis à 1.

Calcul de la racine carrée

- Débordement pendant l'opération
Le calcul de la racine carrée est uniquement effectué sur les valeurs positives. Le résultat est, par conséquent, toujours positif. Si l'opérande de racine carrée est négatif, le bit système %S18 est mis à 1 et le résultat est incorrect.

Certaines erreurs mathématiques détectées peuvent avoir un impact important sur l'exécution de l'application. Il vous incombe de surveiller ces erreurs potentielles et de programmer des instructions pour contrôler efficacement l'exécution de l'application en cas de détection d'une ou de plusieurs de ces erreurs. L'impact dépend de la configuration, des équipements utilisés et des instructions de programme exécutées avant et après la détection de ces erreurs.

AVERTISSEMENT

COMPOREMENT IMPREVU DE L'EQUIPEMENT

- Ecrivez des instructions de programmation pour tester la validité des opérandes destinés à être utilisés dans des opérations mathématiques.
- Evitez d'utiliser des opérandes de différents types de données dans des opérations mathématiques.
- Surveillez toujours les bits système affectés pour indiquer des résultats mathématiques non valides.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

NOTE : Le programme utilisateur gère les bits système %S17 et %S18. Le contrôleur les met à 1. Ils doivent être remis à 0 par le programme pour être réutilisés (voir l'exemple, page précédente).

Exemple d'application

Débordement pendant l'addition :

Réseau	Instruction
0	LD %M0 [%MW0 := %MW1 + %MW2]
1	LDN %S18 [%MW10 := %MW0]
2	LD %S18 [%MW10 := 32767]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Si %MW1 = 23 241 et %MW2 = 21 853, le résultat (45 094) ne peut pas être exprimé dans un mot de 16 bits signé. Par conséquent, le bit %S18 est mis à 1 et la valeur de %MW0 (-20 442) est incorrecte. Dans cet exemple, la valeur de ce mot est fixée à 32 767 lorsque le résultat est supérieur à cette valeur.

Instructions logiques

Introduction

Les opérateurs logiques permettent d'effectuer une opération logique entre deux opérandes de mot ou, dans le cas d'un opérateur **NOT** logique, sur un opérande de mot.

Le tableau suivant répertorie les différents types d'instructions logiques :

Instruction	Fonction
AND	AND (bit à bit) entre deux opérandes
OR	OR logique (bit à bit) entre deux opérandes
XOR	OR exclusif (bit à bit) entre deux opérandes
NOT	Complément logique (bit à bit) d'un opérande

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions logiques :

Opérateur	Syntaxe	Op1	Op2 et Op3
AND, OR, XOR	[Op1: = Op2 opérateur Op3]	%MWi, %QWi, %SWi, %BLK.x	Valeur immédiate (1), %MWi, %KWi, %IWi, %QWi, %IWSi, %QWSi, %SWi, %BLK.x
NOT	[Op1:=NOT(Op2)]		
(1) Avec NOT, Op2 ne peut pas être une valeur immédiate.			

Structure

Exemples d'instructions logiques :

Réseau	Instruction
0	LD %M0 [%MW0:=%MW10 AND 16#00FF]
1	LD 1 [%MW0:=%KW5 OR %MW10]
2	LD %I0.3 [%MW102:=NOT(%MW100)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Exemple d'application

Instruction AND logique :

```
[%MW15 := %MW32 AND %MW12]
```

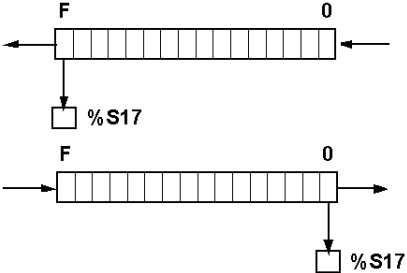
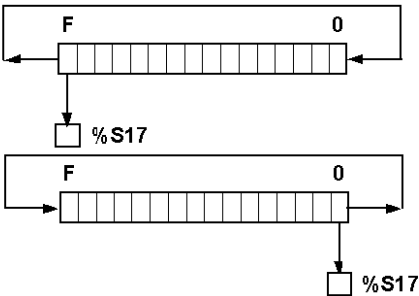
Lorsque %MW32 = 0001 1011 (binaire) (27 (décimal)) et %MW12 = 0011 0110 (binaire) (54 (décimal)), le résultat est %MW15 = 0001 0010 (binaire) (18 (décimal)).

Instructions de décalage

Introduction

Les instructions de décalage déplacent les bits d'un opérande d'un nombre spécifié de positions vers la droite ou vers la gauche.

Le tableau suivant répertorie les différents types d'instructions de décalage :

Instruction	Fonction	
Décalage logique		
SHL (op2, n)	Décalage logique de n positions vers la gauche.	
SHR (op2, n)	Décalage logique de n positions vers la droite.	
Décalage circulaire		
ROL (op2, n)	Décalage circulaire de n positions vers la gauche.	
ROR (op2, n)	Décalage circulaire de n positions vers la droite.	
n Valeur immédiate entière de : ● mot : 1 à 16 compris, ● mot double : 1 à 32 compris.		

NOTE : le bit système %S17 indique la valeur du dernier bit éjecté.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions (*voir page 19*) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de décalage :

Opérateur	Syntaxe
SHL, SHR	[Op1: = opérateur (Op2,n)]
ROL, ROR	
n Valeur immédiate entière de : ● mot : 1 à 16 compris, ● mot double : 1 à 32 compris.	

Le tableau suivant détaille les opérandes :

Types	Op1	Op2
Mots	%MWi, %QWi, %SWi %BLK.x	%MWi, %KWi, %IW, %QWi, %IWSi, %QWSi, %SWi, %BLK.x
Mots doubles	%MDi %BLK.x	%MDi, %KDi %BLK.x

Structure

Exemples d'instructions de décalage :

Réseau	Instruction
0	LDR %I0.1 [%MW0:=SHL(%MW10,5)]
1	LDR %I0.2 [%MW10:=ROR(%KW9,8)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Instruction de conversion BCD/binaire

Introduction

Les instructions de conversion permettent d'effectuer la conversion entre les différentes représentations numériques.

Ce tableau répertorie les différents types d'instruction de conversion BCD/binaire :

Instruction	Fonction
BTI	Conversion BCD/binaire
ITB	Conversion binaire/BCD

Analyse du code BCD

Le codage BCD (Binary Coded Decimal - décimal codé binaire) représente les décimaux (entre 0 et 9) par un code à quatre bits binaires. Un objet mot de 16 bits peut ainsi contenir un nombre à quatre chiffres (0000 à 9999) ; et un objet mot double de 32 bits, un nombre à huit chiffres.

Lors d'une conversion, le bit système %S18 est mis à 1 si la valeur n'est pas BCD. Ce bit doit être testé et remis à 0 par le programme.

Représentation BCD des décimaux :

Décimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Exemples :

- Le mot %MW5 exprime la valeur BCD "2450" qui correspond à la valeur binaire 0010 0100 0101 0000.
- Le mot %MW12 exprime la valeur décimale "2450" qui correspond à la valeur binaire 0000 1001 1001 0010.

Le mot %MW5 est converti en mot %MW12 à l'aide de l'instruction BTI.

Le mot %MW12 est converti en mot %MW5 à l'aide de l'instruction ITB.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions (*voir page 19*) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de conversion BCD/binaire :

Opérateur	Syntaxe
BTI, ITB	[Op1: = opérateur (Op2)]

Le tableau suivant détaille les opérandes :

Types	Op1	Op2
Mots	%MWi, %QWi, %SWi %BLK.x	%MWi, %KWi, %IWi, %QWi, %IWSi, %QWSi, %SWi, %BLK.x
Mot double	%MDi %BLK.x	%MDi, %KDi %BLK.x

Structure

Exemples d'instructions de conversion BCD/binaire :

Réseau	Instruction
0	LD %M0 [%MW0:=BTI(%MW10)]
1	LD %IO.2 [%MW10:=ITB(%KW9)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Exemple d'application

L'instruction **BTI** permet de traiter une valeur de consigne dans les entrées du contrôleur via des roues codeuses en BCD.

L'instruction **ITB** permet d'afficher des valeurs numériques (résultat d'un calcul, valeur courante d'un bloc fonction, par exemple) sur des afficheurs codés en BCD.

Instructions de conversion de mot simple/double

Introduction

Le tableau suivant décrit les instructions de conversion entre des mots simples et doubles :

Instruction	Fonction
LW	Extrait l'octet de poids faible d'un mot double vers un mot.
HW	Extrait l'octet de poids fort d'un mot double vers un mot.
CONCATW	Concatène deux mots pour créer un mot double.
DWORD	Convertit un mot de 16 bits en un mot double de 32 bits.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de conversion de mots simples/doubles :

Opérateur	Syntaxe	Op1	Op2	Op3
LW, HW	Op1 = opérateur (Op2)	%MWi	%MDi, %KDi, %BLK.x	[-]
CONCATW	Op1 = opérateur (Op2, Op3))	%MDi, %KDi, %BLK.x	%MWi, %KWi, valeur immédiate	%MWi, %KWi, valeur immédiate
DWORD	Op1 = opérateur (Op2)	%MDi, %KDi, %BLK.x	%MWi, %KWi	[-]

Structure

Exemples d'instructions de conversion de mots simples/doubles :

Réseau	Instruction
0	LD %M0 [%MW0:=HW(%MD10)]
1	LD %I0.2 [%MD10:=DWORD(%KW9)]
2	LD %I0.3 [%MD11:=CONCATW(%MW10,%MW5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Sous-chapitre 3.3

Programme

Objet de cette section

Cette section présente les instructions de programme.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Instructions END	83
Instructions NOP	84
Instructions de saut	85
Instructions de sous-programme	87

Instructions END

Introduction

Les instructions **END** définissent la fin de l'exécution de la scrutation d'un programme.

END, ENDC et ENDCN

Trois instructions **END** sont disponibles :

- **END** : fin de programme inconditionnelle ;
- **ENDC** : fin de programme si le résultat booléen de l'instruction de test précédente est 1 ;
- **ENDCN** : fin de programme si le résultat booléen de l'instruction de test précédente est 0.

Par défaut (en mode Normal), les sorties sont mises à jour et la scrutation suivante débute dès la fin du programme.

Si la scrutation est périodique, les sorties sont mises à jour et la scrutation suivante débute dès que la période écoulée.

Exemples

Exemple d'instruction **END** inconditionnelle :

Réseau	Instruction
0	LD %M1 ST %Q0.1
1	LD %M2 ST %Q0.2
2	END

Exemple d'instruction **END** conditionnelle :

Réseau	Instruction
0	LD %I0.0 ST %Q0.0
1	LD %I0.1 ST %Q0.1
2	LD %I0.2 ENDC
3	LD %I0.3 ST %Q0.2
4	END

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Instructions NOP

Introduction

Les instructions `NOP` n'effectuent aucune opération. Utilisez-les pour "réserver" des lignes d'un programme afin d'insérer ultérieurement des instructions, sans modifier les numéros de ligne.

Instructions de saut

Introduction

Les instructions de saut interrompent immédiatement l'exécution d'un programme et la reprennent à partir de la ligne de programme contenant l'étiquette %L*i* (i = numéro de module maximum).

JMP, JMPC et JMPCN

Trois instructions de saut sont disponibles :

- JMP : saut de programme inconditionnel ;
- JMPC : saut de programme si le résultat booléen de la logique précédente est 1 ;
- JMPCN : saut de programme si le résultat booléen de la logique précédente est 0.

Exemples

Exemples d'instructions de saut :

Réseau	Instruction
0	LD %M15 JMPC %L8
1	LD [%MW24<%MW12] ST %Q0.3 JMPC %L12
2	%L8 : LD %M12 AND %M13 ST %M12 JMPC %L12
3	LD %M11 S %Q0.0
4	%L12 : LD %I0.0 ST %Q0.4

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Instructions

- Les instructions de saut sont interdites entre parenthèses et ne doivent pas être placées entre les instructions AND, OR et une parenthèse fermante ")".
- L'étiquette ne peut être placée que devant une instruction LD, LDN, LDR, LDF ou BLK.
- Le numéro de l'étiquette %Li doit être défini une seule fois dans un programme.
- Le saut de programme est effectué vers une ligne de programmation en amont ou en aval. Lorsque le saut est en amont, le temps de scrutation doit être contrôlé. Un temps de scrutation trop long peut provoquer le déclenchement du chien de garde.

Instructions de sous-programme

Introduction

Les instructions de sous-programme obligent un programme à exécuter un sous-programme puis à revenir au programme principal, au point d'appel du sous-programme.

Procédure

Un sous-programme est créé dans un POU libre. Pour plus d'informations sur la création d'un POU libre et la définition du numéro du sous-programme, consultez la section POU libres (*voir SoMachine Basic, Operating Guide*). Pour plus d'informations sur la gestion des POU avec une tâche et des réseaux, consultez la section Gestion des POU (*voir SoMachine Basic, Operating Guide*).

Appel d'un sous-programme en trois étapes :

- 1 L'instruction `SRn` appelle le sous-programme référencé par un POU libre `SRn`, si le résultat de l'instruction booléenne précédente est 1.
- 2 Le sous-programme est référencé par un POU libre `SRn`, où `n` est le nombre de sous-programmes.
- 3 L'instruction de sous-programme doit être écrite dans un POU libre indépendant du programme principal.

Pour plus d'informations sur les sous-programmes, consultez la section Création d'une tâche périodique (*voir SoMachine Basic, Operating Guide*).

Exemples

Exemple d'instructions contenant un sous-programme :

Réseau	Instruction
0	LD %M15 AND %M5 ST %Q0.0
1	LD [%MW24>%MW12] SR1
2	LD %I0.4 AND %M13 ST %Q0.1 END

Exemple d'instruction de sous-programme (SR1) :

Réseau	Instruction
0 (SR1)	LD %I0.0 ST %Q0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Consignes

- Un sous-programme ne peut pas appeler un autre sous-programme. Toute tentative d'appeler un sous-programme dans un POU libre génère une erreur détectée de compilation.
- Les instructions de sous-programme sont interdites entre parenthèses et ne doivent pas être placées entre les instructions AND, OR et une parenthèse fermante ")".
- Soyez prudent lorsqu'une instruction d'affectation est suivie d'un appel de sous-programme en langage IL. En effet, le sous-programme risque de modifier le contenu de l'accumulateur booléen. Au retour, sa valeur risque donc d'être différente de celle qu'il avait avant l'appel.

Sous-chapitre 3.4

Virgule flottante

Objet de cette section

Cette section décrit les instructions avancées à virgule flottante.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Instructions arithmétiques sur les objets à virgule flottante	90
Instructions trigonométriques	93
Instructions de conversion d'angle	95
Instructions de conversion d'entiers/nombres à virgule flottante	96

Instructions arithmétiques sur les objets à virgule flottante

Introduction

Ces instructions permettent d'effectuer une opération arithmétique entre deux opérandes à virgule flottante ou sur un opérande à virgule flottante :

Instruction	Objectif
+	Addition de 2 opérandes
-	Soustraction de 2 opérandes
*	Multiplication de 2 opérandes
/	Division de 2 opérandes
LOG	Logarithme en base 10
LN	Logarithme népérien
SQRT	Racine carrée d'un opérande
ABS	Valeur absolue d'un opérande
TRUNC	Partie entière d'une valeur à virgule flottante
EXP	Exponentiel naturel
EXPT	Puissance d'un entier par un réel

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Opérateurs et syntaxe des instructions arithmétiques sur une valeur à virgule flottante :

Opérateurs	Syntaxe
+, - *, /	Op1:= Op2 opérateur Op3
SQRT, ABS, TRUNC, LOG, EXP, LN	Op1:= opérateur (Op2)
EXPT	Op1:= opérateur (Op2,Op3)

Opérandes des instructions arithmétiques sur des nombres à virgule flottante :

Opérateurs	Op1	Op2	Op3
+, - *, /	%MFi	%MFi, %KFi, valeur immédiate	%MFi, %KFi, valeur immédiate
SQRT, ABS, LOG, EXP, LN	%MFi	%MFi, %KFi	[-]
TRUNC	%MFi, %MDi	%MFi, %KFi	[-]
EXPT	%MFi	%MFi, %KFi	%MWi, %KWi, valeur immédiate
Remarque : SoMachine Basic empêche l'utilisation de la fonction avec %MWi comme Op1.			

Structure

Exemple d'instruction arithmétique :

Réseau	Instruction
0	LD %M0 [%MF0:=%MF10+129.7]
1	LD %I0.2 [%MF1:=SQRT(%MF10)]
2	LDR %I0.3 [%MF2:=ABS(%MF20)]
3	LDR %I0.4 [%MF3:=TRUNC(%MF2)]
4	LD %M1 [%MF4:=LOG(%MF10)]
5	LD %I0.5 [%MF5:=LN(%MF20)]
6	LD %I0.0 [%MF6:=EXP(%MF30)]
7	LD %I0.1 [%MF7:=EXPT(%MF40,%MW52)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Règles d'utilisation

- Les opérations sur les valeurs à virgule flottante et les entiers ne peuvent pas être mélangées directement. Les opérations de conversion (voir page 95) assurent la conversion dans l'un ou l'autre de ces formats.
- Le bit système %S18 est géré de la même façon que les opérations sur des entiers (voir page 95). Le mot %SW17 indique la cause de l'erreur détectée.
- Lorsque l'opérande de la fonction est un nombre incorrect (par exemple, le logarithme d'un nombre négatif), il produit un résultat indéterminé ou infini et met le bit %S18 à 1. Le mot %SW17 indique la cause de l'erreur détectée.

NOTE : Pour l'instruction TRUNC, le bit système %S17 n'est pas affecté.

Exemples d'application de l'instruction TRUNC avec %MDi

Le tableau ci-dessous fournit des exemples d'instruction TRUNC lorsque %MDi sert à stocker le résultat :

Exemple	Résultat
TRUNC (3.5)	3
TRUNC (324.18765)	324
TRUNC (927.8904)	927
TRUNC (-7.7)	-7
TRUNC (45.678E+20)	2 147 483 647 (mot double signé maximum) (1) %S18 est mis à 1.
TRUNC (-94.56E+13)	- 2 147 483 648 (mot double signé minimum) (1) %S18 est mis à 1.
(1) Cet exemple s'applique à l'instruction TRUNC lorsqu'elle est utilisée avec %MDi. Lorsqu'elle est utilisée avec %MFi, l'instruction TRUNC n'a pas de dépassement et n'a donc pas de limite maximum/minimum.	

Instructions trigonométriques

Introduction

Ces instructions permettent de réaliser des opérations trigonométriques :

SIN	Sinus d'un angle exprimé en radians	ASIN	Arc sinus (résultat entre $-\frac{\pi}{2}$ et $\frac{\pi}{2}$)
COS	Cosinus d'un angle exprimé en radians	ACOS	Arc cosinus (résultat entre 0 et π)
TAN	Tangente d'un angle exprimée en radians	ATAN	Arc tangente (résultat entre $-\frac{\pi}{2}$ et $\frac{\pi}{2}$)

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Opérateurs, opérands et syntaxe des instructions pour des opérations trigonométriques

Opérateurs	Syntaxe	Op1	Op2
SIN, COS, TAN, ASIN, ACOS, ATAN	Op1:= opérateur (Op2)	%MFi	%MFi, %KFi

Structure

Exemple d'instructions trigonométriques (Trigonometric) :

Réseau	Instruction
0	LD %M0 [%MF0:=SIN(%MF10)]
1	LD %I0.0 [%MF1:=TAN(%MF20)]
2	LD %I0.3 [%MF2:=ATAN(%MF30)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Règles d'utilisation

- Lorsque l'opérande de la fonction est un nombre incorrect (par exemple, l'arc cosinus d'un nombre supérieur à 1), il produit un résultat indéterminé ou infini et met le bit %S18 à 1. Le mot %SW17 indique la cause de l'erreur détectée.
- Les fonctions SIN/COS/TAN admettent en paramètre un angle entre -4096π et 4096π , mais leur précision décroît progressivement pour les angles en dehors de l'intervalle entre -2π et $+2\pi$ en raison de l'imprécision apportée par le modulo 2π effectué sur le paramètre avant toute opération.

Instructions de conversion d'angle

Introduction

Ces instructions permettent de réaliser des opérations de conversion :

DEG_TO_RAD	Conversion de degrés en radians. Le résultat est la valeur de l'angle compris entre 0 et 2π .
RAD_TO_DEG	Conversion d'un angle exprimé en radians. Le résultat est la valeur de l'angle compris entre 0 et 360 degrés.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Opérateurs, opérandes et syntaxe des instructions de conversion :

Opérateurs	Syntaxe	Op1	Op2
DEG_TO_RAD RAD_TO_DEG	Op1:= opérateur (Op2)	%MFi	%MFi, %KFi

Structure

Exemple d'instructions de conversion :

Réseau	Instruction
0	LD %M0 [%MF0 := DEG_TO_RAD (%MF10)]
1	LD %M2 [%MF2 := RAD_TO_DEG (%MF20)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Règles d'utilisation

L'angle à convertir doit être compris entre -737 280,0 et +737 280,0 (pour une conversion

DEG_TO_RAD) ou entre -4096π et 4096π (pour une conversion RAD_TO_DEG).

Pour les valeurs situées hors de ces plages, le résultat affiché sera +1.#QNAN, les bits %S18 et %SW17:X0 étant mis à 1.

Instructions de conversion d'entiers/nombres à virgule flottante

Introduction

Quatre instructions de conversion sont proposées :

INT_TO_REAL	Conversion d'un mot entier en valeur flottante
DINT_TO_REAL	Conversion d'un mot double (entier) en valeur flottante
REAL_TO_INT	Conversion d'une valeur flottante en mot entier (le résultat étant la valeur algébrique la plus proche)
REAL_TO_DINT	Conversion d'une valeur flottante en mot double entier (le résultat étant la valeur algébrique la plus proche)

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Opérateurs et syntaxe (conversion d'un mot entier en valeur flottante) :

Opérateurs	Syntaxe
INT_TO_REAL	Op1=INT_TO_REAL(Op2)

Opérandes (conversion d'un mot entier en valeur flottante) :

Op1	Op2
%MFi	%MWi, %KWi

Exemple : conversion d'un mot entier en valeur flottante : 147 en 1,47e+02.

Opérateurs et syntaxe (conversion d'un mot double entier en valeur flottante) :

Opérateurs	Syntaxe
DINT_TO_REAL	Op1=DINT_TO_REAL(Op2)

Opérandes (conversion d'un mot double entier en valeur flottante) :

Op1	Op2
%MFi	%MDi, %KDi

Exemple : conversion d'un mot entier en valeur flottante : 68 905 000 en 6,8905e+07.

Opérateurs et syntaxe (conversion d'une valeur flottante en mot entier ou double entier) :

Opérateurs	Syntaxe
REAL_TO_INT	Op1=opérateur (Op2)
REAL_TO_DINT	

Opérateurs (conversion d'une valeur flottante en mot entier ou double entier) :

Type	Op1	Op2
Mots	%MWi	%MFi, %KFi
Mots doubles	%MDi	%MFi, %KFi

Exemple :

- Conversion d'une valeur flottante en mot entier : 5978,6 en 5979
- Conversion d'une valeur flottante en mot double entier : 1 235 978,6 en 1 235 979

NOTE : Si, lors d'une conversion d'un réel en entier (ou d'un réel en entier double), la valeur flottante est en dehors des limites du mot (ou du mot double), le bit %S18 est mis à 1.

Structure

Exemple d'instruction de conversion d'entier/valeur flottante :

Réseau	Instruction
0	LD 1 [%MF0:=INT_TO_REAL(%MW10)]
1	LD I0.8 [%MD2:=REAL_TO_DINT(%MF9)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Précision d'arrondi

La norme IEEE 754 définit 4 modes d'arrondi pour les opérations sur les valeurs flottantes.

Le mode utilisé par les instructions ci-dessus est le mode "arrondi au plus près" :

"si les valeurs représentables les plus proches sont à égale distance du résultat théorique, la valeur fournie sera celle dont le bit de poids faible est égal à 0".

En fait, la valeur va être arrondie au nombre supérieur ou inférieur, mais pas au nombre pair.

Par exemple :

- Arrondi de la valeur 10,5 à 10.
- Arrondi de la valeur 11,5 à 12.

Sous-chapitre 3.5

ASCII

Objet de cette section

Cette section décrit les instructions avancées d'ASCII.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Instructions ROUND	99
Instructions de conversion ASCII/entier	101
Instructions de conversion entier/ASCII	103
Instruction de conversion ASCII/valeur flottante	105
Instructions de conversion valeur flottante/ASCII	107

Instructions ROUND

Introduction

L'instruction `ROUND` arrondit une représentation flottante stockée dans une chaîne ASCII.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Pour l'instruction `ROUND`, utilisez la syntaxe : `Op1 := ROUND ( Op2, Op3 )`.

Par exemple :

```
[ %MW0 : 7 := ROUND ( %MW8, 4 ) ]
```

Paramètres

Le tableau ci-dessous décrit les paramètres de la fonction `ROUND` :

Paramètres	Description
Op1	%MW qui stocke le résultat.
Op2	%MW qui contient la valeur flottante à arrondir.
Op3	Nombre de chiffres significatifs requis pour arrondir. Entier de 1 à 8

Règles d'utilisation

Les règles de l'instruction `ROUND` sont les suivantes :

- L'opérande est toujours arrondi à la valeur inférieure.
- Le caractère de fin de la chaîne de l'opérande est utilisé comme caractère de fin de la chaîne de résultat.
- Il peut s'agir de tout caractère ASCII non compris dans l'intervalle [0 - 9] ([16#30 - 16#39]), à l'exception des caractères suivants :
 - le point (.) (16#2E) ;
 - le signe moins '-' (16#2D) ;
 - le signe plus '+' (16#2B) ;
 - l'exposant 'e' ou 'E' (16#65 ou 16#45).
- Le résultat et l'opérande ne doivent pas compter plus de 13 octets : la taille maximum d'une chaîne ASCII est de 13 octets.
- La notation scientifique n'est pas autorisée.

Cas particuliers

Le logiciel vérifie la syntaxe. Les exemples suivants risquent d'entraîner des erreurs de syntaxe :

Syntaxe incorrecte	Syntaxe correcte
%MW10:= ROUND(%MW1,4) ":7" manquant dans le résultat	%MW10:7 := ROUND(%MW1,4)
%MW10:13:= ROUND(%MW1,4) %MW10 : n où n ≠ 7 est incorrect	%MW10:7 := ROUND(%MW1,4)

Exemple d'application

Ce tableau montre des exemples de l'instruction `ROUND` :

Exemple	Résultat
ROUND("987654321", 5)	"987650000"
ROUND("-11.1", 8)	"-11.1"
ROUND("NAN")	"NAN"

Instructions de conversion ASCII/entier

Introduction

L'instruction de conversion ASCII/entier convertit une chaîne ASCII en une valeur entière.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Pour les instructions de conversion ASCII/entier, respectez la syntaxe suivante :

```
Op1 := ASCII_TO_INT( Op2 )
```

Par exemple :

```
[ %MW0 := ASCII_TO_INT( %MW8 ) ]
```

Paramètres

Le tableau suivant décrit les paramètres de la fonction de conversion ASCII/entier :

Paramètres	Description
Op1	%MW qui stocke le résultat.
Op2	%MW ou %KW

Règles d'utilisation

Les règles de conversion ASCII/entier sont les suivantes :

- Op2 doit être compris entre -32 768 et 32 767.
- La fonction lit toujours l'octet de poids fort en premier.
- Tout caractère ASCII non compris dans l'intervalle [0 - 9] ([16#30 - 16#39]) est considéré comme un caractère de fin, à l'exception du signe moins '-' (16#2D) lorsqu'il est placé comme premier caractère.
- En cas de débordement (> 32 767 ou < -32 768), le bit système %S18 (erreur détectée ou débordement arithmétique) est mis à 1 et la valeur 32 767 ou -32 768 est renvoyée.
- Si le premier caractère de l'opérande est un caractère de « séparateur », la valeur 0 est renvoyée et le bit %S18 est mis à 1.
- La notation scientifique n'est pas autorisée.

Exemple d'application

Considérons que les données ASCII suivantes ont été stockées dans les mots %MW10 à %MW13 :

Paramètre	Valeur hexadécimale	Valeur ASCII
%MW10	16#3932	9, 2
%MW11	16#3133	1, 3
%MW12	6#2038	' ', 8
%MW13	16#3820	8, ' '

Le tableau ci-dessous fournit des exemples de conversion ASCII/entier :

Exemple	Résultat
%MW20 := ASCII_TO_INT(%MW10)	%MW20 = 29318
%MW20 := ASCII_TO_INT(%MW12)	%MW20 = 8
%MW20 := ASCII_TO_INT(%MW13)	%MW20 = 0 et %S18 est mis à 1.

Instructions de conversion entier/ASCII

Introduction

Les instructions de conversion entier/ASCII convertissent un entier en chaîne ASCII.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Pour les instructions de conversion entier/ASCII, respectez la syntaxe :

```
Op1 := INT_TO_ASCII ( Op2 )
```

Par exemple :

```
[ %MW0:4:=INT_TO_ASCII (%MW8) ]
```

Paramètres

Le tableau suivant décrit les paramètres de la fonction de conversion entier/ASCII :

Paramètres	Description
Op1	%MW qui stocke le résultat.
Op2	%MW, %KW, %SW, %IW, %QW ou tout MOT (Les valeurs immédiates ne sont pas acceptées.)

Règles d'utilisation

Les règles de conversion entier/ASCII sont les suivantes :

- Op2 doit être compris entre -32 768 et 32 767.
- La fonction écrit toujours l'octet de poids fort en premier.
- Le caractère de fin est "Entrée" (ASCII 13).
- La fonction détermine automatiquement le nombre de %MWs devant contenir des valeurs ASCII (de 1 à 4).

Erreurs de syntaxe

Le logiciel vérifie la syntaxe. Les exemples suivants risquent d'entraîner des erreurs de syntaxe :

Syntaxe incorrecte	Syntaxe correcte
%MW10 := INT_TO_ASCII(%MW1) ":4" manquant dans le résultat	%MW10:4 := INT_TO_ASCII(%MW1)
%MW10:n := INT_TO_ASCII(%MW1) %MW10 : n où n ≠ 4 est incorrect	%MW10:4 := INT_TO_ASCII(%MW1)

Exemple d'application

Pour l'instruction `MW10:4 := INT_TO_ASCII(%MW1)` :

Si...	Alors...	
Valeur entière	Valeur hexadécimale	Valeur ASCII
%MW1 = 123	%MW10 = 16#3231	2, 1
	%MW11 = 16#0D33	3
%MW1 = 45	%MW10 = 16#3534	5, 4
	%MW11 = 16#000D	'entrée'
%MW1 = 7	%MW10 = 16#0D37	'Entrée', 7
%MW1 = -12369	%MW10 = 16#3145	1, '-'
	%MW11 = 16#3332	3, 2
	%MW10 = 16#3936	9, 6
	%MW11 = 16#000D	'entrée'

Instruction de conversion ASCII/valeur flottante

Introduction

L'instruction de conversion ASCII/valeur flottante convertit une chaîne ASCII en valeur flottante.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Pour les instructions de conversion ASCII/valeur flottante, respectez la syntaxe suivante :

```
Op1 := ASCII_TO_FLOAT( Op2 ).
```

Par exemple :

```
[ %MF0 := ASCII_TO_FLOAT ( %MW8 ) ]
```

Paramètres

Le tableau suivant décrit les paramètres de la fonction de conversion ASCII/valeur flottante :

Paramètres	Description
Op1	%MF
Op2	%MW or %KW

Règles d'utilisation

Les règles de conversion d'une chaîne ASCII en valeur flottante sont les suivantes :

- La fonction lit toujours l'octet de poids fort en premier.
- Tout caractère ASCII non compris dans l'intervalle [0 - 9] ([16#30 - 16#39]) est considéré comme un caractère de "fin", sauf les caractères suivants :
 - le point (.) (16#2E) ;
 - le signe moins '-' (16#2D) ;
 - le signe plus '+' (16#2B) ;
 - l'exposant 'e' ou 'E' (16#65 ou 16#45).
- Le format de la chaîne ASCII peut être une notation scientifique (par ex., "-2.34567e+13") ou décimale (par ex., "9826.3457").
- En cas de débordement (résultat du calcul > 3.402824E+38 ou < -3.402824E+38) :
 - le bit système %S18 (erreur détectée ou débordement arithmétique) est mis à 1 ;
 - %SW17:X3 est mis à = 1 ;
 - la valeur +/- 1.#INF (valeur infinie + ou -) est renvoyée.
- Si le résultat du calcul est compris entre -1,175494E-38 et 1,175494E-38, il est arrondi à 0,0.
- Si l'opérande n'est pas un nombre :
 - La valeur 1.#QNAN est renvoyée ;
 - le bit %SW17:X0 est mis à 1.

Exemple d'application

Considérons que les données ASCII suivantes ont été stockées dans les mots %MW10 à %MW14 :

Paramètre	Valeur hexadécimale	Valeur ASCII
%MW10	16#382D	8, '-'
%MW11	16#322E	2, '.'
%MW12	16#3536	5, 6
%MW13	16#2B65	+', 'e'
%MW14	16#2032	',' , 2

Le tableau ci-dessous fournit des exemples de conversion ASCII/valeur flottante :

Exemple	Résultat
%MF20 := ASCII_TO_FLOAT(%MW10)	%MF20 = -826.5
%MF20 := ASCII_TO_FLOAT(%MW11)	%MF20 = 1.#QNAN
%MF20 := ASCII_TO_FLOAT(%MW12)	%MF20 = 6500.0
%MF20 := ASCII_TO_FLOAT(%MW13)	%MF20 = 1.#QNAN
%MF20 := ASCII_TO_FLOAT(%MW14)	%MF20 = 2.0

Instructions de conversion valeur flottante/ASCII

Introduction

L'instruction de conversion valeur flottante/ASCII convertit une valeur flottante en chaîne ASCII.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Pour les instructions de conversion valeur flottante/ASCII, respectez la syntaxe :

```
Op1 := FLOAT_TO_ASCII (Op2).
```

Par exemple :

```
[ %MW0:7:=FLOAT_TO_ASCII (%MF8) ]
```

Paramètres

Le tableau suivant décrit les paramètres de la fonction de conversion valeur flottante/ASCII :

Paramètre	Description
Op1	%MW
Op2	%MF or %KF

Règles d'utilisation

Les règles de conversion d'une valeur flottante en valeur ASCII sont les suivantes :

- La fonction écrit toujours l'octet de poids fort en premier.
- La représentation s'effectue à l'aide d'une notation scientifique conventionnelle.
- Le résultat « Infini » ou « Pas un nombre » renvoie la chaîne « NAN ».
- Le caractère de fin est "Entrée" (ASCII 13).
- La fonction détermine automatiquement le nombre de %MW devant contenir des valeurs ASCII.
- La précision de conversion est de 6 chiffres.
- La notation scientifique n'est pas autorisée.

Erreurs de syntaxe

Le logiciel vérifie la syntaxe. Les exemples suivants risquent d'entraîner des erreurs de syntaxe :

Syntaxe incorrecte	Syntaxe correcte
%MW10 := FLOAT_TO_ASCII (%MF1) ":7" manquant dans le résultat	%MW10:7 := FLOAT_TO_ASCII (%MF1)
%MW10:n := FLOAT_TO_ASCII (%MF1) %MW10:n où n ≠ 7 est incorrect	%MW10:7 := FLOAT_TO_ASCII (%MF1)

Exemple d'application

Pour l'instruction `%MW10:7 := FLOAT_TO_ASCII(%MF1)` :

Nombre à convertir	Résultat
1234567800	1.23456e+09
0.000000921	9.21e-07
9.87654321	9.87654
1234	1.234e+03

Sous-chapitre 3.6

Opérateurs de pile

Instructions de pile (MPS, MRD, MPP)

Introduction

Les instructions de pile gèrent le routage vers les bobines. Les instructions `MPS`, `MRD` et `MPP` utilisent une zone de stockage temporaire, appelée pile, pouvant mémoriser jusqu'à 32 expressions booléennes.

NOTE : Elles ne peuvent pas être utilisées dans une expression comprise entre parenthèses.

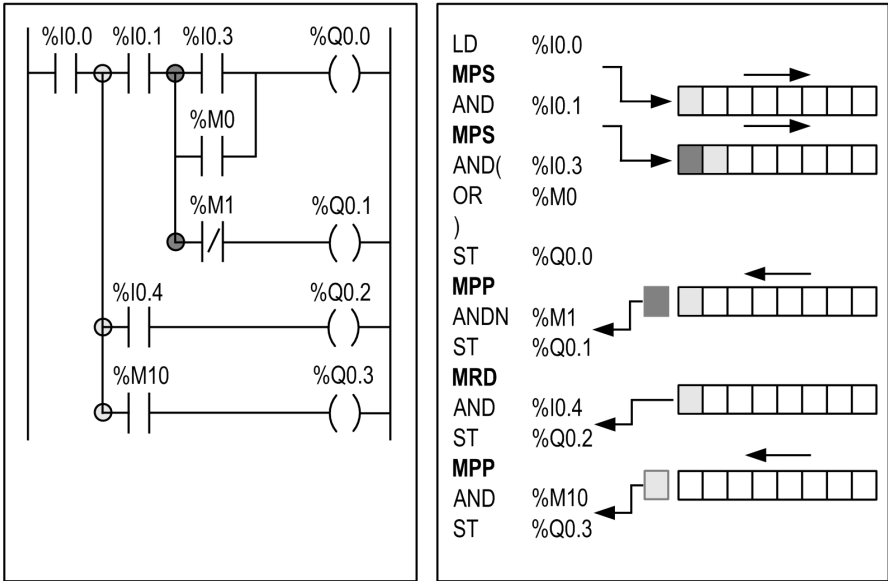
Syntaxe

Le tableau suivant décrit les trois instructions de pile :

Instruction	Description	Fonction
MPS	Abréviation de Memory Push onto Stack (Mise en mémoire dans la pile)	Stocke le résultat de la dernière instruction logique (contenu de l'accumulateur) en haut de la pile. Ceci a pour effet de décaler les autres valeurs de la pile vers le bas.
MRD	Abréviation de Memory Read from stack (Lecture mémoire depuis la pile)	Lit la valeur stockée en haut de la pile et la transmet à l'accumulateur.
MPP	Abréviation de Memory Pop from Stack (Extraction mémoire depuis la pile)	Copie la valeur située en haut de la pile dans l'accumulateur et décale les autres valeurs de la pile vers le haut.

Fonctionnement

Ce schéma explique comment les instructions de pile fonctionnent :



Exemple d'application

Exemple d'utilisation des instructions de pile :

Réseau	Instruction
0	LD %I0.0 AND %M1 MPS AND %I0.1 ST %Q0.0 MRD AND %I0.2 ST %Q0.1 MRD AND %I0.3 ST %Q0.2 MPP AND %I0.4 ST %Q0.3

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Sous-chapitre 3.7

Instructions sur tables d'objets

Objet de cette section

Cette section décrit les instructions pour les tables d'objets :

- de mots doubles,
- d'objets à virgule flottante.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Affectation de tables de mots, mots doubles et valeurs flottantes	112
Fonctions de sommation de table	114
Fonction de comparaison de tables	116
Fonctions de recherche dans une table	118
Fonctions de recherche de valeurs maximum et minimum dans des tables	120
Nombre d'occurrences d'une valeur dans une table	122
Fonctions de décalage circulaire dans une table	123
Fonctions de tri de table	125
Fonctions LKUP (interpolation de table à virgule flottante)	126
Fonctions MEAN des valeurs d'une table de valeurs flottantes	131

Affectation de tables de mots, mots doubles et valeurs flottantes

Introduction

Les instructions d'affectation peuvent s'exécuter sur les tables d'objets suivants :

- Valeur immédiate à table de mots (voir le réseau 0 de l'exemple de structure (voir page 113)) ou table de mots doubles
- Mot à table de mots (voir le réseau 1 de l'exemple de structure (voir page 113))
- Table de mots à table de mots (voir le réseau 2 de l'exemple de structure (voir page 113))
La longueur (L) doit être la même pour les deux tables.
- Mot double à table de mots doubles
- Table de mots doubles à table de mots doubles
La longueur (L) doit être la même pour les deux tables.
- Valeur flottante immédiate à table de valeurs flottantes
- Valeur flottante à table de valeurs flottantes
- Table de valeurs flottantes à table de valeurs flottantes
La longueur (L) doit être la même pour les deux tables.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions (voir page 19) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des affectations de tables de mots, mots doubles et valeurs flottantes :

Opérateur	Syntaxe
:=	[Op1: = Op2] Op1 prend la valeur de Op2.

Le tableau suivant détaille les opérandes :

Type	Op1	Op2
Table de mots	%MWi:L, %SWi:L	%MWi:L, %SWi:L, valeur entière immédiate, %MWi, %KWi, %IW, %QW, %SWi, %BLK.x
Tables de mots doubles	%MDi:L	Valeur entière immédiate, %MDi, %KDi, %MDi:L, %KDi:L
Tables de mots flottants	%MFi:L	Valeur flottante immédiate, %MFi, %KFi, %MFi:L, %KFi:L
L Longueur de la table (255 au maximum).		

NOTE : L'abréviation %BLK.x (R3.I, par exemple) permet de désigner n'importe quel mot de bloc fonction.

Structure

Exemples d'affectations de tables de mots

Réseau	Instruction
0	LD 1 [%MW0:10:=100]
1	LD %I0.0 [%MW0:10:=%MW11]
2	LDR %I0.3 [%MW10:20:=%KW20:30]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Fonctions de sommation de table

Introduction

La fonction SUM_ARR effectue la somme de tous les éléments d'un tableau d'objet :

- Si la table est constituée de mots doubles, le résultat est donné sous la forme d'un mot double.
- Si la table est constituée de mots flottants, le résultat est donné sous la forme d'un mot flottant.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions (*voir page 19*) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Format de l'instruction de sommation de table :

Res:=SUM_ARR (Tab)

Paramètres de l'instruction de sommation de table :

Type	Résultat (Res)	Table (Tab)
Tables de mots doubles	%MDi	%MDi:L, %KDi:L
Tables de mots flottants	%MFi	%MFi:L, %KFi:L
L Longueur de la table (255 au maximum).		

NOTE : Le bit %S18 est mis à 1 lorsque le résultat est hors des limites du format de mot double correspondant à l'opérande de la table.

Structure

Exemple de fonction de sommation :

Réseau	Instruction
0	LD %I0.2 [%MD5:=SUM_ARR (%MD3:1)]
1	LD 1 [%MD5:=SUM_ARR (%KD5:2)]
2	LD 1 [%MF2:=SUM_ARR (%MF8:5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Exemple d'application

`%MD4:=SUM(%MD30:4)`

où `%MD30=10`, `%MD32=20`, `%MD34=30`, `%MD36=40`

Donc `%MD4:=10+20+30+40`

Fonction de comparaison de tables

Introduction

La fonction `EQUAL_ARR` compare deux tables, élément par élément.

Si une différence apparaît, le rang des premiers éléments dissemblables est retourné sous la forme d'un mot, sinon la valeur renvoyée est égale à -1.

La comparaison porte sur la totalité de la table.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Format de l'instruction de comparaison de tables :

`Res:=EQUAL_ARR(Tab1,Tab2)`

Paramètres des instructions de comparaison de tables :

Type	Résultat (Res)	Tables (Tab1 et Tab2)
Tables de mots doubles	%MWi	%MDi:L, %KDi:L
Tables de mots flottants	%MWi	%MFi:L, %KFi:L
L Longueur de la table (255 au maximum).		

NOTE : Les tables doivent être obligatoirement de même longueur et de même type.

Structure

Exemple de fonction de comparaison de tables :

Réseau	Instruction
0	LD %I0.2 [%MW5:=EQUAL_ARR(%MD20:7, %KD0:7)]
1	LD 1 [%MW0:=EQUAL_ARR(%MD20:7, %KD0:7)]
2	LD 1 [%MF2:=SUM_ARR(%MF8:5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Exemple d'application

`%MW5:=EQUAL_ARR(%MD30:4,%KD0:4)`

Comparaison des deux tables :

Rang	Table de Mots	Tables de mots constants	Différence
0	%MD30=10	%KD0=10	=
1	%MD32=20	%KD2=20	=
2	%MD34=30	%KD4=60	Différent
3	%MD36=40	%KD6=40	=

La valeur du mot `%MW5` est 2 (premier rang différent).

Fonctions de recherche dans une table

Introduction

Trois fonctions de recherche sont proposées :

- **FIND_EQR** : recherche, dans une table de mots doubles ou flottants, de la position du premier élément égal à une valeur donnée.
- **FIND_GTR** : recherche, dans une table de mots doubles ou flottants, de la position du premier élément supérieur à une valeur donnée.
- **FIND_LTR** : recherche, dans une table de mots doubles ou flottants, de la position du premier élément inférieur à une valeur donnée.

Le résultat de ces instructions est égal au rang du premier élément trouvé ou à -1 si la recherche est infructueuse.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Format des instructions de recherche dans une table :

Fonction	Syntaxe
FIND_EQR	Res:=Fonction(Tab,Val)
FIND_GTR	
FIND_LTR	

Paramètres des instructions de recherche dans des tables de mots doubles et flottants :

Type	Résultat (Res)	Table (Tab)	Valeur (Val)
Tables de mots flottants	%MWi	%MFi:L, %KFi:L	%MFi, %KFi
Tables de mots doubles	%MWi	%MDi:L, %KDi:L	%MDi, %KDi
L Longueur de la table (255 au maximum).			

Structure

Exemple de fonction de recherche dans une table :

Réseau	Instruction
0	LD %I0.2 [%MW5:=FIND_EQR(%MD20:7,%KD0)]
1	LD %I0.3 [%MW6:=FIND_GTR(%MD20:7,%KD0)]
2	LD 1 [%MW7:=FIND_LTR(%MF40:5,%KF4)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Exemple d'application

%MW5:=FIND_EQR(%MD30:4,%KD0)

Recherche de la position du premier mot double = %KD0=30 dans la table :

Rang	Table de Mots	Résultat
0	%MD30=10	-
1	%MD32=20	-
2	%MD34=30	Valeur (Val), rang
3	%MD36=40	-

Fonctions de recherche de valeurs maximum et minimum dans des tables

Introduction

Deux fonctions de recherche sont proposées :

- `MAX_ARR` : recherche de la valeur maximum dans une table de mots doubles et flottants.
- `MIN_ARR` : recherche de la valeur minimum dans une table de mots doubles et flottants.

Le résultat de ces instructions est égal à la valeur maximum (ou minimum) trouvée dans la table.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de recherche des valeurs maximum et minimum dans des tables :

Fonction	Syntaxe
<code>MAX_ARR</code>	Résultat:=Fonction(Onglet)
<code>MIN_ARR</code>	

Paramètres des instructions de recherche des valeurs maximum et minimum dans des tables :

Type	Résultat (Res)	Table (Tab)
Tables de mots doubles	<code>%MDi</code>	<code>%MDn:L, %KDn:L</code>
Tables de mots flottants	<code>%MFi</code>	<code>%MFn:L, %KFn:L</code>
i Identificateur d'instance d'objet pour la variable mémoire. n Index mémoire de la table, indiquant l'adresse de base de la recherche. L Nombre de positions à prendre en compte lors d'une recherche, notamment l'index de l'adresse de base (valeur maximum de L : 255).		

NOTE : L ne compte que les adresses qui ne se recouvrent pas pendant la recherche. Pour plus d'informations, consultez la section Possibilité de recouvrement entre des objets ([voir page 36](#)).

Structure

Exemple de fonction de recherche dans une table :

Réseau	Instruction
0	LD %I0.2 [%MD0:=MIN_ARR(%MD20:7)]
1	LD 1 [%MF8:=MIN_ARR(%MF40:5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Nombre d'occurrences d'une valeur dans une table

Introduction

La fonction `OCCUR_ARR` recherche dans une table de mots doubles ou flottants le nombre d'éléments égaux à une valeur donnée.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de recherche de valeurs maximum et minimum dans une table :

Fonction	Syntaxe
<code>OCCUR_ARR</code>	<code>Res:=Fonction(Tab,Val)</code>

Paramètres des instructions de recherche de valeurs maximum et minimum dans une table :

Type	Résultat (Res)	Table (Tab)	Valeur (Val)
Tables de mots doubles	<code>%MWi</code>	<code>%MDi:L, %KDi:L</code>	<code>%MDi, %KDi</code>
Tables de mots flottants	<code>%MFi</code>	<code>%MFi:L, %KFi:L</code>	<code>%MFi, %KFi</code>
L Longueur de la table (255 au maximum).			

Structure

Exemple de nombre d'occurrences :

Réseau	Instruction
0	<code>LD %I0.3</code> <code>[%MW5:=OCCUR_ARR(%MF20:7, %KF0)]</code>
1	<code>LD %I0.2</code> <code>[%MW5:=OCCUR_ARR(%MD20:7, %MD1)]</code>

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

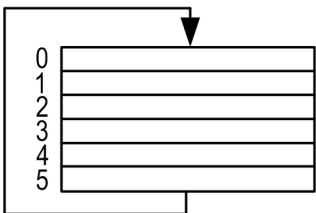
Fonctions de décalage circulaire dans une table

Introduction

Deux fonctions de décalage sont proposées :

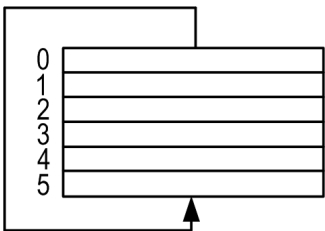
- `ROL_ARR` : réalise le décalage circulaire de n positions des éléments du haut vers le bas dans une table de mots flottants.

Illustration des fonctions `ROL_ARR`



- `ROR_ARR` : réalise le décalage circulaire de n positions des éléments du bas vers le haut dans une table de mots flottants.

Illustration des fonctions `ROR_ARR`



Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe des instructions de décalage circulaire dans des tables de doubles mots ou flottants

`ROL_ARR` et `ROR_ARR`

Fonction	Syntaxe
<code>ROL_ARR</code>	Fonction(n,Tab)
<code>ROR_ARR</code>	

Paramètres des instructions de décalage circulaire pour les tables de mots flottants : **ROL_ARR** et **ROR_ARR** :

Type	Nombre de positions (n)	Table (Tab)
Tables de mots flottants	%MWi, immediate value	%MFi:L
Tables de mots doubles	%MWi, immediate value	%MDi:L
L Longueur de la table (255 au maximum).		

NOTE : Si la valeur de n est négative ou nulle, aucun décalage n'est effectué.

Structure

Exemple de fonction de décalage circulaire :

Réseau	Instruction
0	LD %IO.2 [ROL_ARR(%KW0,%MD20:7)]
1	LD %IO.3 [ROR_ARR(2,%MD20:7)]
2	LD %IO.4 [ROR_ARR(2,%MF40:5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Fonctions de tri de table

Introduction

La fonction `SORT_ARR` trie par ordre croissant ou décroissant les éléments d'une table de mots doubles ou flottants, et stocke le résultat dans cette même table.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Format des fonctions de tri de table :

Fonction	Syntaxe
<code>SORT_ARR</code>	<code>Fonction(sens,Tab)</code>

Le paramètre "sens" indique l'ordre de tri :

- Sens > 0 : le tri est croissant.
- Sens < 0 : le tri est décroissant.
- Sens = 0 : aucun tri n'est effectué.

Le résultat (tableau trié) est renvoyé dans le paramètre Tab (table à trier).

Paramètres des fonctions de tri de table :

Type	Sens du tri	Table (Tab)
Tables de mots doubles	<code>%MWi, immediate value</code>	<code>%MDi:L</code>
Tables de mots flottants	<code>%MWi, immediate value</code>	<code>%MFi:L</code>
L Longueur de la table (255 au maximum).		

Structure

Exemple de fonction de tri de table :

Réseau	Instruction
0	<code>LD %I0.1</code> <code>[SORT_ARR(%MW20,%MF0:6)]</code>
1	<code>LD %I0.2</code> <code>[SORT_ARR(%MW20,%MF0:6)]</code>
2	<code>LD %I0.3</code> <code>[SORT_ARR(0,%MF40:8)]</code>

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Fonctions LKUP (interpolation de table à virgule flottante)

Introduction

La fonction `LKUP` sert à interpoler un ensemble de données flottantes X par rapport à Y pour une valeur X donnée.

Description de l'interpolation linéaire

La fonction `LKUP` suit la règle d'interpolation linéaire, définie dans l'équation suivante :

$$Y = Y_i + \left[\frac{(Y_{i+1} - Y_i)}{(X_{i+1} - X_i)} \cdot (X - X_i) \right] \quad (\text{Equation 1})$$

pour $X_i \leq X \leq X_{i+1}$, où $i = 1 \dots (m-1)$;

en supposant que les valeurs X_i sont classées par ordre croissant :

$$X_1 \leq X_2 \leq \dots X_{m-1} \leq X_m .$$

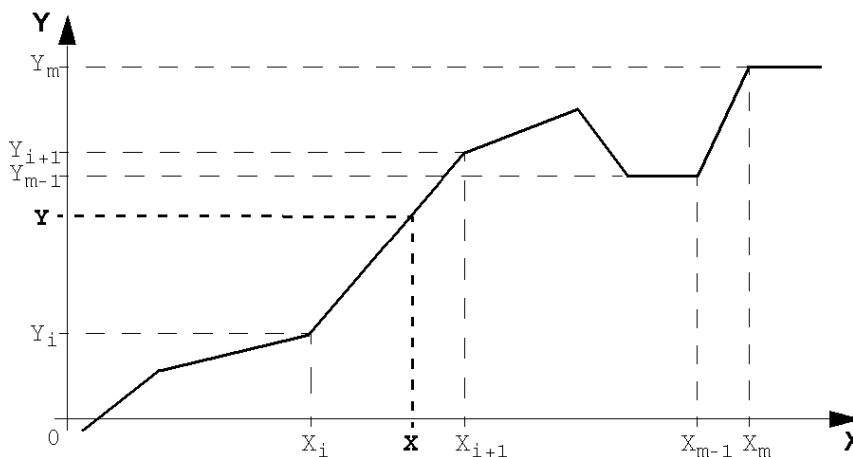
NOTE : Si deux valeurs X_i consécutives sont égales ($X_i = X_{i+1} = X$), l'équation (1) génère une exception non valide. Dans ce cas, l'algorithme suivant est utilisé à la place de l'équation (1) :

$$\text{NOTE : } Y = \left[\frac{(Y_{i+1} - Y_i)}{2} \right] \quad (\text{Equation 2})$$

NOTE : pour $X_i = X_{i+1} = X$, où $i = 1 \dots (m-1)$.

Représentation graphique

Le graphique suivant illustre la règle d'interpolation linéaire décrite ci-dessus :



Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

La fonction **LKUP** utilise trois opérands, dont deux sont des attributs de fonction, comme indiqué dans le tableau suivant :

Syntaxe	Op1 Variable de sortie	Op2 Valeur (X) définie par l'utilisateur	Op3 Tableau de variables (X_i, Y_i) défini par l'utilisateur
[Op1: = LKUP(Op2, Op3)]	%MWi	%MFO	Valeur entière, %MWi, or %KWi

Définition de Op1

Op1 est le mot mémoire qui contient la variable de sortie de la fonction d'interpolation.

Selon la valeur de Op1, l'utilisateur peut savoir si l'interpolation a abouti ou non, ainsi que les causes de l'échec, comme indiqué dans le tableau suivant :

Op1 (%MWi)	Description
0	Interpolation réussie
1	Erreur détectée d'interpolation : tableau incorrect, $X_m < X_{m-1}$
2	Erreur détectée d'interpolation : Op2 hors limites, $X < X_1$
4	Erreur détectée d'interpolation : Op2 hors limites, $X > X_m$
8	Taille incorrecte du tableau de données : • Op3 est défini comme un nombre impair ou • $Op3 < 6$.

NOTE : Op1 ne contient pas la valeur d'interpolation calculée (Y). Pour une valeur (X) donnée, le résultat de l'interpolation (Y) est contenu dans %MF2 du tableau Op3 ([voir page 128](#)).

Définition de Op2

Op2 est la variable flottante (%MF0 du tableau flottant Op3) qui contient la valeur (X) définie par l'utilisateur pour laquelle la valeur (Y) interpolée est calculée :

Plage valide de Op2 : $X_1 \leq Op2 \leq X_m$.

Définition de Op3

Op3 définit la taille (Op3 / 2) du tableau de valeurs flottantes, où les paires de données (X_i , Y_i) sont stockées.

Les données X_i et Y_i sont stockées dans des objets flottants avec des index pairs, à partir de %MF4 (notez que les objets flottants %MF0 et %MF2 sont réservés respectivement à la consigne utilisateur X et à la valeur Y interpolée).

Avec un tableau de (m) paires de données (X_i , Y_i), l'index supérieur (u) du tableau de valeurs flottantes (%MFu) est défini à l'aide des relations suivantes :

- $Op3 = 2 \cdot m$ (Equation 3)
- $u = 2 \cdot (Op3 - 1)$ (Equation 4)

La structure du tableau de valeurs flottantes Op3 (%MF*i*) est similaire à celle de l'exemple suivant (où Op3 = 8) :

(X)		(X ₁)		(X ₂)		(X ₃)	
%MF0		%MF4		%MF8		%MF12	
	%MF2		%MF6		%MF10		%MF14
	(O)		(Y ₁)		(Y ₂)		(Y ₃)
							(Op3 = 8)

NOTE : En raison de la structure du tableau de valeurs flottantes ci-dessus, Op3 doit respecter les exigences suivantes, sous peine de générer une erreur détectée dans la fonction LKUP :

- Op3 est un chiffre pair et
- Op3 ≥ 6 (au moins 2 points de données doivent être disponibles pour permettre l'interpolation linéaire).

Structure

Les opérations d'interpolation sont effectuées de la façon suivante :

Réseau	Instruction
0	LD %I0.2 [%MW20:=LKUP(%MF0,%KW1)]
1	LD %I0.3 [%MW22:=LKUP(%MF0,10)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Exemple d'application

Utilisation d'une fonction d'interpolation LKUP :

```
[ %MW20 := LKUP ( %MF0, 10 ) ]
```

Dans cet exemple :

- %MW20 est Op1 (la variable de sortie).
- %MF0 est la valeur (X) définie par l'utilisateur, dont la valeur (Y) correspondante doit être calculée par interpolation linéaire.
- %MF2 enregistre la valeur calculée (Y) générée par l'interpolation linéaire.
- 10 est Op3 (comme indiqué par l'équation 3 ci-dessus). Il définit la taille du tableau flottant.
%MFu, élément ayant le classement le plus élevé, où u=18 est fourni par l'équation 4 ci-dessus.

Quatre paires de points de données sont stockées dans le tableau Op3 [%MF4, . . . %MF18] :

- %MF4 contient X_1 , %MF6 contient Y_1 .
- %MF8 contient X_2 , %MF10 contient Y_2 .
- %MF12 contient X_3 , %MF14 contient Y_3 .
- %MF16 contient X_4 , %MF18 contient Y_4 .

Fonctions MEAN des valeurs d'une table de valeurs flottantes

Introduction

La fonction **MEAN** permet de calculer la moyenne d'un nombre donné de valeurs dans une table de valeurs flottantes.

Syntaxe

La syntaxe en langage Liste d'instructions est décrite ci-après. Vous pouvez insérer des opérations et des instructions d'affectation en langage Liste d'instructions ([voir page 19](#)) dans des réseaux en langage Schéma à contacts, à l'aide d'un élément graphique **Bloc opération**.

Syntaxe de la fonction de calcul de la moyenne d'une table de valeurs flottantes :

Fonction	Syntaxe
MEAN	Résultat=Fonction(Op1)

Paramètres de la fonction de calcul d'un nombre donné L (255 au maximum) de valeurs dans un tableau de nombres à virgule flottante :

Op1	Résultat (Res)
%MFi:L, %KFi:L	%MFi

Structure

Exemple de fonction de calcul de moyenne :

Réseau	Instruction
0	LD %I3.2 [%MF0:=MEAN(%MF10:5)]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Chapitre 4

Objets logiciels

Contenu de ce chapitre

Ce chapitre contient les sous-chapitres suivants :

Sous-chapitre	Sujet	Page
4.1	Utilisation des blocs fonction	134
4.2	Timer (%TM)	140
4.3	LIFO/FIFO Register (%R)	148
4.4	Shift Bit Register (%SBR)	157
4.5	Step Counter (%SC)	162
4.6	Counter (%C)	168
4.7	Bloc fonction Fast Counter (%FC)	176
4.8	Bloc fonction High Speed Counter (%HSC)	177
4.9	Drum Register (%DR)	178
4.10	Bloc fonction Pulse (%PLS)	186
4.11	Bloc fonction Pulse Width Modulation (%PWM)	187
4.12	Message (%MSG) et échange (EXCH)	188

Sous-chapitre 4.1

Utilisation des blocs fonction

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Principes de programmation des blocs fonction	135
Ajout d'un bloc fonction	137
Configuration d'un bloc fonction	139

Principes de programmation des blocs fonction

Présentation

Un bloc fonction est un objet réutilisable qui accepte une ou plusieurs valeurs d'entrée et renvoie une ou plusieurs valeurs de sortie.

Les paramètres du bloc fonction ne sont pas disponibles si :

- votre contrôleur ne prend pas en charge le bloc fonction ;
- le bloc fonction n'est pas configuré.

Programmes en langage Schéma à contacts

Pour utiliser un bloc fonction dans un programme en langage Schéma à contacts, procédez comme suit :

1. Insérez (*voir page 137*) le bloc fonction dans un réseau.
2. Câblez les entrées et les sorties, si nécessaire.
3. Configurez (*voir page 139*) le bloc fonction en définissant les valeurs de ses paramètres.

Programmes en langage Liste d'instructions

Pour ajouter un bloc fonction dans un programme en langage Liste d'instructions, procédez au choix comme suit :

- Instructions de bloc fonction (par exemple, `BLK %TM2`) : cette méthode de programmation réversible permet d'effectuer des opérations sur le bloc en un emplacement du programme.
- Instructions spécifiques (par exemple, `CU %C1`) : cette méthode non réversible permet d'effectuer des opérations sur les entrées du bloc fonction dans plusieurs endroits du programme. Par exemple :

Ligne	Instruction
1000	<code>CU %C1</code>
1074	<code>CD %C1</code>
1209	<code>R %C1</code>

Utilisez les instructions `BLK`, `OUT_BLK` et `END_BLK` pour la programmation réversible des blocs fonction :

- `BLK` : indique le début du bloc.
- `OUT_BLK` : permet de câbler directement les sorties du bloc.
- `END_BLK` : indique la fin du bloc.

NOTE : Seules les instructions de test et d'entrée sur le bloc correspondant peuvent être placées entre les instructions `BLK` et `OUT_BLK` (ou entre `BLK` et `END_BLK` lorsque `OUT_BLK` n'est pas programmé).

Exemple avec des sorties câblées

Cet exemple montre un bloc fonction Counter dans un programme avec des sorties câblées :

Réseau	Instruction
0	BLK %C8 LDF %I0.1 R LD %I0.1 AND %M0 CU OUT_BLK LD D AND %M1 ST %Q0.0 END_BLK

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Exemple sans aucune sortie câblée

Cet exemple montre la programmation réversible d'un bloc fonction Counter sans aucune sortie câblée :

Réseau	Instruction
0	BLK %C8 LDF %I0.1 R LD %I0.2 AND %M0 CU END_BLK
1	LD %C8.D AND %M1 ST %Q0.4

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Ajout d'un bloc fonction

Pour insérer un bloc fonction dans un programme en langage Schéma à contacts

Procédez comme suit :

Etape	Action
1	Créez un réseau en langage Schéma à contacts dans l'espace de travail de programmation de SoMachine Basic. Pour plus d'informations, consultez le document SoMachine Basic - Guide d'utilisation (voir <i>SoMachine Basic, Operating Guide</i>).
2	Cliquez sur le bouton Fonction dans la barre d'outils graphique située en haut de l'espace de travail de programmation. Résultat : la liste des objets logiciels disponibles s'affiche (consultez le tableau ci-dessous).
3	Sélectionnez le bloc fonction.
4	Déplacez-le vers la position souhaitée dans le réseau, puis cliquez sur le bouton gauche de la souris pour l'insérer.

Objets de bloc fonction disponibles

Ce tableau présente les objets logiciels disponibles :

Objet logiciel	Description
	Timer
	LIFO/FIFO Register
	Shift Bit Register
	Step Counter
123	Counter
1123	Fast Counter
11123	High Speed Counter

Objet logiciel	Description
	Drum Register
	Pulse
	Pulse Width Modulation
	Message

Configuration d'un bloc fonction

Pour configurer un bloc fonction dans un programme en langage Schéma à contacts

Procédez comme suit :

Etape	Action
1	Cliquez sur le libellé [Adresse] dans le bloc fonction. Une adresse par défaut s'affiche dans la zone de texte, par exemple "%TM0" pour un bloc fonction <i>Timer</i>. Pour modifier l'adresse par défaut, supprimez son dernier chiffre (identificateur de l'instance). La liste des adresses disponibles s'affiche. Sélectionnez l'adresse à utiliser pour identifier cette instance du bloc fonction. Les propriétés du bloc fonction s'affichent au centre de l'objet bloc fonction et dans le tableau Propriétés dans la moitié inférieure de l'espace de travail de programmation. A tout moment, double-cliquez dans le bloc fonction pour afficher ses propriétés.
2	Eventuellement, cliquez sur le libellé [Entrer un commentaire] dans le bloc fonction, puis saisissez une brève description du bloc fonction. Par exemple, Temporisateur d'impulsion.
3	Eventuellement, cliquez sur le libellé [Symbole] dans le bloc fonction et saisissez le nom du symbole à associer à ce bloc fonction. La liste des symboles existants, avec des noms commençant par le ou les caractères saisis, s'affiche. Cliquez sur celui que vous souhaitez utiliser. Pour créer un symbole de ce bloc fonction, saisissez le nom du symbole à créer, puis sélectionnez l'objet à associer au symbole. Pour plus d'informations sur l'utilisation des symboles, consultez le document SoMachine Basic - Guide d'utilisation (<i>voir SoMachine Basic, Operating Guide</i>).
4	Configurez les paramètres disponibles de chaque bloc fonction, comme indiqué dans la section "Paramètres" de la description d'un bloc fonction.

NOTE : Vous pouvez également afficher le tableau **Propriétés** en double-cliquant sur le bloc fonction dans un réseau.

Sous-chapitre 4.2

Timer (%TM)

Utilisation des blocs fonction Timer

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction `Timer`.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	141
Configuration	142
TON : Timer On-Delay	144
TOF : Timer Off-Delay	145
TP : Pulse Timer	146
Exemple de programmation	147

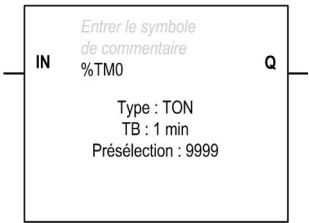
Description

Introduction

Le bloc fonction `Timer`  permet de spécifier un délai à respecter avant d'effectuer une opération (par exemple, déclencher un événement).

Illustration

Cette illustration décrit le bloc fonction `Timer`.



Entrées

Le bloc fonction `Timer` a l'entrée suivante :

Libellé	Description	Valeur
IN	Adresse de l'entrée (ou instruction)	Démarre le bloc fonction <code>Timer</code> lorsqu'un front montant (de type TON ou TP) ou descendant (de type TOF) est détecté.

Sorties

Le bloc fonction `Timer` a la sortie suivante :

Libellé	Description	Valeur
Q	Adresse de la sortie (%Tmi.Q)	Le bit associé %Tmi.Q est mis à 1 (selon le type de <code>Timer</code>) à l'expiration du bloc fonction <code>Timer</code> .

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction ([voir page 139](#)) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction `Timer` a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet <code>Timer</code> (%T <i>Mi</i>)	Un programme ne peut contenir qu'un nombre limité d'objets <code>Timer</code> . Consultez le Guide de programmation de la plate-forme concernée pour connaître le nombre maximum d'objets <code>Timer</code> .
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Type	Type de <code>Timer</code>	Un des types suivants : ● TON (voir page 144) : Timer on-Delay (par défaut) ● TOF (voir page 145) : Timer off-Delay ● TP (voir page 146) : Pulse timer (monostable)
Base	Base de temps	Base de temps du temporisateur. Plus la base de temps du bloc fonction <code>Timer</code> est petite, plus ce dernier est précis : ● 1 ms (uniquement pour les 6 premières instances) ● 10 ms ● 100 ms ● 1 s ● 1 min (par défaut)
Présélection	Valeur de présélection	0 à 9999. La valeur par défaut est 9999. Période du temporisateur = Présélection x base de temps Retard du temporisateur = Présélection x base de temps Cette valeur de présélection configurée peut être lue, testée et modifiée à l'aide de l'objet associé %T <i>Mi</i> . P.
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Objets

Le bloc fonction `Timer` a les objets suivants :

Objet	Description	Valeur
<code>%Tmi.P</code>	Valeur de présélection	Consultez la description dans le tableau Paramètres ci-avant.
<code>%Tmi.V</code>	Valeur courante	Mot qui s'incrémente de 0 à la valeur de présélection (<code>%Tmi.P</code>) lorsque le temporisateur est en cours d'exécution. Le programme peut lire et tester cette valeur, mais pas l'écrire. Cette valeur peut être modifiée dans une table d'animation.
<code>%Tmi.Q</code>	Sortie du bloc fonction <code>Timer</code>	Consultez la description dans le tableau Sorties ci-avant.

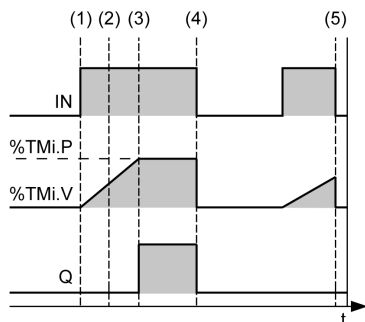
TON : Timer On-Delay

Introduction

Le type de temporisateur TON *On-Delay Timer* permet de réguler les actions de délai à l'activation. Le logiciel permet de programmer ce délai.

Chronogramme

Ce schéma illustre le fonctionnement du bloc fonction *Timer* de type TON.



- (1) Le *Timer* débute sur le front montant de l'entrée IN.
- (2) La valeur actuelle $\%TMI.V$ augmente de 1 à $\%TMI.P$, par incréments d'une unité à chaque pulsation du paramètre de base de temps TB
- (3) Le bit de sortie $\%TMI.Q$ est réglé sur 1 lorsque la valeur actuelle atteint la valeur de présélection $\%TMI.P$.
- (4) Le bit de sortie $\%TMI.Q$ conserve la valeur 1 tant que la valeur de l'entrée IN est à 1.
- (5) Lorsqu'un front descendant est détecté sur l'entrée IN, le bloc fonction *Timer* est arrêté, même si le *Timer* n'a pas atteint $\%TMI.P$. $\%TMI.V$ est mis à 0.

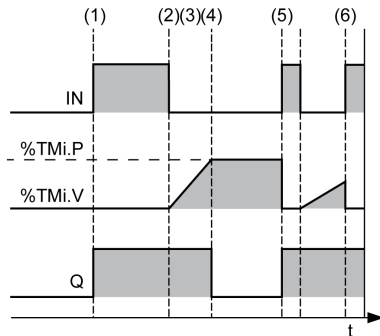
TOF : Timer Off-Delay

Introduction

Utilisez le type TOF (Off-Delay Timer) de bloc fonction `Timer` pour réguler les actions à délai de désactivation. Le logiciel permet de programmer ce délai.

Chronogramme

Ce schéma illustre le fonctionnement du bloc fonction `Timer` de type TOF.



- (1) Lors d'un front montant de l'entrée `IN`, `%TMI.Q` est mis à 1.
- (2) Le bloc fonction `Timer` démarre sur le front descendant de l'entrée `IN`.
- (3) La valeur actuelle `%TMI.V` augmente jusqu'à la valeur de présélection `%TMI.P`, par incréments d'une unité, à chaque impulsion du paramètre de base de temps (TB).
- (4) Le bit de sortie `%TMI.Q` est réglé sur 0 lorsque la valeur actuelle atteint la valeur de présélection `%TMI.P`.
- (5) Lors d'un front montant de l'entrée `IN`, `%TMI.V` est mis à 0.
- (6) Lors d'un front montant de l'entrée `IN`, `%TMI.V` est mis à 0, même si la valeur de présélection n'est pas atteinte.

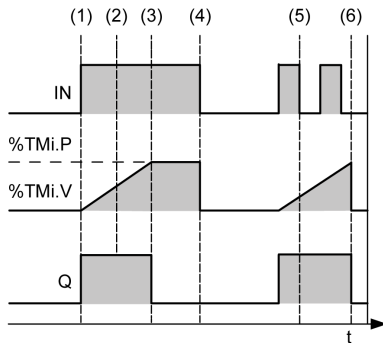
TP : Pulse Timer

Introduction

Le bloc fonction `Timer` de type TP (Pulse Timer) permet de créer des impulsions d'une durée précise. Le logiciel permet de programmer ce délai.

Chronogramme

Ce schéma illustre le fonctionnement du `Timer` de type TP.



- (1) Le `Timer` débute sur le front montant de l'entrée `IN`. La valeur actuelle `%TMI.V` est mise à 0 si le `Timer` n'a pas encore démarré, et `%TMI.Q` est mis à 1 lorsque le `Timer` démarre.
- (2) La valeur actuelle `%TMI.V` du bloc fonction `Timer` passe de 0 à la valeur de présélection `%TMI.P`, par incrément d'une unité à chaque impulsion du paramètre de base de temps (`TB`).
- (3) Le bit de sortie `%TMI.Q` est réglé sur 0 lorsque la valeur actuelle atteint la valeur de présélection `%TMI.P`.
- (4) La valeur actuelle `%TMI.V` est mise à 0 lorsque `%TMI.V` égale `%TMI.P` et que l'entrée `IN` revient à 0.
- (5) Ce type de bloc fonction `Timer` ne peut pas être remis à 0.
- (6) Lorsque `%TMI.V` est égal à `%TMI.P` et que l'entrée `IN` a pour valeur 0, `%TMI.Q` est mis à 0.

Exemple de programmation

Introduction

Les blocs fonction `Timer` ont les modes de fonctionnement suivants :

- TON (Timer On-Delay) (*voir page 144*) : permet de spécifier un délai entre l'activation d'une entrée et la commutation d'un capteur de sortie.
- TOF (Timer Off-Delay) (*voir page 145*) : permet de spécifier un délai entre une sortie associée à un capteur qui n'est plus détecté et la commutation de la sortie correspondante.
- TP (Timer - Pulse) (*voir page 146*) : permet de créer une impulsion d'une durée précise.

Les retards ou périodes d'impulsion des blocs fonction `Timers` sont programmables et configurables dans le logiciel

Programmation

Voici un exemple de bloc fonction `Timer` avec des instructions réversibles :

Réseau	Instruction réversible
0	BLK %TM0 LD %M0 IN OUT _BLK LD Q ST %Q0.0 END _BLK
1	LD [%TM0.V<400] ST %Q0.1
2	LD [%TM0.V>=400] ST %Q0.2

Voici un exemple du même bloc fonction `Timer` avec des instructions non réversibles :

Réseau	Instruction non réversible
0	LD %M0 IN %TM0
1	LD %TM0.Q ST %Q0.0
2	LD [%TM0.V<400] ST %Q0.1
3	LD [%TM0.V>=400] ST %Q0.2

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Sous-chapitre 4.3

LIFO/FIFO Register (%R)

Utilisation des blocs fonction LIFO/FIFO Register

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction LIFO/FIFO Register.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	149
Configuration	151
Fonction du registre LIFO	153
Fonctionnement du registre FIFO	154
Exemple de programmation	155

Description

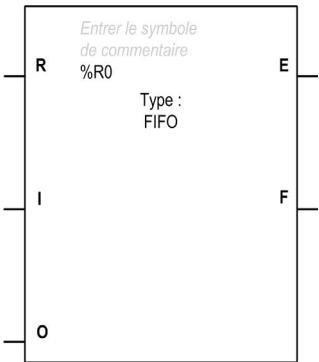
Introduction

Le bloc fonction `LIFO/FIFO Register`  est un bloc mémoire capable de stocker jusqu'à 16 mots de 16 bits de deux manières différentes :

- dans une file d'attente, appelée "FIFO" (First In, First Out – Premier entré, Premier sorti) ;
- dans une pile, appelée "LIFO" (Last In, First Out – Dernier entré, Premier sorti).

Illustration

Cette illustration décrit le bloc fonction `LIFO/FIFO Register`.



Entrées

Le bloc fonction `LIFO/FIFO Register` a les entrées suivantes :

Libellé	Description	Valeur
R	Entrée (ou instruction) de RAZ	A l'état 1, initialise le bloc fonction <code>LIFO/FIFO Register</code> .
I	Entrée (ou instruction) de stockage	En cas de front montant, stocke le contenu du mot associé <code>%Ri.I</code> dans le bloc fonction <code>LIFO/FIFO Register</code> .
O	Entrée (ou instruction) de récupération	En cas de front montant, charge un mot de données du bloc fonction <code>LIFO/FIFO Register</code> dans le mot associé <code>%Ri.O</code> .

Sorties

Le bloc fonction LIFO/FIFO Register a les sorties suivantes :

Libellé	Description	Valeur
E	Sortie vide (%Ri.E)	Le bit associé %Ri.E indique que le bloc fonction LIFO/FIFO Register est vide. La valeur de %Ri.E peut être testée, par exemple dans une table d'animation ou avec une instruction.
F	Sortie pleine (%Ri.F)	Le bit associé %Ri.F indique que le bloc fonction LIFO/FIFO Register est saturé. La valeur de %Ri.F peut être testée, par exemple dans une table d'animation ou avec une instruction.

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction (*voir page 139*) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction LIFO/FIFO Register a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet LIFO/FIFO Register	Un programme ne peut contenir qu'un nombre limité d'objets LIFO/FIFO Register. Consultez le Guide de programmation de la plate-forme matérielle pour connaître le nombre maximum de registres.
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Type	Type de LIFO/FIFO Register	FIFO (file d'attente) ou LIFO (pile).
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Objets

Le bloc fonction LIFO/FIFO Register a les objets suivants :

Objet	Description	Valeur
%Ri.I	Mot d'entrée LIFO/FIFO Register	Peut être lu, testé et écrit. Peut être modifié dans une table d'animation.
%Ri.O	Mot de sortie LIFO/FIFO Register	Peut être lu, testé et écrit. Peut être modifié dans une table d'animation.
%Ri.E	Sortie vide	Consultez le tableau Sorties ci-dessus.
%Ri.F	Sortie pleine	Consultez le tableau Sorties ci-dessus.

Cas particuliers

Ce tableau contient une liste de cas particuliers de programmation du bloc fonction Bloc fonction LIFO/FIFO Register :

Cas particulier	Description
Effet d'un redémarrage à froid (%S0=1) ou INIT	Initialise le contenu du bloc fonction LIFO/FIFO Register. Le bit de sortie %Ri.E associé à la sortie E est mis à 1.
Effet d'un redémarrage à chaud (%S1=1) ou d'un arrêt du contrôleur	Aucun effet sur la valeur courante du bloc fonction LIFO/FIFO Register, ni sur l'état de ses bits de sortie.

NOTE : L'effet d'**INIT** est identique à celui de %S0=1.

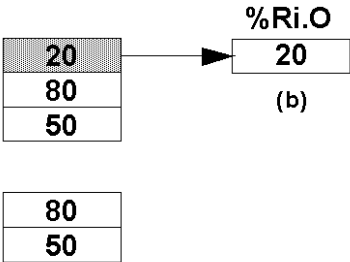
Fonction du registre LIFO

Introduction

En fonctionnement LIFO (Last In, First Out), la dernière information entrée est la première à être récupérée.

Fonctionnement

Ce tableau décrit le fonctionnement LIFO :

Etape	Description	Exemple
1	Stockage : Lorsqu'une demande de stockage est reçue (front montant sur l'entrée I ou activation de l'instruction I), le contenu du mot d'entrée $\%Ri.I$ est stocké en haut de la pile (fig. a). Lorsque la pile est pleine (sortie $E=1$), aucun autre stockage n'est possible.	Stockage du contenu de $\%Ri.I$ en haut de la pile. 
2	Récupération : Lors de la réception d'une demande de récupération (front montant sur l'entrée O ou activation de l'instruction O), le mot de données le plus haut (le dernier entré) est chargé dans le mot $\%Ri.O$ (fig. b). Lorsque le bloc fonction LIFO/FIFO Register est vide (sortie $E=1$), aucune autre récupération n'est possible. Le mot de sortie $\%Ri.O$ n'est pas modifié et conserve sa valeur.	Récupération du mot de données au plus haut de la pile. 
3	Résultat : La pile peut être réinitialisée à tout moment (état 1 de l'entrée R ou activation de l'instruction R). L'élément indiqué par le pointeur est alors le plus haut dans la pile.	—

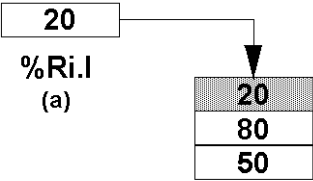
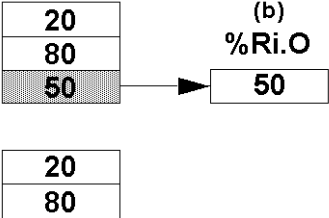
Fonctionnement du registre FIFO

Introduction

En fonctionnement FIFO (First In, First Out), la première information entrée est la première à être récupérée.

Fonctionnement

Ce tableau décrit le fonctionnement FIFO :

Etape	Description	Exemple
1	Stockage : Lorsqu'une demande de stockage est reçue (front montant sur l'entrée I ou activation de l'instruction I), le contenu du mot d'entrée $\%Ri.I$ est stocké en haut de la file d'attente (fig. a). Lorsque la file d'attente est pleine (sortie $F=1$), aucun autre stockage n'est possible.	Stockage du contenu de $\%Ri.I$ en haut de la file d'attente. 
2	Récupération : Lorsqu'une demande de récupération est reçue (front montant sur l'entrée O ou activation de l'instruction O), le mot de données le plus bas dans la file d'attente est chargé dans le mot de sortie $\%Ri.O$ et le contenu du registre LIFO/FIFO (LIFO/FIFO Register) est déplacé d'une place vers le bas dans la file d'attente (fig. b). Lorsque le bloc fonction LIFO/FIFO Register est vide (sortie $E=1$), aucune autre récupération n'est possible. Le mot de sortie $\%Ri.O$ n'est pas modifié et conserve sa valeur.	Récupération de la première information qui est ensuite chargée dans $\%Ri.O$. 
3	Résultat : La file d'attente peut être réinitialisée à tout moment (état 1 sur l'entrée R ou activation de l'instruction R).	—

Exemple de programmation

Introduction

L'exemple de programmation suivant montre le contenu d'un mot mémoire (%MW34) chargé dans un bloc fonction LIFO/FIFO Register (%R2.I) en cas de réception d'une requête de stockage (%I0.2) si le bloc fonction LIFO/FIFO Register %R2 n'est pas saturé (%R2.F = 0). La requête de stockage dans le LIFO/FIFO Register est émise par %M1. La demande de récupération est confirmée par l'entrée %I0.3, et %R2.O est chargé dans %MW20 si le registre n'est pas vide (%R2.E = 0).

Programmation

Voici un exemple de LIFO/FIFO Register avec des instructions réversibles :

Réseau	Instruction réversible
0	BLK %R2 LD %M1 I LD %I0.3 ANDN %R2.E O END_BLK
1	LD %I0.3 [%MW20:=%R2.O]
2	LD %I0.2 ANDN %R2.F [%R2.I:=%MW34] ST %M1

Voici un exemple du même bloc fonction LIFO/FIFO Register avec des instructions non réversibles :

Réseau	Instruction non réversible
0	LD %M1 I %R2
1	LD %I0.3 ANDN %R2.E O %R2
2	LD %I0.3 [%MW20:=%R2.O]
3	LD %I0.2 ANDN %R2.F [%R2.I:=%MW34] ST %M1

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Sous-chapitre 4.4

Shift Bit Register (%SBR)

Utilisation de blocs fonction Shift Bit Register

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction Shift Bit Register.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	158
Configuration	159
Exemple de programmation	161

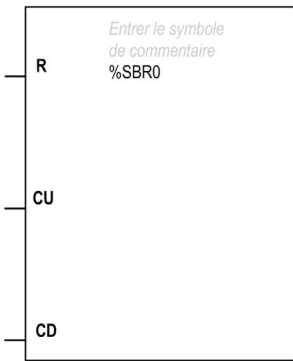
Description

Introduction

Le bloc fonction Shift Bit Register  décale les bits de données binaires (0 ou 1) vers la gauche ou vers la droite.

Illustration

Cette illustration décrit le bloc fonction Shift Bit Register :



La valeur courante du bloc fonction Shift Bit Register s'affiche au centre de ce dernier :

- Valeur décimale (par exemple, 7)
- Valeur binaire (par exemple, 111)
- Valeur hexadécimale (par exemple, 16#7)

Entrées

Le bloc fonction Shift Bit Register a les entrées suivantes :

Libellé	Description	Valeur
R	Entrée (ou instruction) de RAZ	Lorsque le paramètre de fonction R est à 1, les bits 0 à 15 du registre %SBRi.j sont réglés sur 0.
CU	Entrée (ou instruction) de décalage à gauche	En cas de front montant, décale un bit du registre vers la gauche.
CD	Entrée (ou instruction) de décalage à droite	En cas de front montant, décale un bit du registre vers la droite.

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction ([voir page 139](#)) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction `Shift Bit Register` a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet Shift Bit Register	Un programme ne peut contenir qu'un nombre limité d'objets <code>Shift Bit Register</code> . Consultez le Guide de programmation de la plate-forme matérielle pour connaître le nombre maximum de registres.
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

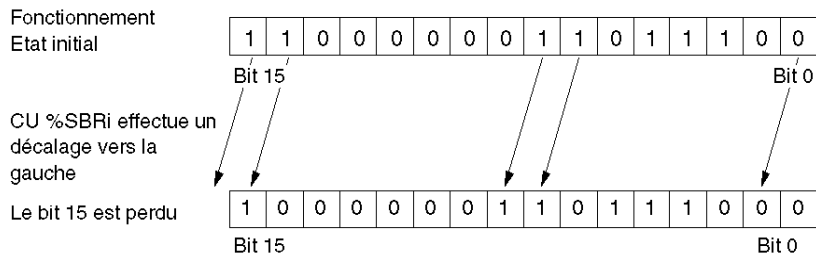
Objets

Le bloc fonction `Shift Bit Register` a les objets suivants :

Objet	Description	Valeur
<code>%SBRi</code>	Numéro de registre	0 à 7 Peut être modifié dans une table d'animation.
<code>%SBRi.j</code>	Bit de registre	Les bits 0 à 15 (j = 0 à 15) du registre à décalage peuvent être testés par une instruction de test et écrit à l'aide d'une instruction d'affectation.

Fonctionnement

L'illustration suivante présente une configuration de bits avant et après une opération de décalage :



Ceci vaut également pour une requête de décalage d'un bit vers la droite (bit 15 à bit 0) à l'aide de l'instruction `CD`. Le bit 0 est perdu.

Si un registre de 16 bits n'est pas adapté, il est possible d'utiliser le programme pour utiliser plusieurs registres en cascade.

Cas particuliers

Ce tableau contient une liste de cas particuliers de programmation du bloc fonction `Shift Bit Register` :

Cas particulier	Description
Effet d'un redémarrage à froid (%S0=1)	Règle tous les bits du mot de registre sur 0.
Effet d'un redémarrage à chaud (%S1=1)	N'a aucun effet sur les bits du mot de registre.

Exemple de programmation

Introduction

Le bloc fonction `Shift Bit Register` décale les bits de données binaires (0 ou 1) vers la gauche ou vers la droite.

Programmation

Dans l'exemple suivant, un bit est décalé vers la gauche à chaque seconde et le bit 0 prend l'état du bit 15.

Dans les instructions réversibles :

Réseau	Instruction réversible
0	BLK %SBR0 LD %S6 CU END_BLK
1	LD %SBR0.15 ST %SBR0.0

Dans les instructions non réversibles :

Réseau	Instruction non réversible
0	LD %S6 CU %SBR0
1	LD %SBR0.15 ST %SBR0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (*voir page 16*).

Sous-chapitre 4.5

Step Counter (%SC)

Utilisation des blocs fonction Step Counter

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction `Step Counter`.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	163
Configuration	164
Exemple de programmation	165

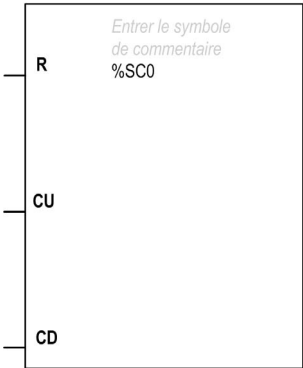
Description

Introduction

Un bloc fonction `Step Counter`  fournit une série d'étapes auxquelles des actions peuvent être affectées. Le passage d'une étape à l'autre dépend d'événements internes ou externes. Chaque fois qu'une étape est active, le bit associé (bit `%SCi.j` de `Step Counter`) est mis à 1. Une seule étape d'un bloc fonction `Step Counter` peut être active à la fois.

Illustration

Cette illustration décrit le bloc fonction `Step Counter` :



Entrées

Le bloc fonction `Step Counter` a les entrées suivantes :

Libellé	Description	Valeur
R	Entrée (ou instruction) de RAZ	Lorsque le paramètre de fonction <code>R</code> est à 1, le bloc fonction <code>Step Counter</code> est réinitialisé.
CU	Entrée (ou instruction) d'incréméntation	En cas de front montant, elle incrémente le bloc fonction <code>Step Counter</code> d'une étape.
CD	Entrée (ou instruction) de décrémentation	En cas de front montant, elle décrémente le bloc fonction <code>Step Counter</code> d'une étape.

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction (voir page 139) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction `Step Counter` a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet <code>Step Counter</code>	Un programme ne peut contenir qu'un nombre limité d'objets <code>Step Counter</code> . Consultez le Guide de programmation de la plate-forme matérielle pour connaître le nombre maximum de blocs fonction <code>Step Counter</code> .
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Objets

Le bloc fonction `Step Counter` a l'objet suivant :

Objet	Description	Valeur
<code>%Sci.j</code>	Bit du bloc fonction <code>Step Counter</code>	Les bits de bloc fonction <code>Step Counter</code> 0 à 255 ($j = 0$ à 255) peuvent être testés par une instruction logique de chargement et écrits à l'aide d'une instruction d'affectation. Peut être modifié dans une table d'animation.

Cas particulier

Ce tableau contient une liste de cas particuliers d'utilisation du bloc fonction `Step Counter` :

Cas particulier	Description
Effet d'un redémarrage à froid ($\%S0=1$)	Initialise le bloc fonction <code>Step Counter</code> .
Effet d'un redémarrage à chaud ($\%S1=1$)	N'a aucun effet sur le bloc fonction <code>Step Counter</code> .

Exemple de programmation

Introduction

Voici un exemple de bloc fonction Step Counter.

- Le bloc Step Counter 0 est décrémenté par l'entrée %I0.1.
- Le bloc Step Counter 0 est incrémenté par l'entrée %I0.2.
- Le bloc Step Counter 0 est remis à 0 par l'entrée %I0.3 ou lorsqu'il arrive à l'étape 3.
- L'étape 0 commande la sortie %Q0.1, l'étape 1 commande la sortie %Q0.2 et l'étape 2 commande la sortie %Q0.3.

Programmation

Voici un exemple de bloc fonction Step Counter avec des instructions réversibles :

Réseau	Instruction réversible
0	BLK %SC0 LD %SC0.3 OR %I0.3 R LD %I0.2 CU LD %I0.1 CD END_BLK
1	LD %SC0.0 ST %Q0.1
2	LD %SC0.1 ST %Q0.2
3	LD %SC0.2 ST %Q0.3

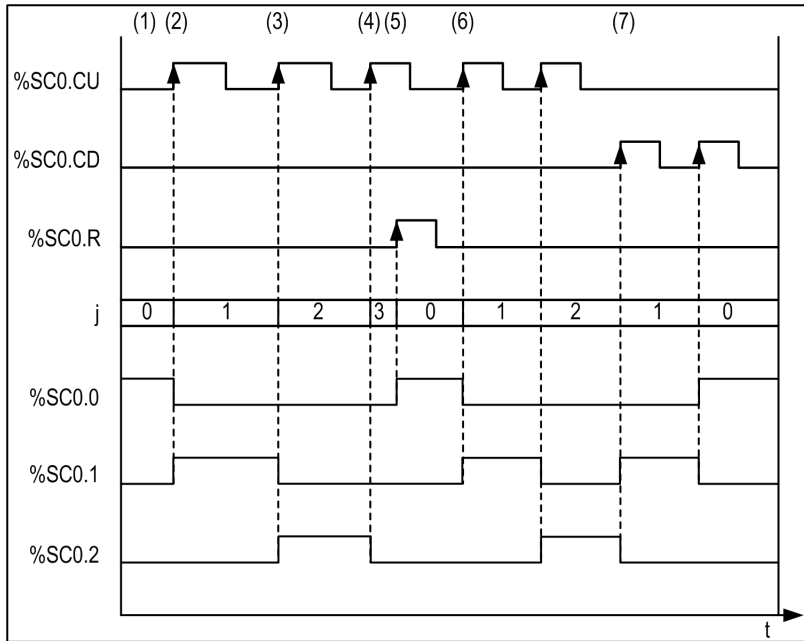
Voici un exemple de bloc fonction `Step Counter` avec des instructions non réversibles :

Réseau	Instruction non réversible
0	LD %SC0.3 OR %I0.3 R %SC0
1	LD %I0.2 CU %SC0
2	LD %I0.1 CD %SC0
3	LD %SC0.0 ST %Q0.1
4	LD %SC0.1 ST %Q0.2
5	LD %SC0.2 ST %Q0.3

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Ce schéma illustre le fonctionnement du bloc fonction Step Counter :



- (1) L'étape 0 étant active, %SC0.0 est mis à 1.
- (2) Lors du front montant de l'entrée CU, l'étape est incrémentée et les sorties sont mises à jour.
- (3) L'étape est incrémentée et les sorties sont mises à jour.
- (4) L'étape 3 étant active, l'entrée **Reset** est active après un cycle d'UC.
- (5) Lorsque **Reset** est active, l'étape courante est mise à 0 et l'entrée de réinitialisation est mise à 0 après un cycle d'UC.
- (6) L'étape courante est incrémentée lors du front montant de l'entrée CU.
- (7) Lors du front montant de l'entrée CD, l'étape est décrémentée et les sorties sont mises à jour.

Sous-chapitre 4.6

Counter (%C)

Utilisation des blocs fonction Counter

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction Counter.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	169
Configuration	171
Exemple de programmation	174

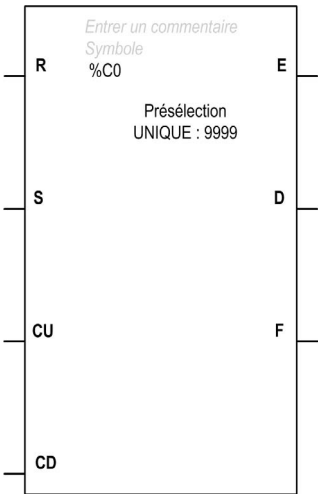
Description

Introduction

Le bloc fonction Counter **123** permet de compter ou de décompter des événements. Ces deux opérations peuvent être réalisées simultanément.

Illustration

Cette illustration décrit le Counter.



Entrées

Le bloc fonction Counter possède les entrées suivantes :

Libellé	Description	Valeur
R	Entrée (ou instruction) de RAZ	Met le compteur (%Ci.V) à 0 lorsque l'entrée de réinitialisation (R) est mise à 1.
S	Entrée (ou instruction) de configuration	Met le compteur (%Ci.V) à la valeur de présélection (%Ci.P) lorsque l'entrée de configuration (S) est mise à 1.
CU	Comptage croissant	Incrémente la valeur du compteur (%Ci.V) de 1 lors d'un front montant sur l'entrée de comptage croissant (CU).
CD	Comptage décroissant	Décrément la valeur du compteur (%Ci.V) de 1 lors d'un front montant sur l'entrée de comptage décroissant (CD).

Sorties

Le bloc fonction **Counter** a les sorties suivantes :

Libellé	Description	Valeur
E	Dépassement du comptage décroissant	Le bit associé %Ci.E (compteur vide) est mis à 1 lorsque le compteur atteint la valeur 0. En cas de décrémentation, la valeur du compteur passe à 9999.
D	Sortie prédéfinie atteinte	Le bit associé %Ci.D (comptage terminé) est mis à 1 lorsque %Ci.V = %Ci.P.
F	Dépassement du comptage croissant	Le bit associé %Ci.F=1 (compteur saturé), lorsque %Ci.V passe de 9999 à 0 (mis à 1 lorsque %Ci.V atteint 0 et remis à 0 si le bloc fonction Counter poursuit le comptage croissant).

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction ([voir page 139](#)) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction `Counter` a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet <code>Counter</code>	Un programme ne peut contenir qu'un nombre limité d'objets <code>Counter</code> . Pour connaître le nombre maximum de compteurs, consultez le <i>Guide de programmation</i> de votre contrôleur.
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Présélection	Valeur de présélection	Valeurs acceptées par la valeur de présélection [0 à 9999]. La valeur par défaut est 9999. Cette valeur configurée peut être lue, testée et modifiée à l'aide de l'objet associé <code>%Ci.P</code> .
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Objets

Le bloc fonction `Counter` a les objets suivants :

Objet	Description	Valeur
<code>%Ci.V</code>	Valeur actuelle du bloc fonction <code>Counter</code>	Ce mot est incrémenté ou décrémenté en fonction des entrées (ou des instructions) CU et CD (voir le tableau Entrées (voir page 169)). Ne peut être que lu. Peut être modifié dans une table d'animation.
<code>%Ci.P</code>	Valeur de présélection	Consultez le tableau Paramètres (voir page 171). Peut être modifié dans une table d'animation.
<code>%Ci.E</code>	Vide	Consultez le tableau Sorties (voir page 170). Peut être modifié dans une table d'animation.
<code>%Ci.D</code>	Terminé	Consultez le tableau Sorties (voir page 170). Peut être modifié dans une table d'animation.
<code>%Ci.F</code>	Complet	Consultez le tableau Sorties (voir page 170). Peut être modifié dans une table d'animation.

Opérations

Ce tableau décrit les principales étapes des opérations du bloc fonction Counter :

Fonctionnement	Action	Résultat
Réinitialisation	L'entrée R est mise à 1 (ou instruction R activée).	Force la remise à 0 de la valeur %Ci.V. Les sorties %Ci.E, %Ci.D et %Ci.F sont réglées sur 0. L'entrée réinitialisée est prioritaire.
Définition	Si l'entrée S est mise à 1 (ou instruction S activée) et si l'entrée de réinitialisation est à 0 (ou instruction R inactive).	La valeur courante %Ci.V prend la valeur de %Ci.P et la sortie %Ci.D est mise à 1.
Comptage croissant Comptage décroissant	Un front montant apparaît sur l'entrée de comptage croissant CU (ou instruction CU activée).	La valeur courante de %Ci.V est augmentée d'une unité.
	La valeur courante de %Ci.V est égale à la valeur de présélection de %Ci.P.	Le bit de sortie "présélection atteinte" %Ci.D passe à 1.
	La valeur courante de %Ci.V passe de 9999 à 0.	Le bit de sortie %Ci.F (débordement de comptage croissant) passe à 1.
	Si le bloc fonction Counter continue son comptage croissant.	Le bit de sortie %Ci.F (débordement de comptage croissant) est remis à 0.
	Un front montant apparaît sur l'entrée de comptage décroissant CD (ou instruction CD activée).	La valeur courante de %Ci.V est diminuée d'une unité.
	La valeur courante de %Ci.V passe de 0 à 9999.	Le bit de sortie %Ci.E (débordement de comptage décroissant) passe à 1.
	Si le bloc fonction Counter continue son comptage décroissant.	Le bit de sortie %Ci.F (débordement de comptage décroissant) est remis à 0.

Cas particuliers

Le tableau suivant présente une liste de cas particuliers d'utilisation et de configuration du bloc fonction `Counter` :

Cas particulier	Description
Effet d'un redémarrage à froid (%S0=1) ou INIT	• La valeur courante de %Ci.V est mise à 0. • Les bits de sortie %Ci.E, %Ci.D, et %Ci.F sont mis à 0. • La valeur de présélection est initialisée avec la valeur définie lors de la configuration.
Effet d'un redémarrage à chaud (%S1=1) d'un contrôleur	Aucun effet sur la valeur courante du bloc fonction <code>Counter</code> (%Ci.V).
Effet de la modification de la valeur de présélection de %Ci.P	La modification de la valeur de présélection à l'aide d'une instruction ou d'un réglage ne prend effet qu'au moment du traitement du bloc par l'application (activation de l'une des entrées).

NOTE : L'effet d'INIT est identique à celui de %S0=1.

Exemple de programmation

Introduction

L'exemple suivant illustre un bloc fonction Counter permettant de compter jusqu'à 5000 articles. Chaque impulsion sur l'entrée %I0.2 (lorsque le bit interne %M0 est mis à 1) incrémente la valeur du bloc fonction Counter %C8 d'une unité, jusqu'à sa valeur de présélection finale (bit %C8.D=1). Le compteur est remis à zéro par l'entrée %I0.1.

Programmation

Voici un exemple de Counter avec des instructions réversibles :

Réseau	Instruction réversible
0	BLK %C8 LD %I0.1 R LD %I0.2 AND %M0 CU END_BLK
1	LD %C8.D ST %Q0.0

Voici un exemple du même bloc fonction Counter avec des instructions non réversibles :

Réseau	Instruction non réversible
0	LD %I0.1 R %C8
1	LD %I0.2 AND %M0 CU %C8
2	LD %C8.D ST %Q0.0

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

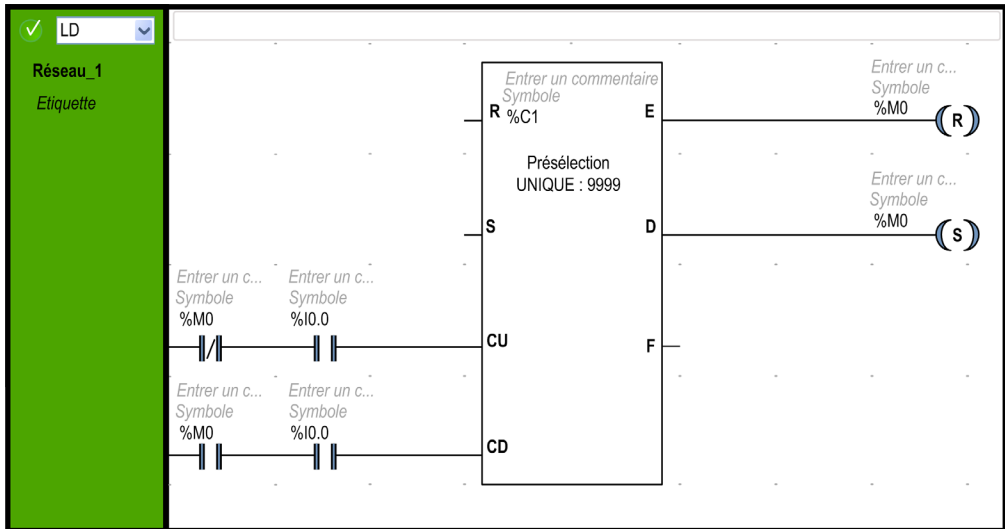
Configuration

Les paramètres suivants doivent être saisis lors de la configuration :

Valeur **Présélection** (%Ci.P) : 5000 dans cet exemple.

Exemple de compteur croissant/décroissant

Cette illustration montre un bloc fonction Counter.



Dans cet exemple, %M0 donne l'ordre d'incrémement (%M0 = False) et de décrémentation (%M0 = True). Le compteur compte le front montant de %I0.0. Si %M0 a pour valeur False, lors de chaque front sur %I0.0, %C1.V est incrémenté jusqu'à atteindre la valeur de présélection %C1.P, et l'indicateur Done %C1.D passe à TRUE. La sortie %C1.D configure %M0 et bascule l'instruction vers l'ordre de décrémentation. Ensuite, à chaque front sur %I0.0, %C1.V est décrémentation jusqu'à atteindre 0. L'indicateur Empty (%C1.E) est activé et réinitialise %M0 (ordre d'incrémement).

Sous-chapitre 4.7

Bloc fonction Fast Counter (%FC)

Fast Counter

Présentation

Consultez le *Guide de programmation* de votre contrôleur.

Sous-chapitre 4.8

Bloc fonction High Speed Counter (%HSC)

High Speed Counter

Présentation

Consultez le *Guide de programmation* de votre contrôleur.

Sous-chapitre 4.9

Drum Register (%DR)

Utilisation des blocs fonction Drum Register

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction `Drum Register`.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	179
Configuration	180
Exemple de programmation	183

Description

Introduction

Le bloc fonction `Drum Register`  fonctionne selon un principe similaire à celui d'un bloc fonction `Drum Register` électromécanique qui change d'étape selon des événements externes. A chaque étape, le point haut d'une came émet une commande exécutée par le contrôleur. Dans le cas d'un bloc fonction `Drum Register`, ces points hauts sont symbolisés par l'état 1 de chaque étape et sont affectés à des bits de sortie `%Qi.j` ou des bits internes `%Mi`.

Illustration

Cette illustration décrit le `Drum Register` en mode hors ligne.



Etapas Affiche le nombre total d'étapes configurées dans l'**Assistant Drum**.

Etape S'affiche en mode hors ligne lors de la création d'un bloc. En mode en ligne, affiche le numéro de l'étape courante.

Entrées

Le `Drum Register` a les entrées suivantes :

Libellé	Description	Valeur
R	Retour à l'étape (ou à l'instruction) 0	A l'état 1, règle le bloc fonction Drum Register sur l'étape 0.
U	Entrée (ou instruction) d'avancement	En cas de front montant, provoque le passage du bloc fonction Drum Register à l'étape suivante et met à jour les bits de contrôle.

Sorties

Le `Drum Register` a la sortie suivante :

Libellé	Description	Valeur
F	Sortie (<code>%DRi.F</code>)	Indique que l'étape courante est égale à la dernière étape définie. Le bit associé <code>%DRi.F</code> peut être testé.

Configuration

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction (voir page 139) et lisez la description des Modes d'allocation de mémoire dans le Guide d'utilisation de SoMachine Basic.

Le bloc fonction `Drum Register` a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet <code>Drum Register</code>	Un programme ne peut contenir qu'un nombre limité d'objets <code>Drum Register</code> . Pour connaître le nombre maximum d'objets <code>Drum Register</code> , consultez le <i>Guide de programmation</i> de votre contrôleur.
Symbole	Symbole	Symbole associé à cet objet. Pour plus d'informations, consultez la section Définition et utilisation des symboles dans le document SoMachine Basic - Guide d'utilisation.
Configuration	Assistant Drum	Nombre d'étapes : 1...8. Sorties ou bits mémoire associés aux étapes : Bit 0 à Bit 15 .
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Objets

Le bloc fonction `Drum Register` a l'objet suivant :

Objet	Description	Valeur
<code>%DRi.S</code>	Numéro de l'étape courante	$0 \leq \%DRi.S \leq 7$. Mot pouvant être lu et écrit. La valeur écrite doit être une valeur décimale immédiate. Une fois écrite, la valeur sera prise en compte à la prochaine exécution du bloc fonction. Peut être modifié dans une table d'animation.
<code>%DRi.F</code>	Complet	Consultez le tableau Sorties (voir page 179).

Fonctionnement

Le bloc fonction `Drum Register` comprend :

- une matrice de données constantes (les cames) organisées en huit étapes (0 à 7) et 16 bits de données (état de l'étape) disposés en colonnes numérotées de 0 à 15.
- Une liste de bits de contrôle est associée à une sortie configurée (`%Qi.j`) ou un mot mémoire (`%Mi`). Pendant l'étape en cours, les bits de contrôle prennent les états binaires définis pour cette étape.

L'exemple suivant résume les principales caractéristiques du bloc fonction Drum Register:

Assistant Drum

Nombre d'étapes : 8

Etape 0

Etape 1

Etape 2

Etape 3

Etape 4

Etape 5

Etape 6

Etape 7

Bit 0

%Q0.0

Bit 1

%Q0.1

Bit 2

%Q0.2

Bit 3

%Q0.3

Bit 4

%Q0.4

Bit 5

%Q0.5

Bit 6

Bit 7

Bit 8

NOTE : La configuration est également possible sur les bits mémoire (%Mi).

Réseau	Instruction
0	BLK %DR0 LD %M10 R LD %M11 U END_BLK

Créez les entrées suivantes dans une table d'animation : %M10, %M11, %DR0, %Q0.0 à %Q0.5.

Observez les valeurs de %DR0.S, %DR0.F, %Q0.0 à %Q0.5 lors de la modification de la valeur de %M11 (évolution du Drum) et de %M10 (réinitialisation du Drum). Ensuite, examinez le cas du dépassement (retour à l'étape 0).

Cas particuliers

Ce tableau contient une liste de cas particuliers de fonctionnement du bloc fonction `Drum Register` :

Cas particulier	Description
Effets d'un redémarrage à froid (%S0=1)	Réinitialise le bloc fonction <code>Drum Register</code> à l'étape 0 (mise à jour des bits de contrôle).
Effet d'un redémarrage à chaud (%S1=1)	Met à jour les bits de contrôle après l'étape courante.
Effet d'un saut de programme	L'arrêt de la scrutation du bloc fonction <code>Drum Register</code> implique que les bits de contrôle conservent leur dernier état.
Mise à jour des bits de contrôle	Survient uniquement en cas de changement d'étape ou lors d'un démarrage à froid ou à chaud.

Exemple de programmation

Introduction

Voici un exemple de programmation d'un bloc fonction `Drum Register` configuré de telle sorte qu'aucune des commandes ne soit définie à l'étape 0, mais qu'elles le soient pour les étapes 1 à 6 sur les sorties `%Q0.0` à `%Q0.5` respectivement (consultez la section Configuration [\(voir page 185\)](#)).

Les 6 premières sorties `%Q0.0` à `%Q0.5` sont activées successivement chaque fois que l'entrée `%I0.1` est mise à 1. L'entrée `%I0.0` remet les éléments suivants à 0 lorsqu'elle est élevée :

- Sortie du bloc fonction **Drum F** (`%DRi.F = 0`)
- Numéro de l'étape courante (`%DRi.S = 0`)

Programmation

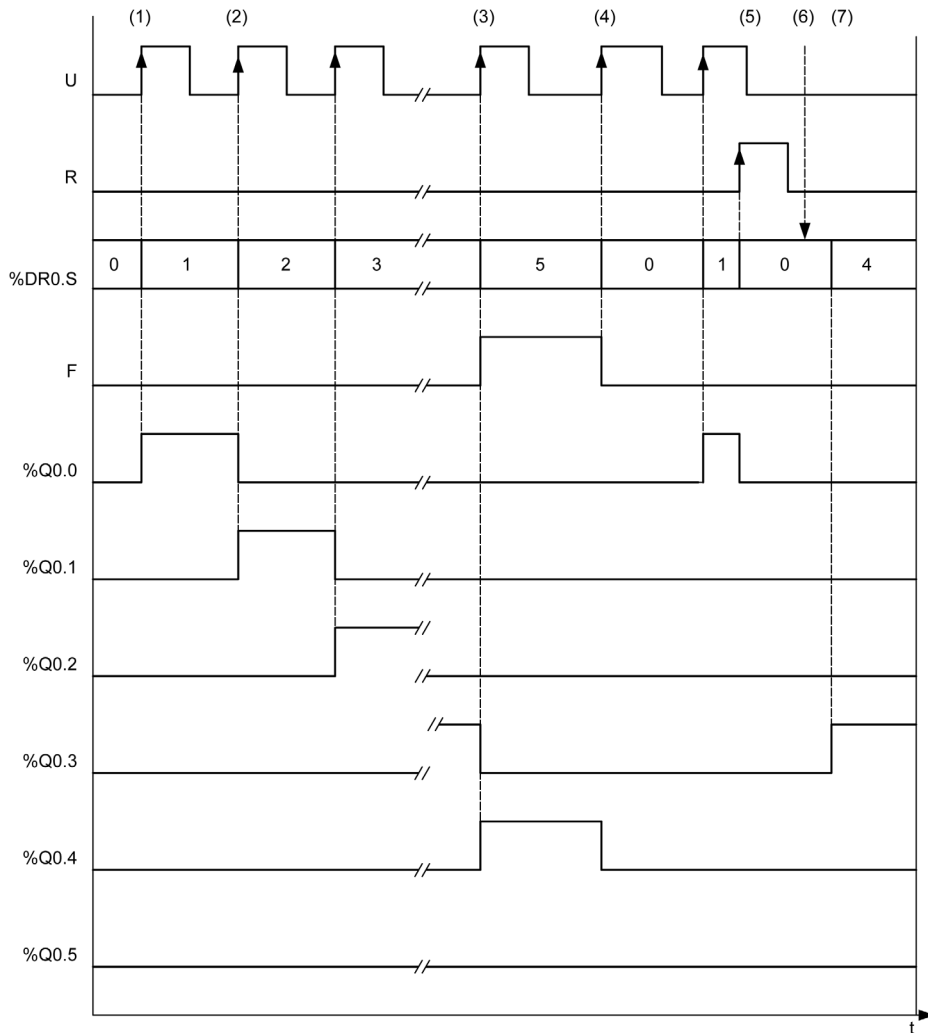
Voici un exemple de programme de bloc fonction `Drum Register` :

Réseau	Instruction
0	BLK %DR1 LD %I0.0 R LD %I0.1 U OUT_BLK LD F ST %Q0.7 END_BLK

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité [\(voir page 16\)](#).

Chronogramme

Ce schéma illustre le fonctionnement du bloc fonction `Drum Register` :



- (1) Lors d'un front montant sur l'entrée U , l'étape courante est incrémentée.
- (2) Lorsque l'étape courante est mise à jour, les sorties le sont également.
- (3) Lorsque la dernière étape est atteinte, la sortie F est mise à 1.
- (4) Un front montant sur l'entrée U lorsque la dernière étape est active, remet l'étape courante à 0.
- (5) Si $\%DR0.R = 1$ (front montant), la valeur courante est mise à 0.
- (6) L'utilisateur écrit le numéro de l'étape : $\%DR0.S = 4$.
- (7) La valeur écrite par l'utilisateur est mise à jour lors de la prochaine exécution.

Configuration

Les informations suivantes sont définies au moment de la configuration :

- Nombre d'étapes : 6
- Etats de sortie (bits de contrôle) pour chaque étape du bloc fonction `Drum Register` :

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Etape 0 :	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Etape 1 :	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Etape 2 :	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
Etape 3 :	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Etape 4 :	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
Etape 5 :	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

- Affectation des bits de contrôle :

Ce tableau présente les sorties associées aux bits de contrôle :

Bit	Sortie associée
0	Sortie non associée
1	%Q0.1
2	%Q0.2
3	%Q0.3
4	%Q0.4
5	%Q0.5

Sous-chapitre 4.10

Bloc fonction Pulse (%PLS)

Pulse

Présentation

Consultez le *Guide de programmation* de votre Logic Controller.

Sous-chapitre 4.11

Bloc fonction Pulse Width Modulation (%PWM)

Pulse Width Modulation

Présentation

Consultez le *Guide de programmation* de votre Logic Controller.

Sous-chapitre 4.12

Message (%MSG) et échange (EXCH)

Utilisation des blocs fonction Message

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction Message.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Présentation	189
Description	191
Configuration	194
Exemple de programmation	199
Exemples ASCII	201
Requêtes et exemples Modbus standard	203

Présentation

Introduction

Il est possible de configurer un Logic Controller pour qu'il communique avec le protocole Modbus ou qu'il puisse envoyer et/ou recevoir des messages en mode caractères (ASCII).

SoMachine Basic propose les fonctions suivantes pour ces communications :

- Instruction **Exchange** (EXCH pour émettre/recevoir des messages.
- Bloc fonction Message (%MSG) pour contrôler les échanges de données.

Le Logic Controller utilise le protocole configuré pour le port spécifié, lors du traitement d'une instruction **Exchange**. Il est possible d'affecter un protocole différent à chaque port de communication. Pour accéder aux ports de communication, il suffit d'ajouter leur numéro à la fin de l'instruction **Exchange** (EXCH1, EXCH2) ou du bloc fonction Message (%MSG1, %MSG2).

Les Logic Controller mettent en œuvre la messagerie Modbus TCP sur le réseau Ethernet grâce à l'instruction EXCH3 et au bloc fonction %MSG3.

Ce tableau montre l'instruction **Exchange** et le bloc fonction Message utilisés pour accéder aux ports de communication du contrôleur :

Port de communication	Instruction d'échange	Bloc fonction Message
2 lignes série	EXCH1	%MSG1
	EXCH2	%MSG2
1 ligne série et 1 Ethernet	EXCH1	%MSG1
	EXCH3	%MSG3

Instruction d'échange

L'instruction **Exchange** permet à un Logic Controller d'envoyer des informations à des équipements ASCII ou Modbus, et/ou d'en recevoir. Vous définissez une table de mots (%MWi : L) contenant les informations de contrôle ainsi que les données à envoyer et/ou recevoir. Consultez la section Configuration de la table d'émission (*voir page 196*). Un échange de message s'effectue à l'aide de l'instruction **Exchange**.

Syntaxe

La syntaxe à utiliser pour l'instruction **Exchange** est la suivante :

[EXCHx %MWi:L]

Où x = numéro de port et L = nombre total de mots dans la table de mots.

Le Logic Controller doit terminer l'échange ordonné par la première instruction **Exchange** pour qu'une nouvelle instruction **Exchange** puisse démarrer. Le bloc fonction Message doit être utilisé pour envoyer plusieurs messages.

Protocole ASCII

Le protocole ASCII offre aux Logic Controller un protocole en mode caractères simple, permettant d'émettre et/ou de recevoir des données à l'aide d'un seul équipement. Ce protocole est pris en charge par l'instruction **Exchange** et contrôlé par le bloc fonction *Message*.

Le protocole ASCII autorise trois types de communication :

- Emission seule
- Emission/réception
- Réception seule

Protocole Modbus

Dans le cas d'une liaison série, le protocole Modbus est un protocole maître-esclave qui permet à un seul maître de demander des réponses à des esclaves ou d'agir en fonction de la requête. Sur Ethernet, plusieurs maîtres (clients) peuvent échanger avec un esclave (serveur). Une adresse unique est affectée à chaque esclave. Le maître peut s'adresser à plusieurs esclaves individuellement ou envoyer un message général à tous les esclaves. Les esclaves renvoient un message (réponse) aux requêtes qui leur sont adressées individuellement. Les réponses aux requêtes de diffusion générale du maître ne sont pas renvoyées.

Le mode Modbus maître permet au contrôleur d'envoyer une requête Modbus à un esclave et d'attendre la réponse. Le mode maître Modbus n'est pris en charge que par l'instruction **Exchange**. Les modes Modbus ASCII et RTU sont tous les deux pris en charge en mode maître Modbus.

Le mode esclave Modbus permet au contrôleur de répondre à des requêtes Modbus standard provenant d'un maître Modbus.

Pour plus d'informations sur le protocole Modbus, consultez le document *Protocole d'application Modbus* disponible à l'adresse <http://www.modbus.org>.

Esclave Modbus

Le protocole Modbus prend en charge deux formats de couche de liaison de données du Modèle OSI : ASCII et RTU. Chaque format est défini par l'implémentation de la couche physique. Le format ASCII utilise sept bits de données, tandis que le format RTU en utilise huit.

En mode Modbus ASCII, chaque octet d'un message est envoyé sous la forme de deux caractères ASCII. La trame Modbus ASCII commence par un caractère de début (':') et se termine par deux caractères de fin (CR et LF). Par défaut, le caractère de fin de trame est 0x0A (LF). La valeur de contrôle de la trame Modbus ASCII correspond à un simple complément de deux de la trame (caractères de début et de fin non compris).

Le mode Modbus RTU ne reformate pas le message avant de l'émettre. Mais, il utilise un mode de calcul de somme de contrôle différent, baptisé CRC.

Les limitations de la couche de liaison de données Modbus sont les suivantes :

- Adresse : 1 à 247
- Bits : 128 bits sur demande
- Mots : 125 mots de 16 bits sur demande

Description

Introduction

Le bloc fonction `Message`  gère les échanges de données et remplit trois fonctions :

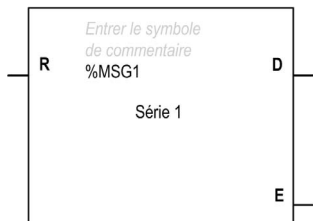
- Vérification des erreurs de communication
Ce contrôle vérifie la taille de chaque table d'**échange**, ainsi que la validité de l'échange par rapport à la configuration.
- Coordination de plusieurs messages :
Pour garantir la coordination en cas d'envoi de plusieurs messages, le bloc fonction `Message` fournit les informations permettant de savoir quand le message précédent est terminé.
- Emission de messages prioritaires :
Le bloc fonction `Message` permet d'interrompre l'émission du message en cours afin d'en envoyer un plus urgent.

La programmation du bloc fonction `Message` est facultative.

Lorsque des erreurs sont détectées, les codes sont écrits dans les mots système `%SW63`, `%SW64` et `%SW65` des blocs fonction `EXCH1`, `EXCH2` et `EXCH3`, respectivement. Pour plus d'informations, consultez le *Guide de programmation* de votre contrôleur.

Illustration

Cette illustration décrit le bloc fonction `Message` :



Entrées

Le bloc fonction `Message` a l'entrée suivante :

Libellé	Description	Valeur
R	Entrée RAZ	Etat 1 : réinitialisation de la communication ; <code>%MSGx.E = 0</code> et <code>%MSGx.D = 1</code> . Etat 0 : mode normal.

Sorties

Le bloc fonction **Message** a les sorties suivantes :

Libellé	Description	Valeur
D	Communication terminée (%MSGx.D)	Etat 1 : ● Fin d'émission (si émission) ● Fin de réception (réception du caractère de fin) ● Erreur ● Réinitialisation du bloc Etat 0 : requête en cours.
E	Erreur de communication détectée (%MSGx.E)	Etat 1 : ● Commande non définie ● Configuration incorrecte de la table ● Réception d'un caractère incorrect (vitesse, parité, etc.) ● Table de réception pleine (non mise à jour) Etat 0 : message de longueur correcte, liaison établie. Pour connaître les codes écrits dans les mots système en cas d'erreur de communication détectée, consultez le tableau ci-dessous.

Codes des erreurs de communication

Ce tableau décrit les codes d'erreur écrits dans les mots système lorsqu'une erreur de communication est détectée :

Mot système	Fonction	Description
%SW63	Code d'erreur du bloc EXCH1	Code d'erreur EXCH1 : 0 - opération réussie 1 - nombre d'octets à émettre trop important (> 250) 2 - table d'émission trop petite 3 - table de mots trop petite 4 - débordement de la table de réception 5 - timeout écoulé 6 - émission 7 - commande incorrecte dans la table 8 - port sélectionné non configuré/disponible 9 - erreur de réception : ce code d'erreur signale une trame de réception incorrecte. Cela peut être dû à une mauvaise configuration des paramètres physiques (par exemple, la parité, le débit en bauds, etc.) ou à une connexion physique incorrecte entraînant une dégradation du signal. 10 - impossible d'utiliser %KW en cas de réception 11 - décalage d'émission plus important que la table d'émission 12 - décalage de réception plus important que la table de réception 13 - interruption du traitement EXCH par le contrôleur
%SW64	Code d'erreur de bloc EXCH2	Code d'erreur EXCH2 : voir %SW63.

Mot système	Fonction	Description
%SW65	Code d'erreur de bloc EXCH3	1-4, 6-13 : voir %SW63. (Remarque : le code d'erreur 5 est incorrect. Il est remplacé par les codes d'erreur 109 et 122 spécifiques à Ethernet qui sont décrits ci-dessous.) Les codes d'erreur suivants sont spécifiques à Ethernet ; 101 - aucune adresse IP de ce type 102 - connexion TCP interrompue 103 - aucun socket disponible (toutes les voies de connexion sont occupées) 104 - réseau non opérationnel 105 - réseau inaccessible 106 - connexion interrompue par le réseau lors de la réinitialisation 107 - connexion abandonnée par équipement homologue 108 - connexion réinitialisée par équipement homologue 109 - timeout écoulé pour la connexion 110 - rejet de la tentative de connexion 111 - hôte non opérationnel 120 - index inconnu (équipement distant non indexé dans le tableau de configuration) 121 - erreur fatale (MAC, puce, IP en double) 122 - timeout du processus de réception après l'envoi des données 123 - initialisation d'Ethernet en cours

Configuration

Erreur détectée

Si une erreur est détectée lors de l'utilisation d'une instruction d'**échange**, les bits %MSGx.D et %MSGx.E sont mis à 1, le mot système %SW63 contient le code d'erreur du port 1 et %SW64 le code d'erreur du port 2. Consultez le chapitre Mots système du Guide de la bibliothèque de fonctions de votre Logic Controller.

Opérations

Ce tableau décrit les principales étapes des opérations du bloc fonction Message :

Fonctionnement	Action	Résultat
Réinitialisation	L'entrée R est mise à 1 (ou l'instruction R est activée).	● L'émission de tous les messages est interrompue. ● La sortie d'erreur de communication est remise à 0. ● Le bit Done est mis à 1. Un nouveau message peut être envoyé.
Communication terminée	La sortie D est mise à 1.	Le Logic Controller est prêt à envoyer un autre message. L'utilisation de la sortie %MSGx.D est recommandée en cas d'envoi de plusieurs messages. Si cette sortie n'est pas utilisée, les messages risquent d'être perdus.
Erreur de communication détectée	La sortie d'erreur de communication est mise à 1 : ● En cas d'erreur de programmation des communications ou d'erreur de transmission du message. ● Si le nombre d'octets définis dans le bloc de données associé à l'instruction Exchange (mot 1, octet de poids faible) est supérieur à 128 (+80 en hexadécimal par FA). ● S'il existe un problème d'envoi d'un message Modbus à un équipement Modbus. Dans ce cas, vérifiez le câblage et assurez-vous que l'équipement de destination prend en charge la communication Modbus.	

Cas particuliers

Ce tableau contient une liste de cas particuliers du fonctionnement du bloc fonction `Message` :

Cas particulier	Description
Effet d'un redémarrage à froid (%S0=1) ou INIT	Force la réinitialisation de la communication.
Effet d'un redémarrage à chaud (%S1=1)	Aucun effet
Effet d'un arrêt du contrôleur	Si un message est en cours d'émission, le contrôleur interrompt le transfert et réinitialise les sorties %MSGx.D et %MSGx.E.

NOTE : L'effet d'**INIT** est identique à celui de %S0=1.

Limitations

Prenez note des limitations suivantes :

- La disponibilité du port 2 (pour le protocole ASCII) et son type (voir %SW7) ne sont vérifiés qu'à la mise sous tension ou à la réinitialisation.
- La présence du port 2 (pour le protocole Modbus) et sa configuration (RS-485) sont vérifiées lors de la mise sous tension ou de la réinitialisation.
- **Tout message en cours de traitement sur le port 1 est abandonné** lorsque SoMachine Basic est connecté.
- Les instructions **Exchange** abandonnent le traitement des esclaves Modbus actifs.
- Le traitement des instructions **Exchange** ne reprend pas en cas d'erreur détectée.
- Il est possible d'utiliser l'entrée RAZ pour annuler le traitement de la réception d'une instruction **Exchange**.
- Les instructions **Exchange** sont configurées avec un timeout si le protocole Modbus est utilisé.
- Les messages multiples sont contrôlés via %MSGx.D.

Configuration de la table d'émission/réception

La taille maximale des trames émises et/ou reçues est de :

- 250 octets pour le protocole Modbus ;
- 256 octets pour le protocole ASCII.

En outre, la table de mots associée à l'instruction **Exchange** comprend les tables de contrôle, d'émission et de réception :

	Octet de poids fort		Octet de poids faible	
	Modbus	ASCII	Modbus	ASCII
Table de contrôle	Commande		Longueur (émission/réception)	
	Décalage réception	Réservé (0)	Décalage émission	Réservé (0)
Table d'émission	Octet 1 émis		Octet 2 émis	
	...		...	
			Octet n émis	
	Octet n+1 émis			
Table de réception	Octet 1 reçu		Octet 2 reçu	
	...		...	
			Octet p reçu	
	Octet p+1 reçu			

NOTE : Le contrôleur maître Modbus peut envoyer des requêtes à des esclaves individuellement, mais aussi une diffusion générale à tous les esclaves. En cas de diffusion générale, l'octet **Commande** doit être mis à 00, et l'adresse de l'esclave à 0.

Table de contrôle du protocole ASCII

L'octet **Longueur** contient la longueur de la table d'émission (250 octets max.), qui est remplacée par le nombre de caractères reçus à la fin de la réception, si la réception est demandée.

L'octet **Commande** doit contenir l'une des valeurs suivantes :

- 0 : émission seule
- 1 : émission/réception
- 2 : réception seule

Table de contrôle du protocole Modbus

L'octet **Longueur** contient la longueur de la table d'émission (250 octets max.), qui est remplacée par le nombre de caractères reçus à la fin de la réception, si la réception est demandée.

Ce paramètre correspond à la longueur en octets de la table d'émission. Si le paramètre **Décalage émission** est égal à 0, ce paramètre sera égal à la longueur de la trame d'émission. Si le paramètre **Décalage émission** est différent de 0, un octet de la table d'émission (indiqué par la valeur de décalage) ne sera pas émis et ce paramètre sera égal à la longueur de la trame plus 1.

En cas de requête Modbus RTU (sauf pour une diffusion générale), l'octet **Commande** doit toujours être égal à 1 (**Tx** et **Rx**). Pour une diffusion générale, il doit être égal à 0.

L'octet **Décalage émission** contient le rang (1 pour le premier octet, 2 pour le deuxième octet, etc.) dans la table d'émission de l'octet à ignorer, lors de l'émission des octets. Il permet de gérer les problèmes liés aux valeurs d'octet/de mot dans le protocole Modbus. Par exemple, si cet octet est égal à 3, le troisième octet est ignoré, ce qui fait du quatrième octet de la table le troisième octet à émettre.

L'octet **Décalage réception** contient le rang (1 pour le premier octet, 2 pour le deuxième octet, etc.) dans la table de réception à ajouter, lors de l'émission des octets. Il permet de gérer les problèmes liés aux valeurs d'octet/de mot dans le protocole Modbus. Par exemple, si cet octet contenait 3, le troisième octet de la table prendrait la valeur 0 et le troisième octet reçu serait stocké dans le quatrième emplacement de la table.

Tables d'émission/réception du protocole ASCII

En mode émission seule, les tables de contrôle et d'émission sont renseignées avant l'exécution de l'instruction **d'échange** ; elles peuvent être de type `%KW` ou `%MW`. Aucun espace n'est requis pour la réception de caractères en mode émission seule. Une fois tous les octets émis, `%MSGx.D` est mis à 1 et une nouvelle instruction **Exchange** peut être exécutée.

En mode émission/réception, les tables de contrôle et d'émission sont remplies avant l'exécution de l'instruction **Exchange**. Elles doivent être de type `%MW`. Un espace pouvant accueillir jusqu'à 256 octets de réception est requis à la fin de la table d'émission. Une fois tous les octets émis, le Logic Controller passe en mode réception et est prêt à recevoir des octets.

En mode réception seule, la table de contrôle est remplie avant l'exécution de l'instruction **d'échange**. Elle doit être de type `%MW`. Un espace pouvant accueillir jusqu'à 256 octets de réception est requis à la fin de la table de contrôle. Le Logic Controller passe immédiatement en mode réception et est prêt à recevoir des octets.

La réception est terminée une fois que les octets de fin de trame utilisés ont été reçus ou lorsque la table de réception est pleine. Dans ce cas, un code d'erreur détectée (table de réception saturée) s'affiche dans les mots système `%SW63` et `%SW64`. Si un timeout non nul est configuré, la réception se termine à l'expiration du timeout. Si vous sélectionnez une valeur de timeout nulle, il n'existe aucun timeout de réception. Par conséquent, pour arrêter la réception, vous devez activer l'entrée `%MSGx.R`.

Table d'émission/réception du protocole Modbus

Dans l'un ou l'autre mode (Modbus ASCII ou Modbus RTU), le contenu de la requête est stocké dans la table d'émission, avant l'exécution de l'instruction **Exchange**. Lors de l'exécution, le Logic Controller identifie la couche de liaison de données et effectue toutes les conversions nécessaires pour traiter l'émission et la réponse. Les caractères de début, de fin et de contrôle ne sont pas stockés dans les tables d'émission/réception.

Une fois tous les octets émis, le Logic Controller passe en mode réception et est prêt à recevoir des octets.

La réception se termine de l'une des manières suivantes :

- un timeout est détecté sur un caractère ou une trame ;
- des caractères de fin de trame sont reçus en mode ASCII ;
- la table de réception est saturée.

Les x entrées **Octet émis** contiennent les données Modbus (codage RTU) à émettre. Si le port de communication est configuré en Modbus ASCII, les caractères de trame corrects sont ajoutés à l'émission. Le premier octet contient l'adresse de l'équipement (spécifique ou diffusion générale), le deuxième octet contient le code de fonction, et le reste contient les informations associées à ce code de fonction.

NOTE : Il s'agit d'une application type, mais toutes les possibilités ne sont pas définies. Aucune validation des données en cours d'émission n'est effectuée.

Les x entrées **Octets reçus** contiennent les données Modbus (codage RTU) à recevoir. Si le port de communication est configuré en Modbus ASCII, les caractères de trame corrects sont supprimés de la réponse. Le premier octet contient l'adresse de l'équipement, le deuxième octet contient le code de fonction (ou code de réponse), et le reste contient les informations associées à ce code de fonction.

NOTE : Il s'agit d'une application type, mais toutes les possibilités ne sont pas définies. Aucune validation des données en cours de réception n'est effectuée, à l'exception de la vérification de la somme de contrôle.

Exemple de programmation

Introduction

Les exemples ci-dessous illustrent la programmation d'un bloc fonction `Message`.

Programmation de l'émission de plusieurs messages successifs

L'exécution de l'instruction **Exchange** active un bloc fonction `Message` dans le programme de l'application. Le message est émis si le bloc fonction `Message` n'est pas déjà actif (`%MSGx.D = 1`). Si plusieurs messages sont envoyés au cours du même cycle, seul le premier est émis à l'aide du même port.

Exemple d'émission de deux messages successifs sur le port 1 :

Réseau	Instruction réversible	Commentaire
0	LD %M142 [%MW2:=16#0106] [%MW3:=0] [%MW4:=16#0106] [%MW5:=4] [%MW6:=7]	Ecriture sur un esclave, à l'adresse 1 : valeur 7 sur le registre 4. [%MW2:=16#0106] : code de commande : 01 hex, longueur d'émission : 06 hex [%MW3:=0] : aucun décalage de réception ou d'émission [%MW4:=16#0106] : adresse de l'esclave : 01 hex, code fonction : 06 hex (Ecriture d'un registre) [%MW5:=4] : adresse du registre [%MW6:=7] : valeur à écrire
1	LD %MSG2.D AND %M0 [EXCH2%MW2:5] R %M0	%MSG2.D : détecte si le port est occupé ou non, et assure la coordination de plusieurs messages.
2	LDR %I0.0 AND %MSG2.D [EXCH2%MW2:5] S %M0	—

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Programmation d'un échange de réinitialisation

Pour annuler un échange, activez l'entrée (ou l'instruction) R. Cette entrée initialise la communication et remet les sorties %MSGx.E et %MSGx.D à 0 et 1, respectivement. Il est possible de réinitialiser un échange si une erreur est détectée.

Exemple de réinitialisation d'un échange :

Réseau	Instruction réversible	Commentaire
0	BLK %MSG1 LD %M0 R END_BLK	—

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Exemples ASCII

Ecriture d'application

Exemple d'application ASCII :

Réseau	Instruction	Commentaire
0	LD 1 [%MW10:=16#0104] [%MW11:=16#0000] [%MW12:=16#4F4B]	[%MW10:=16#0104] : code de commande : 01 hex, longueur d'émission : 04 hex [%MW11:=16#0000] : 0000 : null [%MW12:=16#4F4B] : OK NOTE : cette table comporte huit éléments :
1	LD 1 AND %MSG2.D [EXCH2 %MW10:8]	
2	LD %MSG2.E ST %Q0.0 END	

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Utilisez SoMachine Basic pour créer un programme contenant trois réseaux :

- Tout d'abord, initialisez les tables de contrôle et d'émission à utiliser pour l'instruction **Exchange**. Dans cet exemple, une commande est configurée pour envoyer et recevoir des données. Le volume de données à envoyer est de 4 octets (comme défini dans l'application), suivis par le caractère de fin de trame spécifié dans la configuration. Les caractères de début et de fin ne sont pas affichés dans la table d'animation, où seuls les caractères de données apparaissent. Quoi qu'il en soit, ces caractères sont automatiquement émis ou vérifiés lors de la réception (par %SW63 et %SW64), lorsqu'ils sont utilisés.
NOTE : Les caractères de fin définis dans la configuration sont envoyés automatiquement dans la fin de la trame. Par exemple, si vous avez configuré 10 comme premier caractère de fin de trame et 13 comme deuxième caractère de fin de trame, 16#0A0D (codes ASCII, 0A = LF et 0D = CR) est envoyé dans la fin de la trame.
- Vérifiez ensuite le bit d'état associé à %MSG2 et exécutez l'instruction EXCH2 uniquement si le port est prêt. Une valeur de 8 mots est spécifiée pour l'instruction EXCH2. Il existe deux mots de contrôle (%MW10 et %MW11), deux mots à utiliser pour émettre des informations (%MW12 et %MW13) et quatre mots pour recevoir des données (%MW14 à %MW17).
- Enfin, l'état d'erreur de %MSG2 est détecté et stocké dans le premier bit de sortie des E/S locales du contrôleur de base. Pour accroître la précision, il est possible d'effectuer un traitement supplémentaire à l'aide de %SW64.

Initialisation de la table d'animation

Exemple d'initialisation d'une table d'animation en mode en ligne :

Adresse	Valeur	Format
%MW10	0104	Hexadécimal
%MW11	0000	Hexadécimal
%MW12	4F4B	Hexadécimal
%MW13	0A0D	Hexadécimal
%MW14	AL	ASCII
%MW15	OH	ASCII
%MW16	A	ASCII

Pour afficher les formats possibles, cliquez avec le bouton droit de la souris sur le champ **Valeurs** dans une table d'animation.

L'étape finale consiste à charger cette application sur le contrôleur et à l'exécuter. Initialisez une table d'animation pour animer et afficher les mots %MW10 à %MW16. Ces informations sont échangées avec un Logic Controller et affichées dans une table d'animation.

Requêtes et exemples Modbus standard

Maître Modbus : lecture de N bits

Cette table représente les requêtes 01 et 02 (01 pour le bit mémoire ou de sortie, 02 pour le bit d'entrée) :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	06 (longueur d'émission) ⁽¹⁾
	1	03 (Décalage réception)	00 (Décalage émission)
Table d'émission	2	Slave@(1 à 247)	01 ou 02 (Code de requête)
	3	Adresse du premier bit à lire dans l'esclave	
	4	N_1 = Nombre de bits à lire	
Table de réception (après réponse)	5	Slave@(1 à 247)	01 ou 02 (Code de réponse)
	6	00 (octet ajouté par l'action de décalage de Rx)	N_2 = Nombre d'octets de données à lire = $[1+(N_1-1)/8]$, où le résultat est la partie entière de la division.
	7	Valeur du premier bit (valeur 00 ou 01) convertie en un octet	Valeur du deuxième bit (si $N_2 > 1$) convertie en un octet
	8	Valeur du troisième bit (si $N_1 > 1$) convertie en un octet	—
	...	...	...
	$(N_2/2) + 6$ (si N_2 est pair) $(N_2/2 + 1) + 6$ (si N_2 est impair)	Valeur du $N_2^{\text{ème}}$ bit (si $N_1 > 1$) convertie en un octet	—
(1) Cet octet reçoit également la longueur de la chaîne émise après réponse..			

Maître Modbus : lecture de N mots

Cette table représente les requêtes 03 et 04 (03 pour le mot mémoire ou de sortie, 04 pour le mot d'entrée) :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	06 (longueur d'émission) ⁽¹⁾
	1	03 (Décalage réception)	00 (Décalage émission)
Table d'émission	2	Slave@(1 à 247)	03 ou 04 (Code de requête)
	3	Adresse du premier mot à lire	
	4	N = Nombre de mots à lire	
Table de réception (après réponse)	5	Slave@(1 à 247)	03 ou 04 (Code de réponse)
	6	00 (octet ajouté par l'action de décalage de Rx)	2*N (nombre d'octets lus)
	7	Premier mot lu	
	8	Deuxième mot lu (si N>1)	
	...	...	
	N+6	Nième mot lu (si N>2)	

(1) Cet octet reçoit également la longueur de la chaîne émise après réponse..

NOTE : Le décalage de 3 de la réception ajoute un octet (valeur = 0) à la troisième position de la table de réception. Ce qui assure un bon positionnement dans la table, du nombre d'octets lus et des valeurs des mots lus.

Maître Modbus : écriture de bit

Cette table représente la requête 05 (écriture d'un bit : sortie ou mémoire) :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	06 (longueur d'émission) ⁽¹⁾
	1	00 (Décalage réception)	00 (Décalage émission)
Table d'émission	2	Slave@(1 à 247) ou 0 en cas de diffusion	05 (Code de requête)
	3	Valeur pour écrire l'octet de poids fort du mot d'index 4 ; 0xFF ou 0x00 ⁽²⁾ .	
	4	Valeur du bit à écrire dans l'esclave (16#0000 = False et 16#FF00 = True)	
Table de réception (après réponse)	5	Slave@(1 à 247)	05 (Code de réponse)
	6	Adresse du bit écrit	
	7	Valeur écrite	

(1) Cet octet reçoit également la longueur de la chaîne émise après réponse..

(2) Pour affecter la valeur 1 à un bit, le mot associé dans la table d'émission doit contenir la valeur FF00H, et 0 pour affecter la valeur 0 à un bit.

NOTE :

- Il n'est pas nécessaire d'utiliser le décalage pour cette requête.
- La trame de la réponse est identique à celle de la requête (normalement).

Maître Modbus : écriture de mot

Cette table représente la requête 06 (écriture d'un mot : sortie ou mémoire) :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	06 (longueur d'émission) ⁽¹⁾
	1	00 (Décalage réception)	00 (Décalage émission)
Table d'émission	2	Slave@(1 à 247) ou 0 en cas de diffusion	06 (Code de requête)
	3	Adresse du mot à écrire	
	4	Valeur du mot à écrire	
Table de réception (après réponse)	5	Slave@(1 à 247)	06 (Code de réponse)
	6	Adresse du mot écrit	
	7	Valeur écrite	
(1) Cet octet reçoit également la longueur de la chaîne émise après réponse..			

NOTE :

- Il n'est pas nécessaire d'utiliser le décalage pour cette requête.
- La trame de la réponse est identique à celle de la requête (normalement).

Maître Modbus : écriture de N bits

Cette table représente la requête 15 (écriture de N bits : sortie ou mémoire) :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	8 + nombre d'octets (émission)
	1	00 (Décalage réception)	07 (Décalage émission)
Table d'émission	2	Slave@(1 à 247) ou 0 en cas de diffusion	15 (Code de requête)
	3	Adresse du premier bit à écrire	
	4	N_1 = Nombre de bits à écrire	
	5	00 (octet non envoyé, effet de décalage)	N_2 = Nombre d'octets de données à écrire = $[1+(N_1-1)/8]$, où le résultat est la partie entière de la division.
	6	Valeur du premier octet	Valeur du deuxième octet
	7	Valeur du troisième octet	Valeur du quatrième octet
	...	...	...
	$(N_2/2) + 5$ (si N_2 est pair) $(N_2/2 + 1) + 5$ (si N_2 est impair)	Valeur du $N_2^{\text{ème}}$ octet	
	Table de réception (après réponse)	Slave@(1 à 247)	15 (Code de réponse)
		Adresse du premier bit écrit	
		Nombre de bits écrits (= N_1)	

NOTE : Le décalage d'émission de 7 supprime le septième octet de la trame envoyée. Ceci permet également d'assurer une bonne correspondance avec les valeurs des mots dans la table d'émission.

Maître Modbus : écriture de N mots

La table suivante représente la requête 16 :

	Table Index	Octet de poids fort	Octet de poids faible
Table de contrôle	0	01 (Emission/Réception)	8 + (2*N) (Longueur émission)
	1	00 (Décalage réception)	07 (Décalage émission)
Table d'émission	2	Slave@(1 à 247) ou 0 en cas de diffusion	16 (Code de requête)
	3	Adresse du premier mot à écrire	
	4	N = Nombre de mots à écrire	
	5	00 (octet non envoyé, effet de décalage)	2*N = Nombre d'octets à écrire
	6	Première valeur de mot à écrire	
	7	Deuxième valeur à écrire	
	...	...	
	N+5	N valeurs à écrire	
Table de réception (après réponse)	N+6	Slave@(1 à 247)	16 (Code de réponse)
	N+7	Adresse du premier mot écrit	
	N+8	Nombre de mots écrits (= N)	

NOTE : Le décalage d'émission de 7 supprime le septième octet de la trame envoyée. Ceci permet également d'assurer une bonne correspondance avec les valeurs des mots dans la table d'émission.

Requête Modbus : lire l'identification des équipements

Cette table représente la requête 43 (lire l'identification des équipements) :

Réseau	Instruction	Commentaire
0	LD 1 [%MW800:=16#0106] [%MW801:=16#0000] [%MW802:=16#032B] [%MW803:=16#0E01] [%MW804:=16#0000]	[%MW800:=16#0106] : en-tête Modbus standard [%MW801:=16#0000] : aucun décalage d'émission et de réception [%MW802:=16#032B] : adresse de l'esclave, code de fonction [%MW803:=16#0E01] : type MEI, lecture du code d'identification des équipements [%MW804:=16#0000] : ID d'objet, inutilisé

Requête Modbus : diagnostic

Cette table représente la requête 8 (diagnostic) :

Réseau	Instruction	Commentaire
0	LD 1 [%MW1000:=16#0106] [%MW1001:=16#0000] [%MW1002:=16#0308] [%MW1003:=16#0000] [%MW1004:=16#1234]	[%MW1000:=16#0106] : en-tête Modbus standard [%MW1001:=16#0000] : aucun décalage d'émission et de réception [%MW1002:=16#0308] : adresse de l'esclave, code fonction [%MW1003:=16#0000] : code de sous-fonction [%MW1004:=16#1234] : toutes les données L'esclave répond en envoyant une copie de la requête. Ce mode est appelé Echo ou Miroir.

Exemple 1 : écriture d'application Modbus

Programme du maître :

Réseau	Instruction	Commentaire
0	LD 1 [%MW0:=16#0106] [%MW1:=16#0300] [%MW2:=16#0203] [%MW3:=16#0000] [%MW4:=16#0004]	[%MW0:=16#0106] : longueur d'émission = 6 [%MW1:=16#0300] : décalage de réception = 3, décalage d'émission = 0 %MW2 à %MW4 : émission [%MW2:=16#0203] : esclave 2, fonction 3 (lecture de plusieurs mots) [%MW3:=16#0000] : adresse du premier mot à lire dans l'esclave : jusqu'à l'adresse 0 [%MW4:=16#0004] : nombre de mots à lire : 4 (%MW0 à %MW3)
1	LD 1 AND %MSG2.D [EXCH2 %MW0:11]	—
2	LD %MSG2.E ST %Q0.0 END	—

Programme de l'esclave :

Réseau	Instruction	Commentaire
0	LD 1 [%MW0:=16#6566] [%MW1:=16#6768] [%MW2:=16#6970] [%MW3:=16#7172] END	—

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

A l'aide de SoMachine Basic, créez un programme d'application pour le maître et l'esclave. Pour l'esclave, écrivez des mots mémoire et attribuez-leur des valeurs connues. Dans le maître, la table de mots de l'instruction **Exchange** est initialisée pour lire quatre mots sur l'esclave à l'adresse Modbus 2, à partir de l'emplacement %MW0.

NOTE : Remarquez l'utilisation du décalage de réception défini dans le mot %MW1 du maître Modbus. Ce décalage de trois ajoute un octet (valeur = 0) à la troisième position de la zone de réception de la table. Il permet d'aligner les mots dans le maître de façon à ce qu'ils correspondent exactement aux limites de mot. Sans ce décalage, chaque mot de données serait fractionné en deux dans le bloc **Exchange**. Ce décalage est utilisé pour des raisons de commodité.

Avant d'exécuter l'instruction **EXCH2**, l'application vérifie le bit de communication associé à %MSG2. Enfin, l'état d'erreur de %MSG2 est détecté et stocké dans le premier bit de sortie des E/S locales du contrôleur de base. Pour accroître la précision, il est possible d'effectuer un contrôle d'erreur supplémentaire à l'aide de %SW64.

Table d'animation s'initialisant en mode en ligne et correspondant à la partie de la table de réception :

Adresse	Valeur	Format
%MW5	0203	Hexadécimal
%MW6	0008	Hexadécimal
%MW7	6566	Hexadécimal
%MW8	6768	Hexadécimal
%MW9	6970	Hexadécimal
%MW10	7172	Hexadécimal

Après le chargement et la configuration de chaque Logic Controller à exécuter, ouvrez une table d'animation sur le maître. Dans la section réponse de la table, vérifiez que le code de réponse est 3 et que le nombre d'octets lus est correct. Notez également, dans cet exemple, que les mots lus sur l'esclave (à partir de %MW7) correspondent aux limites de mot dans le maître.

Exemple 2 : écriture d'application Modbus

Programme du maître :

Réseau	Instruction	Commentaire
0	LD 1 [%MW0:=16#010C] [%MW1:=16#0007] [%MW2:=16#0210] [%MW3:=16#0010] [%MW4:=16#0002] [%MW5:=16#0004] [%MW6:=16#6566] [%MW7:=16#6768]	[%MW0:=16#010C] : longueur de la table d'émission : 0C hex = 12 dec, de %MW2 à %MW7 [%MW1:=16#0007] [%MW2:=16#0210] : adresse de l'esclave 2, code fonction 10h déclenchant l'écriture de mots [%MW3:=16#0010] : à partir de l'adresse 16 dans l'esclave [%MW4:=16#0002] : écriture de 2 mots [%MW5:=16#0004] : nombre d'octets à écrire [%MW6:=16#6566] : valeur du premier mot [%MW7:=16#6768] : valeur du second mot
1	LD 1 AND %MSG2.D [EXCH2 %MW0:12]	—
2	LD %MSG2.E ST %Q0.0 END	—

Programme de l'esclave :

Réseau	Instruction	Commentaire
0	LD 1 [%MW18:=16#FFFF] END	—

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

A l'aide de SoMachine Basic, créez un programme d'application pour le maître et l'esclave. Pour l'esclave, écrivez un mot mémoire (%MW18). Ceci allouera sur l'esclave l'espace nécessaire aux adresses mémoire de %MW0 à %MW18. Sans cet espace, la requête Modbus tenterait d'écrire dans des emplacements qui n'existent pas sur l'esclave.

Dans le maître, la table de mots de l'instruction EXCH2 est initialisée pour lire 4 octets sur l'esclave à l'adresse Modbus 2 et à l'adresse %MW16 (10 en hexadécimal).

NOTE : Remarquez l'utilisation du décalage d'émission défini dans le mot %MW1 de l'application maître Modbus. Ce décalage de sept supprime l'octet de poids fort dans le sixième mot (valeur 00 hexadécimale dans %MW5). Cette action permet d'aligner les valeurs de données dans la table d'émission de la table de mots, de façon à ce qu'elles respectent les limites de mot.

Avant d'exécuter l'instruction EXCH2, l'application vérifie le bit de communication associé à %MSG2. Enfin, l'état d'erreur de %MSG2 est détecté et stocké dans le premier bit de sortie des E/S locales du contrôleur de base. Pour accroître la précision, il est possible d'effectuer un contrôle supplémentaire d'erreur détectée l'aide de %SW64.

Initialisation d'une table d'animation sur le maître :

Adresse	Valeur	Format
%MW0	010C	Hexadécimal
%MW1	0007	Hexadécimal
%MW2	0210	Hexadécimal
%MW3	0010	Hexadécimal
%MW4	0002	Hexadécimal
%MW5	0004	Hexadécimal
%MW6	6566	Hexadécimal
%MW7	6768	Hexadécimal
%MW8	0210	Hexadécimal
%MW9	0010	Hexadécimal
%MW10	0004	Hexadécimal

Initialisation d'une table d'animation sur l'esclave :

Adresse	Valeur	Format
%MW16	6566	Hexadécimal
%MW17	6768	Hexadécimal

Après le chargement et la configuration de chaque Logic Controller à exécuter, ouvrez une table d'animation sur le contrôleur esclave. Les deux valeurs de %MW16 et %MW17 sont écrites sur l'esclave.

Dans le maître, il est possible d'utiliser la table d'animation pour examiner la partie table de réception des données d'échange. Ces données affichent l'adresse de l'esclave, le code de réponse, le premier mot écrit et le nombre de mots écrits à partir de %MW8 dans l'exemple ci-dessus.

Chapitre 5

Blocs fonction Schedule Blocks (%SCH)

Utilisation des blocs fonction Schedule Blocks

Cette section fournit des descriptions et des conseils de programmation pour utiliser les blocs fonction `Schedule blocks`.

Contenu de ce chapitre

Ce chapitre contient les sujets suivants :

Sujet	Page
Description	214
Programmation et configuration	216

Description

Introduction

Les blocs fonction `Schedule blocks` permettent de programmer des actions selon un calendrier précis (mois, jour et heure).

Les blocs fonction `Schedule blocks` ne sont que configurés dans SoMachine Basic. Ils ne sont pas insérés dans un réseau de programme, comme les autres blocs fonction.

NOTE : Vérifiez le bit système `%S51` et le mot système `%SW118` afin de vous assurer que l'option d'horodateur ou RTC (Real-Time Clock) est installée. Cette option est requise pour utiliser les blocs fonction `Schedule blocks`.

Paramètres

Pour configurer les paramètres, suivez la procédure Configuration d'un bloc fonction (voir page 139) et lisez la description des Modes d'allocation de mémoire.

L'objet `Schedule blocks` a les paramètres suivants :

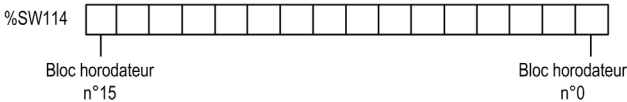
Paramètre	Description	Valeur
Utilisée	Adresse utilisée	Si ce paramètre est sélectionné, cette adresse est utilisée dans un programme.
Adresse	Adresse de l'objet <code>Schedule blocks</code>	Un programme ne peut contenir qu'un nombre limité d'objets <code>Schedule blocks</code> . Consultez le Guide de programmation de la plate-forme matérielle pour connaître le nombre maximum de blocs fonction <code>Schedule blocks</code> .
Configured	Indique si le nombre d'objets <code>Schedule blocks</code> sélectionnés est configuré pour être utilisé.	Si la case est cochée, il est configuré pour être utilisé. Sinon, il n'est pas utilisé.
Bit de sortie	Bit de sortie	L'affectation de sortie est activée par l'objet <code>Schedule blocks : %Mi</code> ou <code>%Qj.k</code> . Cette sortie est mise à 1 lorsque les paramètres de date et d'heure courants sont compris entre les paramètres de début et de fin de la période active.
Jour de début	Jour auquel débute l'objet <code>Schedule blocks</code> .	1 à 31
Mois de début	Mois auquel débute l'objet <code>Schedule blocks</code> .	<code>Schedule blocks</code> .
Jour de fin	Jour auquel s'arrête l'objet <code>Schedule blocks</code> .	1 à 31
Mois de fin	Mois auquel s'arrête l'objet <code>Schedule blocks</code> .	janvier à décembre

Paramètre	Description	Valeur
Heure de début	Heure du jour (avec les minutes) à laquelle débute l'objet Schedule blocks.	Heure : 0 à 23 Minute : 0 à 59
Heure de fin	Heure du jour (avec les minutes) à laquelle s'arrête l'objet Schedule blocks.	Heure : 0 à 23 Minute : 0 à 59
Lundi	Cases à cocher identifiant le ou les jours au cours desquels sera activé l'objet Schedule blocks.	Si la case est cochée, il est configuré pour être utilisé. Sinon, il n'est pas utilisé.
Mardi		
Mercredi		
Jeudi		
Vendredi		
Samedi		
Dimanche		
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Activation d'objets Schedule Blocks

Les bits du mot système %SW114 activent (lorsqu'ils sont mis à 1) ou désactivent (lorsqu'ils sont mis à 0) le fonctionnement des 16 objets Schedule blocks.

Affectation d'objets Schedule blocks dans %SW114 :



Par défaut (ou après un démarrage à froid), tous les bits de ce mot système sont mis à 1. L'utilisation de ces bits par le programme est facultative.

Sortie des objets Schedule Blocks

Si la même sortie (%Mi ou %Qj.k) est affectée par plusieurs blocs, c'est le OR des résultats de chacun des blocs qui est finalement affecté à cet objet. Notez que la même sortie peut avoir plusieurs objets Schedule blocks.

Par exemple, les objets %SCH0 et %SCH1 sont tous deux affectés à la sortie %Q0.0. %SCH0 règle la sortie entre 12h00 et 13h00 le lundi, et %SCH1 la sortie entre 12h00 et 13h00 le mardi. Le résultat est que la sortie passe de 12h00 à 13h00 le lundi et le mardi.

Programmation et configuration

Introduction

Les blocs fonction `Schedule blocks` permettent de programmer des actions selon un calendrier précis (mois, jour et heure).

Exemple de programmation

Le tableau suivant présente les paramètres d'un programme d'arrosage pendant les mois d'été :

Paramètre	Valeur	Description
Adresse	Real-Time Clock 6	<code>Schedule blocks</code> numéro 6
Configured	Case cochée	Case cochée pour configurer le <code>Schedule blocks</code> numéro 6.
Bit de sortie	%Q0.2	Activer la sortie %Q0.2
Jour de début	21	Débuter l'activité le 21 juin
Mois de début	Juin	Débuter l'activité en juin
Heure de début	21	Débuter l'activité à 21:00
Jour de fin	21	Arrêter l'activité le 21 septembre
Mois de fin	Septembre	Arrêter l'activité en septembre
Heure de fin	22	Arrêter l'activité à 22:00
Lundi	Case cochée	Exécuter l'activité le lundi
Mardi	Case non cochée	Aucune activité
Mercredi	Case cochée	Exécuter l'activité le mercredi
Jeudi	Case non cochée	Aucune activité
Vendredi	Case cochée	Exécuter l'activité le vendredi
Samedi	Case non cochée	Aucune activité
Dimanche	Case non cochée	Aucune activité

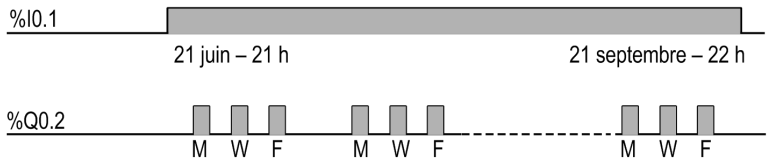
Grâce à ce programme, le `Schedule blocks` peut être désactivé à l'aide d'un commutateur ou d'un détecteur d'humidité relié à l'entrée %I0.1 :

Réseau	Instruction	Commentaire
0	LD %I0.1 ST %SW114:X6	Dans cet exemple, %SCH6 est validé.

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Chronogramme

Le chronogramme suivant illustre l'activation de la sortie %Q0.2 :



Chapitre 6

Fonction PID

Contenu de ce chapitre

Ce chapitre contient les sous-chapitres suivants :

Sous-chapitre	Sujet	Page
6.1	Modes de marche des PID	220
6.2	Configuration Auto-Tuning du PID	222
6.3	Configuration standard du PID	226
6.4	Assistant PID	238
6.5	Programmation du PID	251

Sous-chapitre 6.1

Modes de marche des PID

Modes de fonctionnement PID Modes de fonctionnement

Introduction

Le contrôleur `PID` SoMachine Basic propose quatre modes de fonctionnement, configurables dans l'onglet **Général** ([voir page 241](#)) de l'**Assistant PID** de SoMachine Basic.

Ces modes de fonctionnement `PID` sont les suivants :

- Mode PID
- Mode AT + PID
- Mode AT
- Adresse de mot

Mode PID

Le mode `PID` simple est actif par défaut, au démarrage du contrôleur `PID`. Les valeurs de gain K_p , T_i et T_d à spécifier dans l'onglet **PID** ([voir page 245](#)) doivent être connues à l'avance pour contrôler correctement le processus. Vous pouvez choisir le type de correcteur du contrôleur (`PID` ou `PI`) dans l'onglet **PID** de l'écran ([voir page 238](#)) **Assistant PID**. Si le type de correcteur `PI` est sélectionné, le champ **Td** est désactivé.

Avec le mode `PID`, la fonction Auto-Tuning est désactivée et l'onglet **AT** ([voir page 247](#)) de l'écran **Assistant Configuration** est indisponible.

Mode AT + PID

Dans ce mode, la fonction Auto-Tuning est active au démarrage du contrôleur `PID`. La fonction Auto-Tuning calcule ensuite les valeurs de gain K_p , T_i et T_d ([voir page 245](#)) et le type d'action `PID` ([voir page 249](#)). A l'issue de la séquence d'Auto-Tuning, le contrôleur passe en mode `PID` pour la nouvelle consigne, en utilisant les paramètres calculés par Auto-Tuning.

Si l'algorithme d'Auto-Tuning détecte une erreur ([voir page 255](#)) :

- Aucun `PID` n'est calculé.
- La sortie d'Auto-Tuning est réglée sur celle appliquée au processus avant le début de l'Auto-Tuning.
- Un message d'erreur apparaît dans la liste déroulante **Liste des états du PID**.
- Le contrôle du `PID` est annulé.

En mode `AT + PID`, la transition de Auto-Tuning à `PID` est automatique et transparente.

Mode AT

Dans ce mode, la fonction Auto-Tuning est active au démarrage du contrôleur `PID` et calcule automatiquement les valeurs de gain `Kp`, `Ti` et `Td` ([voir page 245](#)) ainsi que le type d'action `PID` ([voir page 249](#)). Après la convergence du processus Auto-Tuning et la détermination des paramètres `Kp`, `Ti` et `Td`, ainsi que du type d'action `PID` ([voir page 249](#)) (ou après détection d'une erreur dans l'algorithme Auto-Tuning), la sortie numérique d'Auto-Tuning est mise à 0 et le message **Auto-Tuning Complete** s'affiche dans la liste déroulante List of PID States ([voir page 255](#)). Le contrôleur du `PID` s'arrête et attend. Les coefficients `PID` `Kp`, `Ti` et `Td` calculés sont disponibles dans leur mot mémoire respectif (`%MWx`).

Adresse de mot

Ce mode `PID` est sélectionné en affectant la valeur souhaitée à l'adresse de mot associée à cette sélection :

- `%MWxx = 0` : le contrôleur est désactivé.
- `%MWxx = 1` : le contrôleur fonctionne en mode `PID` simple.
- `%MWxx = 2` : le contrôleur fonctionne en mode `AT+ PID`.
- `%MWxx = 3` : le contrôleur ne fonctionne qu'en mode `AT`.
- `%MWxx = 4` : le contrôleur fonctionne en mode `PID` simple, avec le type de correcteur `PI`.

Le mode `word address` vous permet de gérer le mode de fonctionnement du contrôleur `PID` avec l'application et donc de l'adapter à vos besoins.

Sous-chapitre 6.2

Configuration Auto-Tuning du PID

Configuration d'Auto-Tuning du PID

Introduction

Cette section détaille les étapes nécessaires pour configurer le contrôleur SoMachine Basic `PID` avec la fonction Auto-Tuning (AT).

Cette section contient les étapes suivantes :

Etape	Rubrique
1	Configuration de la voie analogique (<i>voir page 222</i>)
2	Conditions requises pour la configuration du PID (<i>voir page 222</i>)
3	Configuration du PID (<i>voir page 223</i>)
4	Configuration du contrôle (<i>voir page 224</i>)

Etape 1 : Configuration de la voie analogique

Un contrôleur `PID` utilise un signal de retour analogique (appelé valeur de processus) afin de calculer l'algorithme utilisé pour réguler le processus. Le Logic Controller a une entrée analogique qui permet de mémoriser cette valeur de processus. Pour plus d'informations sur la configuration de cette entrée analogique, consultez le *Guide de programmation* de votre Logic Controller.

Si une sortie analogique est utilisée pour piloter le système à contrôler, vérifiez qu'elle est correctement configurée. Vérifiez le module d'extension de sorties analogiques de votre Logic Controller.

Etape 2 : Conditions requises pour la configuration du PID

Avant de configurer le contrôleur `PID`, vérifiez que les phases suivantes ont été menées à bien :

Phase	Description
1	PID activé dans le programme (<i>voir page 252</i>)
2	Configuration du mode de scrutation périodique (<i>voir page 254</i>)

Étape 3 : Configuration du PID

Utilisez une sortie à semi-conducteur en association avec la fonction PID. L'utilisation d'une sortie relais risque de provoquer rapidement le dépassement de son nombre limite de cycles de vie, rendant ainsi le relais hors service, soit parce que les contacts restent collés, soit parce qu'ils se bloquent à l'état ouvert.

AVERTISSEMENT

EQUIPEMENT INOPERANT OU COMPORTEMENT IMPREVU DE L'EQUIPEMENT

- N'utilisez pas de sorties relais en conjonction avec la fonction PID.
- Utilisez uniquement des sorties à semi-conducteur si une sortie numérique est requise pour entraîner le système à contrôler.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Pour configurer un contrôleur **PID** avec la fonction Auto-Tuning, procédez comme suit :

Étape	Action
1	Dans l' General tab (<i>voir page 241</i>) de l'écran Assistant PID (en mode hors ligne), sélectionnez AT+PID (ou AT) ou sélectionnez le paramètre Adresse de mot pour régler le mot associé sur 2 ou 3, dans Operating Modes (<i>voir page 220</i>).
2	Cochez la case associée à Etats de PID et indiquez l'adresse du mot mémoire dans le champ.
3	Dans l' Input tab (<i>voir page 244</i>), indiquez l'adresse de l'entrée analogique utilisée comme mesure.
4	Si le paramètre Conversion ou Alarmes est requis, consultez l'onglet Entrée (<i>voir page 244</i>) de l'écran Assistant PID .
5	Dans l'onglet PID (<i>voir page 245</i>), indiquez la valeur de la consigne. En règle générale, cette valeur est une adresse mémoire ou une entrée analogique.
6	Le champ Type de correcteur dans l'onglet PID doit être réglé sur PID ou PI .
7	Sélectionnez les valeurs suivantes dans le champ Paramètres de l'onglet PID : Kp (x 0,01) , Ti (x 0,1 s) et Td (x 0,1 s) . Lorsque AT+PID ou AT est sélectionné dans Operating modes (<i>voir page 220</i>), les paramètres doivent être des adresses de mot mémoire (%MWxx) pour que l'algorithme Auto-Tuning indique la valeur calculée des paramètres.
8	Indiquez la Période d'échantillonnage du PID (Ts (<i>voir page 230</i>))) dans l'onglet PID . La Période d'échantillonnage est un paramètre essentiel qui doit être configuré avec précaution.
9	Dans l'onglet AT , le champ Mode AT doit être réglé sur Allow par défaut. Sélectionnez le Correcteur dynamique dans la liste contenant Fast , Moyen , Lent ou Adresse de mot . Pour plus d'informations, consultez l'onglet AT de l'Assistant PID (<i>voir page 238</i>).

Étape	Action
10	Dans l'onglet AT , entrez le bit mémoire du Déclencheur AT qui va stocker la valeur du changement d'étape pendant l'Auto-Tuning. Pour plus d'informations, consultez l'onglet AT de l'Assistant PID (voir page 238).
11	Dans le Output tab (voir page 249), sélectionnez Adresse de bit dans la liste du champ Action . Entrez l'adresse du bit mémoire dans le champ Bit . Le cas échéant, configurez les limites dans le champ Limites de l'onglet Sortie (voir page 249). Dans le champ Analog output , indiquez l'adresse du mot : une sortie analogique ou un mot mémoire. Sélectionnez Autoriser dans Output PWM (voir page 249). En mode manuel, entrez la valeur dans le champ Période (0,1 s) ou l'adresse du mot mémoire de la sortie dans le champ Sortie . Pour plus d'informations sur le fonctionnement du mode manuel, consultez la section Onglet Sortie (voir page 249).
12	Cliquez sur OK pour confirmer la configuration du contrôleur PID.

Étape 4 : Configuration du contrôle

Utilisez une sortie à semi-conducteur en association avec la fonction PID. L'utilisation d'une sortie relais risque de provoquer rapidement le dépassement de son nombre limite de cycles de vie, rendant ainsi le relais hors service, soit parce que les contacts restent collés, soit parce qu'ils se bloquent à l'état ouvert.

 AVERTISSEMENT
EQUIPEMENT INOPERANT OU COMPORTEMENT IMPREVU DE L'EQUIPEMENT • N'utilisez pas de sorties relais en conjonction avec la fonction PID. • Utilisez uniquement des sorties à semi-conducteur si une sortie numérique est requise pour entraîner le système à contrôler. Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Pour commencer une opération en mode de fonctionnement (*voir page 220*) AT+PID, procédez comme suit :

Étape	Action
1	Connectez le PC au contrôleur et transférez l'application.
2	Faites passer le contrôleur en mode RUN.

NOTE : Avant de faire passer le contrôleur en mode RUN, vérifiez que les conditions de fonctionnement de la machine autorisent ce mode pour le reste de l'application.

Étape	Action
1	Créez une table d'animation contenant les objets définis lors de la configuration. Pour plus d'informations sur la création de tables d'animation, consultez le document <i>SoMachine Basic - Guide d'utilisation</i> .
2	Vérifiez la cohérence de la valeur de processus et des valeurs de l'application. Ce test est important car le bon fonctionnement du contrôleur PID dépend de la précision de la mesure. Si vous avez des doutes concernant la précision de la mesure, mettez le Logic Controller en mode STOP et vérifiez le câblage des voies analogiques. Si l'actionneur n'est pas contrôlé : ● Pour la sortie analogique, vérifiez la tension ou l'intensité de sortie de la voie analogique. ● Pour la sortie PWM, vérifiez : ● que le voyant de la sortie dédiée est allumé ; ● le câblage des alimentations et du circuit 0 V ; ● que l'actionneur est sous tension.
3	Dans la table d'animation, vérifiez que : ● le mode de sortie est automatique ; ● tous les paramètres de votre application ont les valeurs appropriées.
4	Configurez la période de scrutation du Logic Controller, de sorte que la Période d'échantillonnage (Ts) du contrôleur PID soit un multiple exact de la période de scrutation. Pour plus d'informations sur la détermination de la période d'échantillonnage, consultez la section Optimisation du PID (<i>voir page 230</i>).
5	Lorsque la séquence d'Auto-Tuning est terminée, les paramètres Kp , Ti et Td sont stockés dans la RAM du Logic Controller. Les valeurs restent enregistrées tant que l'application est valide (mise hors tension inférieure à 30 jours) et qu'aucun redémarrage à froid n'est effectué.

Le processus Auto-Tuning est répété chaque fois qu'un front montant est détecté sur le bit mémoire **Déclencheur AT**.

Sous-chapitre 6.3

Configuration standard du PID

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Configuration de l'adresse de mot du PID	227
Réglage du PID avec la fonction Auto-Tuning (AT)	230
Mode Manuel	233
Détermination de la période d'échantillonnage (Ts)	235

Configuration de l'adresse de mot du PID

Introduction

Cette section détaille les étapes nécessaires pour configurer le contrôleur `PID` de SoMachine Basic en mode de fonctionnement (*voir page 220*) Adresse de mot. Ce mode offre une plus grande souplesse d'utilisation que les autres modes `PID`.

Cette section contient les étapes suivantes :

Etape	Rubrique
1	Conditions préalables à la configuration du PID (<i>voir page 227</i>)
2	Configuration du PID (<i>voir page 227</i>)
3	Configuration du contrôle (<i>voir page 228</i>)

Etape 1 : Conditions préalables à la configuration du PID

Avant de configurer le contrôleur `PID`, vérifiez que les phases suivantes ont été menées à bien :

Phase	Description
1	Une entrée analogique est configurée ainsi qu'une sortie analogique, le cas échéant. Consultez le <i>Guide de programmation</i> de votre Logic Controller.
2	PID activé dans le programme (<i>voir page 252</i>)
3	Configuration du mode de scrutation périodique (<i>voir page 254</i>)

Etape 2 : Configuration du PID

Utilisez une sortie à semi-conducteur en association avec la fonction PID. L'utilisation d'une sortie relais risque de provoquer rapidement le dépassement de son nombre limite de cycles de vie, rendant ainsi le relais hors service, soit parce que les contacts restent collés, soit parce qu'ils se bloquent à l'état ouvert.

AVERTISSEMENT

EQUIPEMENT INOPERANT OU COMPORTEMENT IMPREVU DE L'EQUIPEMENT

- N'utilisez pas de sorties relais en conjonction avec la fonction PID.
- Utilisez uniquement des sorties à semi-conducteur si une sortie numérique est requise pour entraîner le système à contrôler.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Les étapes suivantes expliquent comment implémenter un contrôleur `PID` en mode **Adresse de mot**. Pour plus d'informations sur la configuration du `PID`, consultez la section Assistant PID (*voir page 238*).

Afin de modifier dynamiquement les paramètres **PID** (en modes hors ligne et en ligne), entrez les adresses de mémoire dans les champs appropriés, afin d'éviter tout basculement en mode hors ligne pour modifier des valeurs à la volée.

Étape	Action
1	Dans l'onglet Général de l'écran PID Assistant (en mode hors ligne), sélectionnez Adresse de mot dans la liste déroulante Operating Modes . Cochez la case associée à Etats de PID et indiquez l'adresse du mot mémoire dans le champ.
2	Dans l'onglet Entrée (voir page 244), indiquez l'adresse de l'entrée analogique utilisée comme mesure. Si le paramètre Conversion ou Alarmes est requis, consultez l'onglet Entrée (voir page 244) de l'Assistant PID (voir page 238).
3	Dans l'onglet PID , entrez une valeur dans le champ Point de consigne . En règle générale, cette valeur est une adresse mémoire ou une entrée analogique. Les Paramètres (Kp, Ti et Td) doivent être des adresses de mot mémoire (%MWxx) . Renseignez le champ Période d'échantillonnage (Ts (voir page 245)) dans l'onglet PID (voir page 245). Ce paramètre peut également être un mot mémoire (dont la valeur est configurable à l'aide de la table d'animation). En mode Adresse de mot , le champ Type de correcteur affiche Auto et est grisé (non modifiable manuellement).
4	Dans l'onglet AT , le mode AT sélectionné doit être Allow . Renseignez les champs Correcteur dynamique et Déclencheur AT . Pour plus d'informations, consultez l'onglet AT (voir page 247) dans l'écran Assistant PID .
5	Dans l'onglet Sortie , le champ Action doit afficher Adresse de bit . Entrez une adresse de bit mémoire . Le cas échéant, vous pouvez configurer les limites dans le champ Limites de l'onglet Sortie (voir page 249). Dans le champ Analog output , indiquez l'adresse du mot : une sortie analogique ou un mot mémoire. Le cas échéant, choisissez une option dans le champ PWM sortie . Pour plus d'informations, consultez l'onglet Sortie (voir page 249) de l'Assistant PID (voir page 238).
6	Cliquez sur OK pour confirmer la configuration du contrôleur PID .

Étape 3 : Vérification de la configuration

Étape	Action
1	Connectez le PC au Logic Controller et transférez l'application.
2	Faites passer le Logic Controller en mode RUN.

NOTE : Avant de faire passer le contrôleur en mode RUN, vérifiez que les conditions de fonctionnement de la machine autorisent ce mode pour le reste de l'application. La procédure est la même que celle utilisée dans les modes **AT** et **AT+PID**. La configuration Adresse de mot vous permet de modifier les modes de fonctionnement du PID par voie logicielle. En mode PID, la procédure est beaucoup plus simple, car les paramètres (Kp, Ti, Td et Ts) sont connus et aucun Auto-Tuning n'est nécessaire.

Le tableau suivant détaille la procédure générique permettant de configurer le contrôleur **PID** :

Étape	Action
1	Créez une table d'animation contenant les objets définis lors de la configuration. Pour plus d'informations, consultez le document <i>SoMachine Basic - Guide d'utilisation</i> .
2	Vérifiez la cohérence de la valeur de processus et des autres valeurs dans la table d'animation. Si vous avez des doutes concernant la précision de la mesure, mettez le Logic Controller en mode STOP et vérifiez le câblage des voies analogiques. Si vous constatez que l'actionneur n'est pas contrôlé : ● Pour la sortie analogique, vérifiez la tension ou l'intensité de sortie de la voie analogique. ● Pour la sortie PWM, vérifiez : ● que le voyant de la sortie dédiée est allumé ; ● le câblage des alimentations et du circuit 0 V ; ● que l'actionneur est sous tension.
3	Configurez la période de scrutation du Logic Controller, de sorte que la Période d'échantillonnage (Ts) du contrôleur PID soit un multiple exact de la période de scrutation. Pour plus d'informations sur la période d'échantillonnage, consultez la section Détermination de la période d'échantillonnage (<i>voir page 235</i>).
4	Si vous prévoyez d'utiliser la fonction Auto-Tuning (<i>voir page 230</i>), il se peut que vous deviez activer le Mode manuel (<i>voir page 233</i>) pour connaître le Correcteur dynamique et le Déclencheur AT définis dans l'onglet AT (<i>voir page 247</i>) de l' Assistant PID .
5	Mettez le contrôleur de boucle sous tension à l'aide de la table d'animation : ● Configurez le mode de fonctionnement (<i>voir page 220</i>). ● Activez le contrôleur de PID (<i>voir page 254</i>). ● Modifiez les valeurs définies lors de la configuration (<i>voir page 227</i>), selon le mode de fonctionnement sélectionné.

Réglage du PID avec la fonction Auto-Tuning (AT)

Introduction

Le mode Auto-Tuning permet de régler automatiquement K_p , T_i , T_d et les paramètres d'action pour affiner la convergence de la fonction `PID`. La fonction Auto-Tuning fournie par SoMachine Basic est particulièrement performante pour régler automatiquement les processus thermiques.

Cette section contient les rubriques suivantes :

- Conditions requises par la fonction Auto-Tuning
- Description du processus Auto-Tuning
- Stockage des coefficients calculés
- Réglage des paramètres du `PID`
- Répétition d'Auto-Tuning
- Limites de l'utilisation d'Auto-Tuning et du contrôle du `PID`

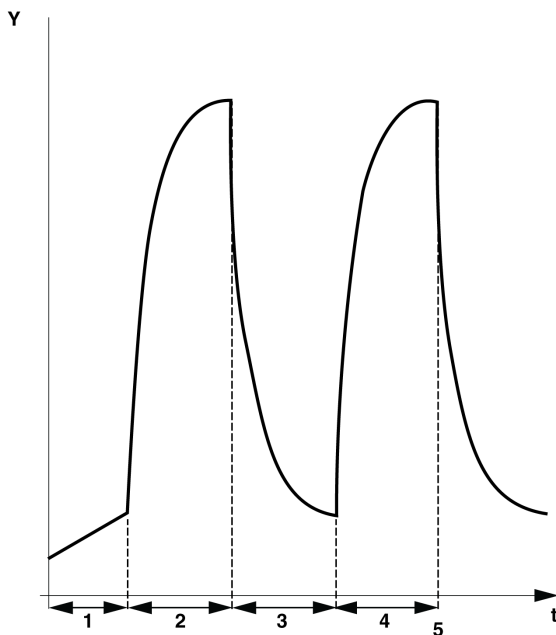
Conditions requises par la fonction Auto-Tuning

Lorsque vous utilisez la fonction Auto-Tuning, vérifiez que le processus de contrôle et le Logic Controller remplissent les conditions suivantes :

- Conditions requises par le processus :
 - Le processus doit être un système stable en boucle ouverte.
 - Le processus doit être principalement linéaire sur toute la plage de fonctionnement.
 - La réponse du processus à une modification du niveau de la sortie analogique doit suivre un schéma transitoire asymptotique.
 - Le processus est prêt, avec une entrée nulle au début de la séquence Auto-Tuning.
 - Le processus doit être intégralement exempt de perturbations. Sinon, les paramètres calculés seront incorrects ou le processus Auto-Tuning ne fonctionnera pas correctement.
- Configuration requise :
 - Configurez le Logic Controller en mode de scrutation périodique pour garantir la bonne exécution de la fonction Auto-Tuning.
 - N'utilisez la fonction Auto-Tuning que lorsqu'aucun autre contrôleur de `PID` n'est en cours d'exécution.
 - Configurez les coefficients K_p , T_i et T_d comme des adresses de mot mémoire (%MWxx).
 - Sélectionnez une adresse de bit mémoire (%Mxx) comme type d'action dans l'onglet **Sortie**.

Description du processus Auto-Tuning

Le processus Auto-Tuning se divise en 4 phases consécutives. Chaque phase doit être finalisée pour réussir l'Auto-Tuning. La courbe de réponse du processus et le tableau suivants décrivent les 4 phases de la fonction Auto-Tuning du PID dans SoMachine Basic :



Les phases d'Auto-Tuning sont décrites dans le tableau suivant :

Phase d'Auto-Tuning	Description
1	La phase de stabilisation ⁽¹⁾ débute lorsque vous lancez le processus Auto-Tuning. Au cours de cette phase, la fonction Auto-Tuning vérifie si la valeur de processus est prête.
2	La première modification d'étape est appliquée au processus. Elle génère une réponse d'étape similaire à celle illustrée dans le schéma ci-dessus.
3	La phase de relâche ⁽¹⁾ débute lorsque la première réponse d'étape est stabilisée.
4	La deuxième modification d'étape est appliquée au processus avec la même ampleur et de la même manière que dans la phase 2 décrite ci-avant. Le processus Auto-Tuning s'arrête. La valeur de processus est restaurée ⁽¹⁾ , et les paramètres d'Auto-Tuning sont calculés et stockés dans leur mot mémoire respectif. Consultez la description (voir page 232) du stockage des coefficients calculés.
(1) La dernière sortie appliquée au processus avant le début de l'Auto-Tuning est utilisée comme point de début et de relâche du processus Auto-Tuning.	

Stockage des coefficients calculés

Une fois la séquence d'Auto-Tuning terminée, les mots mémoire affectés aux coefficients Kp, Ti et Td et au type d'action sont définis à l'aide des valeurs calculées. Ces valeurs sont consignées dans la mémoire RAM et enregistrées dans le Logic Controller tant que l'application est correcte et qu'aucun démarrage à froid n'est effectué (%S0).

Si le système n'est pas influencé par des perturbations extérieures, les valeurs calculées peuvent être inscrites dans les paramètres du contrôleur PID (consultez l'onglet **PID** de l'Assistant [\(voir page 249\)](#) PID). Ainsi, le contrôleur PID peut être configuré en mode de marche PID.

Réglage des paramètres de PID

La méthode Auto-Tuning peut parfois fournir une commande très dynamique, conduisant à des dépassements indésirables pendant le changement des consignes. Pour affiner la régulation du processus grâce aux paramètres PID (Kp, Ti, Td) obtenus par Auto-Tuning, vous pouvez aussi régler ces paramètres manuellement, directement dans l'onglet **PID** de l'écran **Assistant PID** ou via les mots mémoire correspondants (%MW). Pour plus d'informations sur le réglage manuel des paramètres, consultez les annexes [\(voir page 267\)](#).

Répétition d'Auto-Tuning

Dans l'onglet **AT**, le **Déclencheur AT** permet de répéter la séquence d'Auto-Tuning. Le processus d'Auto-Tuning est lancé à chaque front montant du signal lié au **Déclencheur AT**.

Limites de l'utilisation d'Auto-Tuning

Les processus thermiques sont souvent assimilables au modèle de premier ordre avec temporisation pure. Deux paramètres principaux décrivent ce type de modèle :

- la constante de temps (τ),
- le délai de temporisation (θ).

L'**Auto-Tuning** est idéal pour les processus dans lesquels la constante de temps (τ) et le délai de temporisation (θ) répondent aux critères suivants :

- $10 \text{ s} < (\tau + \theta) < 2700 \text{ s}$ (c'est-à-dire 45 minutes)
- $2 < \tau / \theta < 20$

Mode Manuel

Introduction

Le mode manuel est accessible dans l'écran **Assistant PID** (**Output** tab ([voir page 249](#))). Il vous permet de contourner les ordres du **PID**. Le mode manuel a deux objectifs principaux :

- initialiser la configuration,
- déterminer la période d'échantillonnage.

Description

Le mode manuel vous permet de spécifier la **Output value** ([voir page 249](#)). Cette opération est idéale pour tester la réponse du système.

Pour activer le mode manuel, réglez le champ **Adresse de bit** sur 1 dans l'onglet **Sortie** ([voir page 249](#)). Si le paramètre Allow est sélectionné, le mode manuel est le seul mode accessible.

Application

Lorsque le mode manuel est actif, la sortie prend la valeur fixe que vous configurez. Cette sortie est comprise entre 0 et 10 000 (0 à 100 % pour la sortie PWM).

Vous pouvez également utiliser le mode manuel pour déterminer la limite de sortie minimum/maximum par approximation.

Par ailleurs, le mode manuel est obligatoire pour utiliser la méthode de courbe de réponse du processus ([voir page 235](#)), qui permet de trouver la période d'échantillonnage (T_s) correcte.

Démarrage du mode manuel

Avant de passer en mode manuel, vérifiez que le commutateur RUN/STOP du Logic Controller est en position RUN.

Pour passer en mode manuel à l'aide d'une table d'animation, procédez comme suit :

Etape	Description
1	Activez le mode manuel en réglant le bit mémoire dédié sur 1. Pour plus d'informations, consultez l'onglet Sortie (voir page 249).
2	Si vous utilisez le paramètre PWM, configurez la période PWM à la valeur souhaitée.
3	Réglez le mot mémoire associé au mode de fonctionnement sur 1 dans l'onglet Général (voir page 241) de l' Assistant PID (mode PID). Pour plus de détails sur les modes de fonctionnement avec le paramètre Adresse de mot, consultez la description du mode de fonctionnement (voir page 220).
4	Réglez le mot mémoire associé à la sortie manuelle dans l'onglet Sortie (voir page 249) à la valeur souhaitée. Cette valeur de consigne peut être sélectionnée plusieurs fois, à condition que le système conserve son état initial.
5	Activez le contrôleur (voir page 227) de boucle.

Arrêt du mode manuel

Pour arrêter le mode manuel à l'aide d'une table d'animation, procédez comme suit :

Etape	Description
1	Désactivez le contrôleur (voir page 227) de boucle.
2	Inhibez le mode manuel en réglant le bit mémoire dédié sur 0. Pour plus d'informations, consultez l'onglet Sortie (voir page 249).
3	Réglez le mot mémoire associé au mode de fonctionnement dans l'onglet Général (voir page 241) du contrôleur de PID sur 0. Pour plus d'informations sur les modes de fonctionnement avec le paramètre Adresse de mot, consultez la description des modes de fonctionnement (voir page 220).
4	Réglez le mot mémoire associé à la sortie manuelle dans l'onglet Sortie (voir page 249) sur 0.

Détermination de la période d'échantillonnage (Ts)

Introduction

La période d'échantillonnage (Ts) est le principal paramètre de régulation du PID. La période d'échantillonnage (Ts) doit être définie avec soin dans l'onglet **PID** (voir page 245) de l'écran **Assistant PID**. Ce paramètre est étroitement corrélé à la constante de temps (τ) du processus à réguler.

Cette section décrit l'utilisation du mode en ligne et les deux méthodes permettant de déterminer la période d'échantillonnage (Ts) :

- Méthode de la courbe de réponse du processus
- Méthode des essais et erreurs

Méthode de la courbe de réponse du processus

Cette méthode en boucle ouverte permet de déterminer la constante de temps du processus à réguler. Tout d'abord, il convient de vérifier que le processus peut être décrit par un modèle de premier ordre avec délai de temporisation. Le principe est relativement simple. Il consiste à appliquer un changement d'étape à l'entrée du processus tout en enregistrant la courbe de sortie du processus. Ensuite, on utilise une méthode graphique pour déterminer le délai de temporisation du processus.

Pour déterminer la période d'échantillonnage (Ts) à l'aide de la méthode de la courbe de réponse du processus, procédez comme suit :

Etape	Action
1	Supposons que vous ayez déjà configuré les différents paramètres dans les onglets Général , Entrée , PID , AT et Sortie du PID.
2	Sélectionnez l'onglet Sortie (voir page 249) dans l'écran Assistant PID .
3	Sélectionnez Allow ou Address bit dans la liste déroulante Mode manuel pour autoriser la sortie manuelle.
4	Saisissez une valeur élevée (comprise entre 5000 et 10 000) dans le champ Sortie .
5	Chargez votre application dans le Logic Controller. Pour plus d'informations sur le chargement d'une application, consultez le document <i>SoMachine Basic - Guide d'utilisation</i> .
6	Exécutez le PID et vérifiez l'augmentation de la courbe de réponse.
7	Lorsque la courbe de réponse a atteint un état stable, arrêtez la mesure du PID.
(1) L'unité de base de la période d'échantillonnage est 10 ms. Par conséquent, vous devez arrondir la valeur Ts aux 10 ms supérieures ou inférieures. (2) Vous devez choisir "n" pour que la période de scrutation obtenue soit un entier positif compris entre 2 à 150 ms.	

Etape	Action
8	Utilisez la méthode graphique suivante pour déterminer la constante de temps (τ) du processus de régulation : 1. Calculez la sortie de la valeur du processus à 63 % d'augmentation ($S_{[63 \%]}$) à l'aide de la formule suivante : $S_{[63 \%]} = S_{[initial]} + (S_{[final]} - S_{[initial]}) \times 63 \%$ 2. Calculez graphiquement l'abscisse du temps ($t_{[63 \%]}$) correspondant à $S(63 \%)$. 3. Calculez graphiquement le temps initial ($t_{[initial]}$) correspondant au début de l'augmentation de la réponse du processus. 4. Calculez la constante de temps (τ) du processus de régulation en utilisant la relation suivante : $\tau_{[63 \%][initial]}$
9	Calculez la période d'échantillonnage (T_s) ⁽¹⁾ en fonction de la valeur (τ) que vous avez déterminée à l'étape précédente, en utilisant la règle suivante : $T_s = \tau / 75$.
10	Sélectionnez Périodique dans le champ Période de scrutation, afin que la période d'échantillonnage (T_s) soit un multiple de la période de scrutation : Période de scrutation = T_s / n , où n est un entier positif ⁽²⁾ .
(1) L'unité de base de la période d'échantillonnage est 10 ms. Par conséquent, vous devez arrondir la valeur T_s aux 10 ms supérieures ou inférieures. (2) Vous devez choisir "n" pour que la période de scrutation obtenue soit un entier positif compris entre 2 à 150 ms.	

Méthode des essais et erreurs

Cette méthode consiste à définir la période d'échantillonnage de la fonction Auto-Tuning par approximations successives, jusqu'à ce que l'algorithme converge vers des valeurs satisfaisantes de K_p , T_i et T_d .

NOTE : Contrairement à la méthode de courbe de réponse du processus, la méthode des essais et erreurs n'est basée sur aucune loi d'approximation de la réponse au processus. Cependant, elle présente l'avantage de converger vers une valeur de la période d'échantillonnage, du même ordre de grandeur que la valeur actuelle.

Pour définir la valeur d'Auto-Tuning par cette méthode, procédez comme suit :

Etape	Action
1	Cliquez sur l'onglet AT dans la fenêtre de configuration du PID .
2	Sélectionnez 10 000 dans le champ Output limitation de la fonction Auto-Tuning.
3	Chargez votre application dans le Logic Controller. Pour plus d'informations sur le chargement d'une application, consultez le document SoMachine Basic - Guide d'utilisation (voir <i>SoMachine Basic, Operating Guide</i>).
4	Cliquez sur l'onglet PID dans l'écran Assistant PID .
(1) Si vous n'avez pas de première indication sur la plage possible de la période d'échantillonnage, optez pour une valeur la plus petite possible : (2) Si la régulation du PID fournie par cet ensemble de paramètres de contrôle ne produit pas des résultats totalement satisfaisants, affinez l'évaluation de la période d'échantillonnage jusqu'à obtenir les paramètres de contrôle K_p, T_i et T_d appropriés.	

Etape	Action
5	Indiquez la première ou la n ^{ième} hypothèse dans le champ Période d'échantillonnage ⁽¹⁾ .
6	Lancez Auto-Tuning (<i>voir page 222</i>).
7	Patiencez jusqu'à la fin du processus Auto-Tuning.
8	Deux cas peuvent se produire : ● Auto-Tuning aboutit : passez à l'étape 10. ● Auto-Tuning n'aboutit pas : consultez la section Codes d'erreur détectée d'Auto-Tuning (<i>voir page 256</i>). Cela signifie que l'approximation de la période d'échantillonnage (Ts) est incorrecte. Essayez une nouvelle valeur de Ts et répétez les étapes 3 à 8 autant de fois que nécessaire jusqu'à ce que le processus Auto-Tuning converge.
9	Suivez ces instructions pour fournir une nouvelle valeur de Ts : ● Auto-Tuning génère le code d'erreur détectée 800C hex. Cela signifie que la période d'échantillonnage Ts est trop importante. Diminuez la valeur de Ts. ● Auto-Tuning génère le code d'erreur détectée 800A hex. Cela signifie que la période d'échantillonnage Ts est trop petite. Augmentez la valeur de Ts.
10	Au besoin, ajustez les paramètres de contrôle du PID ⁽²⁾ (Kp, Ti et Td) dans l'onglet PID (<i>voir page 245</i>) de l'écran Assistant PID .
(1) Si vous n'avez pas de première indication sur la plage possible de la période d'échantillonnage, optez pour une valeur la plus petite possible : (2) Si la régulation du PID fournie par cet ensemble de paramètres de contrôle ne produit pas des résultats totalement satisfaisants, affinez l'évaluation de la période d'échantillonnage jusqu'à obtenir les paramètres de contrôle Kp, Ti et Td appropriés.	

Mode en ligne

En mode en ligne, lorsque le Logic Controller exécute la tâche périodique, la valeur affichée dans le champ Ts (de l'écran (*voir page 238*) **Assistant PID**) peut être différente du paramètre saisi (%MW). La valeur Ts est un multiple de la tâche périodique, tandis que la valeur %MW est celle qui est lue par le Logic Controller.

Sous-chapitre 6.4

Assistant PID

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Accès à l'Assistant PID	239
Onglet Général	241
Onglet Entrée	244
Onglet PID	245
Onglet AT	247
Onglet Sortie	249

Accès à l'Assistant PID

Introduction

Utilisez la fenêtre **Assistant PID** de SoMachine Basic pour configurer le contrôleur contrôleur `PID`.

Assistant de configuration

Dans le tableau des propriétés `PID`, cliquez sur le bouton **Configuration [...]**. L'écran **Assistant PID** s'affiche.

Cette illustration montre l'écran **Assistant PID** :

The screenshot shows the 'Assistant PID' window with a green title bar and a red close button. The window has a tabbed interface with tabs for 'Général', 'Entrée', 'PID', 'AT', and 'Sortie'. The 'Général' tab is selected. Inside the 'Général' tab, there is a 'Type' dropdown menu set to 'Non configuré', an 'Adresse de mot' text field, and a checkbox labeled 'Etats de PID' which is currently unchecked. At the bottom right of the window are 'OK' and 'Annuler' buttons.

L'écran **Assistant PID** affiche plusieurs onglets, selon que vous êtes en mode hors ligne ou en ligne :

Onglet	Mode d'accès	Lien
Général	Hors ligne	Onglet Général (<i>voir page 241</i>)
Entrée	Hors ligne	Onglet Entrée (<i>voir page 244</i>)
PID	Hors ligne	Onglet PID (<i>voir page 245</i>)
AT	Hors ligne	Onglet AT (<i>voir page 247</i>)
Sortie	Hors ligne	Onglet Sortie (<i>voir page 249</i>)

Dès qu'un mode de fonctionnement est sélectionné, les onglets contenant des champs vides à renseigner s'affichent comme suit  avec une bordure en rouge.

Onglet Général

Introduction

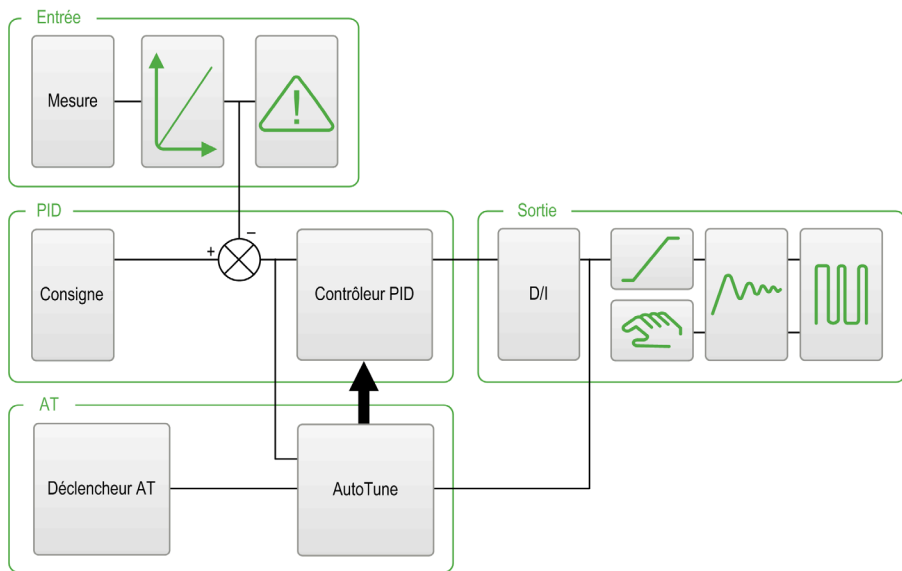
Cette section décrit l'onglet **Général** du PID. L'onglet **Général** s'affiche par défaut lorsque vous accédez à l'Assistant PID en mode hors ligne..

Description

Le tableau suivant décrit les paramètres de l'onglet **Général**.

Paramètre	Description
Operating Mode	Représente le mode PID à utiliser : ● Non configuré ● PID ● AT + PID ● AT ● Adresse de mot Pour plus d'informations sur les modes de fonctionnement, consultez la section Mode de fonctionnement PID (voir page 220).
Adresse de mot	Dans ce champ (%MWxx), vous pouvez spécifier le mot mémoire utilisé pour configurer le mode de fonctionnement dans le programme. Ce mot mémoire accepte quatre valeurs possibles selon le mode de fonctionnement que vous souhaitez définir : ● %MWx = 0 (PID désactivé) ● %MWx = 1 (pour configurer PID uniquement) ● %MWx = 2 (pour configurer PID+Auto tuning) ● %MWx = 3 (pour configurer Auto tuning uniquement) ● %MWx = 4 (pour configurer PI uniquement)
Etats de PID	Si vous cochez la case pour activer cette option, vous pouvez indiquer dans le champ associé (%MWxx) le mot mémoire utilisé par le contrôleur PID pour stocker l'état PID actuel, tout en exécutant le contrôleur PID et/ou la fonction Auto-Tuning. Pour plus d'informations, consultez la section Etats de PID et codes d'erreur détectée (voir page 255).

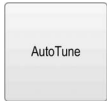
Assistant graphique



L'assistant graphique vous permet de visualiser comment la fonction **PID** s'articule. Ce graphique dynamique est mis à jour en fonction de la configuration.

Les icônes suivantes indiquent lorsqu'il est accessible ou ce qui se passe si vous cliquez dessus :

Affichage	Description
	Cliquez sur ce bouton pour afficher le champ Point de consigne de l'onglet PID (voir page 245).
	Cliquez sur ce bouton pour afficher l'onglet PID (voir page 245).
	Cliquez sur ce bouton pour afficher l'onglet Sortie (voir page 249).
	Cliquez sur ce bouton pour afficher l'onglet Entrée (voir page 244).

Affichage	Description
	Cliquez sur ce bouton pour afficher l'onglet AT (voir page 247).
	Cliquez sur ce bouton pour afficher l'onglet AT (voir page 247).
	Ce bouton s'affiche lorsque l'option Allow est cochée dans la zone Conversion de l'onglet Entrée (voir page 244).
	Ce bouton s'affiche lorsque l'option Allow est cochée dans la zone Alarmes de l'onglet Entrée (voir page 244).
	Ce bouton s'affiche si le champ Limites n'affiche pas l'option Inhiber dans la zone des limites de l'onglet Sortie (voir page 249).
	Ce bouton s'affiche si le mode manuel n'est pas Inhiber dans la zone manuelle de l'onglet Sortie (voir page 249).
	Cliquez sur ce bouton pour afficher l'onglet Sortie (voir page 249).
	Ce bouton s'affiche lorsque l'option Allow est cochée dans la zone PWM sortie de l'onglet Sortie (voir page 249).

Onglet Entrée

Introduction

Cette section décrit l'onglet **Entrée** du **PID**. L'onglet **Entrée** permet de saisir les paramètres d'entrée **PID**.

Cet onglet n'est accessible qu'en mode hors ligne et lorsqu'un mode de fonctionnement est sélectionné dans l'onglet **Général**.

Description

Le tableau suivant décrit les paramètres que vous pouvez définir.

Paramètre	Description	
Mesure	Indiquez la variable qui contient la valeur de processus à réguler. La plage par défaut est comprise entre 0 et 10 000. Vous pouvez saisir soit un mot mémoire (%MWxx), soit une entrée analogique.	
Conversion	Allow	Cochez cette case pour convertir la valeur de processus [0 à 10 000] en une plage linéaire [Min à Max] .
	Valeur min Valeur max	Indiquez les valeurs minimum et maximum de l'échelle de conversion. La valeur de processus est automatiquement remise à l'échelle dans l'intervalle [Valeur min. à Valeur max.] . Valeur min et Valeur max peuvent être des mots mémoire (%MWxx), des mots constants (%KWxx) ou une valeur comprise entre -32 768 et +32 767. Remarque : la Valeur min doit être inférieure à la Valeur max .
Alarmes	Allow	Cochez cette case pour activer les alarmes dans les variables d'entrée. Les valeurs d'alarme sont à déterminer par rapport à la valeur de processus obtenue après la phase de conversion. Elles doivent être comprises entre Valeur min et Valeur max lorsque la conversion est active. Sinon, elles seront comprises entre 0 et 10 000.
	Faible Sortie	Indiquez la valeur d'alarme basse dans le champ Faible . Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Le champ Sortie doit contenir l'adresse du bit qui sera mis à 1 une fois la limite basse atteinte. La Sortie peut être un bit mémoire (%Mxx) ou une sortie.
	Elevé Sortie	Indiquez la valeur d'alarme haute dans le champ Elevé . Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Le champ Sortie doit contenir l'adresse du bit qui sera mis à 1 une fois la limite haute atteinte. La Sortie peut être un bit mémoire (%Mxx) ou une sortie.

Onglet PID

Introduction

Utilisez l'onglet **PID** pour entrer les paramètres **PID** internes.

Cet onglet n'est accessible qu'en mode hors ligne et si un mode de fonctionnement est sélectionné dans l'onglet **Général**.

Description

Le tableau suivant décrit les paramètres que vous pouvez définir :

Paramètre	Description
Point de consigne	Spécifiez la valeur du point de consigne PID . Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Cette valeur doit donc être comprise entre 0 et 10 000 lorsque la conversion est inhibée. Sinon, elle doit être comprise entre la Valeur min. et la Valeur max. de la conversion.
Type de correcteur	Si le mode de fonctionnement PID ou AT + PID a déjà été choisi dans le tableau des propriétés PID , vous pouvez sélectionner le type de correcteur souhaité (PID ou PI) dans la liste déroulante. Si d'autres modes de fonctionnement (AT ou Adresse de mot) ont été choisis, le champ Type de correcteur affiche Auto et devient grisé (non modifiable manuellement). Si vous sélectionnez PI dans la liste déroulante, le paramètre Td est forcé à 0 et ce champ est désactivé.
(1) Lorsque Auto-Tuning est activé, il n'est plus nécessaire de configurer les paramètres Kp, Ti et Td, car ils le sont automatiquement par l'algorithme Auto-Tuning. Dans ce cas, vous ne devez saisir dans ces champs qu'une adresse de mot interne (%MWxx). Ne saisissez pas une constante ou une valeur directe lorsque Auto-Tuning est activé.	

Paramètre	Description
Paramètres ⁽¹⁾	Kp (x 0,01 s) Spécifiez le gain proportionnel du PID, multiplié par 100. Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. La plage valide pour le paramètre Kp est la suivante : $0 < Kp < 10\,000$. Remarque : si Kp est par erreur mis à 0 ($Kp \leq 0$ est incorrect), la valeur par défaut Kp=100 est automatiquement affectée par la fonction PID.
	Ti (x 0,1 s) Spécifiez le temps intégral d'une base de temps de 0,1 seconde. Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Elle doit être comprise entre 0 et 36 000. Remarque : pour désactiver l'action intégrale du PID, réglez ce coefficient sur 0.
	Td (x 0,1 s) Spécifiez le temps dérivé d'une base de temps de 0,1 seconde. Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Elle doit être comprise entre 0 et 10 000. Remarque : pour désactiver l'action dérivée du PID, réglez ce coefficient sur 0.
Période d'échantillonnage	Spécifiez la valeur de la période d'échantillonnage du PID pour une base de temps de 10^{-2} seconde (10 ms). Cette valeur peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur directe. Elle doit être comprise entre 1 (0,01 s) et 10 000 (100 s).
(1) Lorsque Auto-Tuning est activé, il n'est plus nécessaire de configurer les paramètres Kp, Ti et Td, car ils le sont automatiquement par l'algorithme Auto-Tuning. Dans ce cas, vous ne devez saisir dans ces champs qu'une adresse de mot interne (%MWxx). Ne saisissez pas une constante ou une valeur directe lorsque Auto-Tuning est activé.	

Onglet AT

Introduction

L'onglet **AT** concerne la fonction Auto-Tuning. Pour plus d'informations, consultez la section Réglage du PID avec Auto-Tuning (*voir page 230*).

Cet onglet n'est accessible qu'en mode hors ligne et si un mode de fonctionnement est sélectionné dans l'onglet **Général**.

Description

L'Auto-Tuning du **PID** est un processus en boucle ouverte, qui agit directement sur le processus de contrôle sans régulation ni autre limitation que celles imposées par la limite de la variable du processus et la consigne de sortie. Vous devez donc sélectionner avec soin les deux valeurs dans la plage autorisée par le processus, afin d'éviter toute surcharge éventuelle.

AVERTISSEMENT

FONCTIONNEMENT PID INSTABLE

- La limite de la valeur de processus (PV) et les valeurs de consigne de sortie doivent être définies en pleine connaissance de leurs effets sur la machine ou le processus.
- Pour la valeur de processus et la consigne de sortie, gardez des valeurs au sein de la plage admissible.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

AVERTISSEMENT

COMPORTEMENT INATTENDU DE L'EQUIPEMENT

N'utilisez pas de sortie à relais avec la fonction PID.

Le non-respect de ces instructions peut provoquer la mort, des blessures graves ou des dommages matériels.

Le tableau suivant décrit les paramètres que vous pouvez définir :

Champ	Description
Allow	Cochez cette case pour activer l'opération Auto-Tuning. Vous pouvez utiliser la case à cocher de deux façons, selon que vous définissez le mode de fonctionnement manuellement ou via une adresse de mot dans l'onglet Général de la fonction PID : ● Si vous sélectionnez PID+AT ou AT dans le champ Operating Mode de l'onglet Général (<i>voir page 241</i>), l'option Allow est automatiquement cochée et indisponible (vous ne pouvez pas la désactiver). ● Si vous sélectionnez le mode de fonctionnement via une adresse de mot %MWx (%MWx = 2 : PID + AT ; %MWx = 3 : AT), vous devez cocher l'option Allow manuellement pour pouvoir configurer les paramètres d'Auto-Tuning.
Correcteur AT dynamique	Ce paramètre vous permet de choisir la dynamique du processus AT. Il affecte la valeur du gain proportionnel (Kp), calculée par le processus AT.
Déclencheur AT	Ce paramètre vous permet de lancer le processus AT, chaque fois qu'un front montant est détecté sur le bit dédié (bit mémoire ou bit d'entrée numérique).

Coefficients calculés Kp, Ti et Td

Lorsque le processus Auto-Tuning est terminé, les coefficients **PID** Kp, Ti et Td sont stockés dans leurs mots mémoire respectif (%MWx).

Onglet Sortie

Introduction

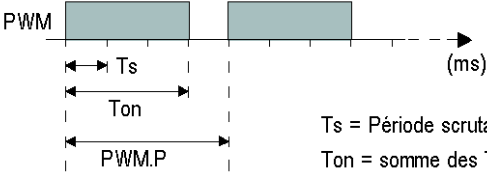
Cet onglet permet d'entrer les paramètres de sortie du PID.

Cet onglet n'est accessible qu'en mode hors ligne et si un mode de fonctionnement est sélectionné dans l'onglet **Général**.

Description

Le tableau suivant décrit les paramètres que vous pouvez définir :

Champ	Description
Action	Spécifiez ici le type d'action PID sur le processus. Trois options sont disponibles : Inverse, Direct et Adresse de bit. Si une augmentation de la sortie entraîne une augmentation de la mesure de la valeur de processus, choisissez l'action Inverse. Si vous constatez une diminution de la valeur de processus, choisissez l'action PID Direct. Si vous choisissez Adresse de bit⁽¹⁾, vous pouvez modifier le type d'action en changeant le bit associé qui est soit un bit mémoire (%Mxx) soit une adresse d'entrée (%Ix.y). Le bit mémoire est mis à 1 si l'action sélectionnée est Direct et à 0 si l'action sélectionnée est Inverse.
Limites	Spécifiez si des limites sont à appliquer à la sortie PID. Trois options sont disponibles : Activer, Désactiver et Adresse de bit. Sélectionnez Activer pour mettre le Bit à 1 ou sélectionnez Désactiver pour mettre le Bit à 0. Sélectionnez Adresse de bit pour gérer les limites du bit en modifiant le bit associé qui est soit un bit mémoire (%Mxx) soit une adresse d'entrée (%Ix.y). Définissez les limites haute et basse de la sortie PID. Min. ou Max. peut être un mot mémoire (%MWxx), un mot constant (%KWxx) ou une valeur comprise entre 1 et 10 000. Remarque : la valeur Min doit être inférieure à la valeur Max.
Mode manuel	Spécifiez si vous souhaitez faire passer le PID en mode manuel. Trois options sont disponibles : Activer, Désactiver et Adresse de bit. Si vous sélectionnez Adresse de bit, vous pouvez passer en mode manuel (bit à 1) ou automatique (bit à 0) à l'aide du programme, en modifiant le bit associé (bit mémoire (%Mxx) ou entrée). La Sortie du mode manuel doit contenir la valeur que vous désirez affecter à la sortie analogique lorsque le PID est en mode manuel (voir page 233). Cette Sortie peut être un mot (%MWxx) ou une valeur directe au format [0 à 10 000].
Sortie analogique	Spécifiez la sortie PID à utiliser en mode AT. Cette Sortie analogique⁽²⁾ peut être une adresse de mot mémoire ou de sortie analogique.
(1) Lorsque la fonction Auto-Tuning est activée, son algorithme détermine automatiquement le type d'action approprié (directe ou inverse) au processus de contrôle. Vous devez ensuite entrer un bit mémoire (%mxx) dans le champ Adresse de bit associé. (2) Indiquez une adresse mémoire (%MWxx) ou une adresse de sortie analogique (%QWx.y).	

Champ	Description
PWM sortie	Cochez cette case pour utiliser la fonction PWM du <code>PID</code>. Spécifiez la période de modulation dans le champ Période (0,1 s). Cette période, obligatoirement comprise entre 1 et 500, peut être un mot mémoire (<code>%MWxx</code>) ou un mot constant (<code>%KWxx</code>). La précision de PWM dépend de la période PWM et de la période de scrutation. La précision augmente lorsque le ratio de PWM (<code>%PWM.R</code>) a le plus grand nombre de valeurs. Par exemple, avec une période de scrutation de 20 ms et une période de PWM de 200 ms, <code>PWM.R</code> peut prendre les valeurs 0 %, 10 %, 20 %, 30 %, 40 %, 50 %, 60 %, 70 %, 80 %, 90 %, 100 %. Avec une période de scrutation de 50 ms et une période de PWM de 200 ms, <code>PWM.R</code> peut prendre les valeurs 0 %, 25 %, 50 %, 75 % et 100 % de la période <code>PWM.P</code>. <u>Exemple</u> : cas de <code>PWM.R = 75%</code>  Dans le champ Sortie, indiquez le bit de sortie PWM. Il peut être un bit mémoire (<code>%Mxx</code>) ou une adresse de sortie. Pour plus d'informations sur la fonction PWM, consultez le <i>Guide de la bibliothèque des fonctions</i> de votre Logic Controller. (1) Lorsque la fonction Auto-Tuning est activée, son algorithme détermine automatiquement le type d'action approprié (directe ou inverse) au processus de contrôle. Vous devez ensuite entrer un bit mémoire (<code>%mxx</code>) dans le champ Adresse de bit associé. (2) Indiquez une adresse mémoire (<code>%MWxx</code>) ou une adresse de sortie analogique (<code>%QWx.y</code>).

Sous-chapitre 6.5

Programmation du PID

Utilisation de la fonction PID

Cette section fournit des descriptions et des conseils de programmation pour utiliser la fonction **PID**.

Contenu de ce sous-chapitre

Ce sous-chapitre contient les sujets suivants :

Sujet	Page
Description	252
Programmation et configuration	254
Etats et codes d'erreur détectée de PID	255

Description

Introduction

Un **PID** (proportionnel-intégral-dérivé) est un mécanisme de boucle de contrôle générique (contrôleur) largement utilisé dans les systèmes de régulation industriels. Le **PID** utilise un algorithme qui met en œuvre trois paramètres constants : les valeurs proportionnelle, intégrale et dérivée, appelées P, I et D respectivement.

Principales fonctionnalités

Les principales fonctionnalités de la fonction PID de SoMachine Basic sont les suivantes :

- Entrée analogique
- Conversion linéaire de la mesure configurable
- Alarme haute et basse d'entrée configurable
- Sortie analogique ou PWM
- Coupure de la sortie configurable
- Action directe ou inverse configurable
- Fonction Auto-Tuning

Illustration

Cette fonction est la fonction **PID** dans l'éditeur de schéma à contacts de SoMachine Basic :



NOTE : Un espace doit séparer la mention PID de son numéro (par exemple, PID<espace>0).

Paramètres

Contrairement aux blocs fonction **Timer** ou **Counter**, il n'y a aucun bloc fonction **PID** dans SoMachine Basic. L'instruction `[PID x]` n'active que la fonction de boucle de contrôle **PID**, où `x` est le numéro du **PID**.

Pour configurer la fonction **PID**, ouvrez la fenêtre **Programmation**, cliquez sur **Outils** → **PID**, puis modifiez les propriétés du **PID** (pour les paramètres de configuration, consultez le tableau ci-dessous).

La fonction **PID** a les paramètres suivants :

Paramètre	Description	Valeur
Utilisée	Cochée si les E/S sont utilisées dans le projet.	True/False False (par défaut)
PID	Nom de l'objet PID actuel	Un programme ne peut contenir qu'un nombre limité de fonctions PID . Consultez le <i>Guide de programmation</i> de la plate-forme matérielle pour connaître le nombre maximum d'objets PID disponibles sur votre Logic Controller.
Symbole	Symbole de l'objet PID actuel	Symbole associé à cet objet PID . Pour plus d'informations, consultez la section Définition des symboles (<i>voir SoMachine Basic, Operating Guide</i>).
[...]	Bouton permettant de démarrer l'assistant	Cliquez dessus pour afficher l'écran Assistant PID . Pour plus d'informations, consultez la section Assistant PID (<i>voir page 238</i>).
Commentaire	Commentaire	Un commentaire peut être associé à cet objet.

Programmation et configuration

Introduction

Cette section décrit comment programmer et configurer le contrôleur SoMachine Basic `PID`.

Activation du contrôleur PID

L'exemple suivant active la boucle du contrôleur `PID 0` si le bit `%M0` est réglé à 1 :

Réseau	Instruction
0	LD %M0 [PID 0]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Mesure analogique du PID

La fonction `PID` effectue une correction du `PID` à l'aide d'une mesure analogique et d'une consigne, puis elle produit une commande analogique au même format ou une PWM sur une sortie numérique.

Pour utiliser le `PID` à pleine mesure (résolution maximale), configurez l'entrée analogique dédiée à la mesure du contrôleur `PID` au format [0 à 10 000]. Toutefois, le contrôleur `PID` fonctionne correctement si vous utilisez la configuration par défaut [0 à 4095].

Configuration de la période de scrutation

Lorsque vous utilisez des contrôleurs SoMachine Basic `PID`, vous devez configurer le mode de scrutation du Logic Controller sur **Périodique** (onglet **Programme, Tâches** → **Tâche maître**). En mode périodique, chaque scrutation du Logic Controller débute à intervalles réguliers, afin que le taux d'échantillonnage reste constant tout au long de la période de mesure. Pour plus d'informations sur la configuration du mode de scrutation, consultez le document *SoMachine Basic - Guide d'utilisation*.

En mode périodique, le système met le bit système `%S19` à 1 si la durée de scrutation du Logic Controller est supérieure à la période définie par le programme de l'utilisateur.

Etats et codes d'erreur détectée de PID

Introduction

Le contrôleur `PID` SoMachine Basic permet d'écrire l'état actuel du contrôleur `PID` et du processus Auto-Tuning dans un mot mémoire défini par l'utilisateur. Pour plus d'informations sur l'activation et la configuration du mot mémoire des états de `PID`, consultez l'onglet **Général** ([voir page 241](#)) de l'Assistant PID ([voir page 238](#)).

Le mot mémoire des états de `PID` peut enregistrer les types suivants d'informations `PID` :

- Etat actuel du contrôleur `PID`
- Etat actuel du processus Auto-Tuning
- Codes d'erreur détectée de `PID`
- Codes d'erreur détectée d'Auto-Tuning

NOTE : Le mot mémoire des états de `PID` est en lecture seule.

Mot mémoire des états de PID

Etat de PID	Description
0000 hex	Le contrôle <code>PID</code> est inactif.
2000 hex	Le contrôle <code>PID</code> est en cours.
4000 hex	La consigne <code>PID</code> est atteinte.

Mot mémoire des états d'Auto-Tuning

Etat d'Auto-Tuning	Description
0100 hex	La phase 1 de l'Auto-Tuning (voir page 231) est en cours.
0200 hex	La phase 2 de l'Auto-Tuning (voir page 231) est en cours.
0400 hex	La phase 3 de l'Auto-Tuning (voir page 231) est en cours.
0800 hex	La phase 4 de l'Auto-Tuning (voir page 231) est en cours.
1000 hex	Phase d'Auto-Tuning terminée

Codes d'erreur détectée de PID

Le tableau suivant décrit les potentielles erreurs détectées pendant le contrôle du PID :

Code d'erreur détectée	Description
8001 hex	Valeur du mode de fonctionnement hors plage.
8002 hex	Les valeurs minimum et maximum de la conversion linéaire sont égales.
8003 hex	La limite supérieure de la sortie TOR est inférieure à la limite inférieure.
8004 hex	La limite de la valeur de processus est hors de la plage de conversion linéaire.
8005 hex	La limite de la valeur de processus est inférieure à 0 ou supérieure à 10 000.
8006 hex	La consigne se trouve hors de la plage de conversion linéaire.
8007 hex	La consigne est inférieure à 0 ou supérieure à 10 000.
8008 hex	L'action de contrôle est différente de l'action déterminée au démarrage d'Auto-Tuning.

Codes d'erreur détectée d'Auto-Tuning

Le tableau suivant enregistre les messages d'erreur d'Auto-Tuning et décrit les causes possibles, ainsi que les actions de dépannage :

Code d'erreur détectée	Description
8009 hex	La limite de la valeur de processus est atteinte. Comme l'Auto-Tuning est un processus en boucle ouverte, la limite de la valeur de processus (PV) est la valeur maximale autorisée.
800A hex	La période d'échantillonnage est trop courte ou la consigne de sortie est trop basse. Augmentez la période d'échantillonnage ou la valeur de consigne de sortie d'Auto-Tuning.
800B hex	Kp est égal à 0.
800C hex	La constante de temps est négative. Il est possible que la période d'échantillonnage soit trop longue. Pour plus d'informations, consultez la section Limites de l'utilisation d'Auto-Tuning (<i>voir page 271</i>).
800D hex	Le retard est négatif.

Code d'erreur détectée	Description
800E hex	Erreur détectée lors du calcul de K_p. L'algorithme Auto-Tuning est instable (pas de convergence). Causes possibles : - Des perturbations du processus pendant l'Auto-Tuning ont causé une distorsion du calcul du gain statique du processus. - La réponse transitoire de la valeur de processus est insuffisante pour que l'Auto-Tuning puisse calculer le gain statique. - Les deux. Vérifiez les paramètres du PID et de l'Auto-Tuning, puis effectuez les réglages pour améliorer la convergence. Vérifiez également qu'aucune perturbation n'affecte la valeur de processus. Essayez de modifier : - la consigne de sortie, - la période d'échantillonnage. Assurez-vous que le processus n'est pas perturbé lorsque l'Auto-Tuning est en cours.
800F hex	La constante de temps dépasse le ratio de retard ($\tau/\theta > 20$). La régulation du PID n'est plus stable. Pour plus d'informations, consultez la section Limites de l'utilisation d'Auto-Tuning (voir page 271).
8010 hex	La constante de temps dépasse le ratio de retard ($\tau/\theta > 2$). La régulation du PID n'est plus stable. Pour plus d'informations, consultez la section Limites de l'utilisation d'Auto-Tuning (voir page 271).
8011 hex	La limite du gain statique K_p a été dépassée ($K_p > 10\,000$). La sensibilité de la mesure de certaines variables d'application est peut-être trop faible. La plage doit être réévaluée dans l'intervalle [0 à 10 000].
8012 hex	La valeur calculée de la constante de temps intégral (T_i) a été dépassée ($T_i > 20\,000$).
8013 hex	La valeur calculée de la constante de temps dérivé (T_d) a été dépassée ($T_d > 10\,000$).

Chapitre 7

Fonctions d'horloge

Présentation

Ce chapitre décrit les fonctions de gestion horaire des contrôleurs.

Contenu de ce chapitre

Ce chapitre contient les sujets suivants :

Sujet	Page
Fonctions d'horloge	260
Horodatage	261
Réglage de la date et de l'heure	263

Fonctions d'horloge

Introduction

Sur les Logic Controller équipés d'une fonction d'horodateur (RTC), vous pouvez utiliser les fonctions d'heure suivantes lorsque SoMachine Basic est connecté au Logic Controller :

- Les blocs fonction de **programmation** (*voir page 213*) permettent de contrôler des actions à des heures prédéfinies ou calculées.
- L'**horodatage** (*voir page 261*) permet d'affecter une date et une heure à des événements, et de mesurer la durée de ces derniers.

Cette horloge peut être réglée par un programme (*voir page 261*). La pile du contrôleur facilite le réglage de l'horloge en continuant de fonctionner pendant 1 an lorsque le contrôleur est hors tension. Le contrôleur n'a pas de batterie rechargeable. La batterie a une durée de vie moyenne de 4 ans et doit être remplacée avant sa fin de vie. Afin d'éviter toute perte de données, vous disposez d'un délai de 120 secondes pour remplacer la pile du contrôleur.

L'horloge affiche l'heure au format "24 heures" et tient compte des années bissextiles.

Horodatage

Introduction

Les mots système %SW49 à %SW53 contiennent la date et l'heure au format BCD, un format utile pour l'affichage sur un périphérique ou la transmission vers un périphérique. Ces mots système permettent de stocker la date et l'heure d'un événement.

Les instructions BTI permettent de convertir les dates et heures du format BCD au format binaire. Pour plus d'informations, consultez la section Instructions de conversion BCD/binaire (voir page 79).

Datage d'un événement

Pour dater un événement, il suffit d'utiliser des opérations d'affectation afin de transférer le contenu de mots système vers des mots mémoire, puis de traiter ces mots mémoire (par exemple, la transmission à l'unité d'affichage à l'aide de l'instruction EXCH).

Exemple de programmation

L'exemple suivant montre comment dater un front montant sur l'entrée %IO.1 :

Réseau	Instruction
0	LDR %IO.1 [%MW11:5 := %SW49:5]

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Dès qu'un événement est détecté, la table de mots contient :

Codage	Octet de poids fort	Octet de poids faible
%MW11	-	Jour de la semaine (1)
%MW12	00	Seconde
%MW13	Heure	Minute
%MW14	Mois	Jour
%MW15	Siècle	Année
(1) 1 = lundi, 2 = mardi, 3 = mercredi, 4 = jeudi, 5 = vendredi, 6 = samedi, 7 = dimanche		

Exemple de table de mots

Exemple de données pour le lundi 03 juin 2013 à 13:40:30 :

Mot	Valeur (hex.)	Signification
%MW11	0001	Lundi
%MW12	0030	30 secondes
%MW13	1340	13 heures, 40 minutes
%MW14	0603	06 = 3 juin
%MW15	2013	2013

Date et heure du dernier arrêt

Les mots système %SW54 à %SW57 contiennent la date et l'heure du dernier arrêt, et le mot %SW58 contient le code affichant la cause du dernier arrêt, au format BCD.

Réglage de la date et de l'heure

Introduction

Pour mettre à jour la date et l'heure, procédez au choix comme suit :

- SoMachine Basic

Deux modes sont proposés pour régler l'heure du Logic Controller :

- Manuel : ce mode affiche des champs de date et d'heure, et vous permet de choisir manuellement l'heure à configurer dans le Logic Controller.
- Automatique : ce mode affiche l'heure courante du PC exécutant SoMachine Basic et qui est utilisée pour régler l'heure du Logic Controller.

(Pour plus d'informations, consultez le document SoMachine Basic - Guide d'utilisation.)

- Mots système

Utilisez les mots système %SW49 à %SW53 ou le mot système %SW59.

NOTE : La date et l'heure sont réglables lorsque la fonction RTC est disponible dans votre Logic Controller (consultez le *guide de programmation* de votre Logic Controller.).

Utilisation des mots %SW49 à %SW53

Pour régler la date et l'heure à l'aide des mots système %SW49 à %SW53, le bit %S50 doit être mis à 0. Dès que vous réglez la date et l'heure, le bit %S51 doit être mis à 1. Cette opération a les conséquences suivantes :

- l'annulation de la mise à jour des mots %SW49 à %SW53 via l'horloge interne ;
- le transfert des valeurs écrites dans les mots %SW49 à %SW53 vers l'horloge interne.

Ce tableau indique le mot système contenant la date et l'heure (au format BCD) pour les fonctions d'horodateur (RTC) :

Mot système	Description
%SW49	xN Jour de la semaine (N=1 pour lundi)
%SW50	00SS : secondes
%SW51	HHMM : heure et minute
%SW52	MMJJ : mois et jour
%SW53	SSAA : siècle et année

Pour obtenir la liste complète des bits et mots système, consultez le *guide de programmation* de votre contrôleur.

Exemple de programmation :

Réseau	Instruction	Commentaire
0	LD %S50 R %S50	—
1	LD %I0.1 [%SW49 := %MW10] [%SW50 := %MW11] [%SW51 := %MW12] [%SW52 := %MW13] [%SW53 := %MW14] S %S50	Consultez la section Instructions de conversion BCD/binaire (voir page 79).

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité ([voir page 16](#)).

Les mots %MW10 à %MW14 contiennent les nouvelles date et heure (consultez la section Analyse du code BCD ([voir page 79](#)), et correspondent au codage des mots %SW49 à %SW53.

La table de mots doit contenir les nouveaux paramètres de date et d'heure :

Codage	Octet de poids fort	Octet de poids faible
%MW10	—	Jour de la semaine (1)
%MW11	—	Seconde
%MW12	Heure	Minute
%MW13	Mois	Jour
%MW14	Siècle	Année
(1) 1 = lundi, 2 = mardi, 3 = mercredi, 4 = jeudi, 5 = vendredi, 6 = samedi, 7 = dimanche		

Exemple pour le lundi 03 juin 2013 :

Mot	Valeur (hex.)	Signification
%MW10	0001	Lundi
%MW11	0030	30 secondes
%MW12	1340	13 heures, 40 minutes
%MW13	0603	06 = 3 juin
%MW14	2013	2013

Utilisation du mot %SW59

Pour mettre à jour la date et l'heure, vous pouvez également utiliser le bit système %S59 et le mot système %SW59.

Le réglage du bit %S59 sur 1 permet de configurer la date et l'heure à l'aide du mot %SW59. Le mot système %SW59 incrémente ou décrémente chacun des composants de la date et de l'heure sur un front montant.

Ce tableau décrit chaque bit du mot système %SW59 permettant de régler les paramètres de date et heure :

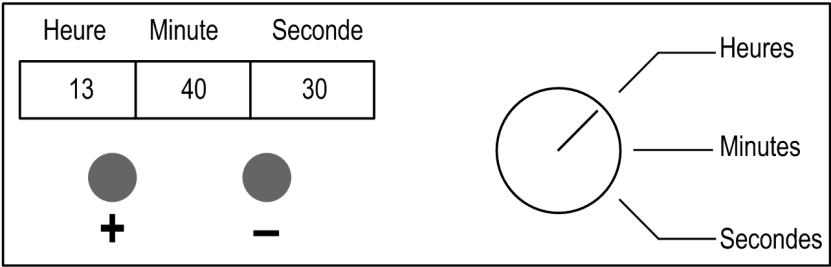
Incrément	Décrément	Paramètre
Bit 0	Bit 8	Jour de la semaine ⁽¹⁾
Bit 1	Bit 9	Secondes
Bit 2	Bit 10	Minutes
Bit 3	Bit 11	Heures
Bit 4	Bit 12	Jours
Bit 5	Bit 13	Mois
Bit 6	Bit 14	Années
Bit 7	Bit 15	Siècles ⁽¹⁾

(1) L'utilisateur ne peut pas modifier (incréméntation ou décréméntation) le jour de la semaine et les siècles.

Pour obtenir la liste complète des bits et mots système, consultez le *guide de programmation* de votre contrôleur.

Exemple d'application

Ce panneau avant permet de modifier le réglage de l'horloge interne (heures, minutes et secondes).



Description des commandes :

- Le commutateur Heures/Minutes/Secondes permet de sélectionner l'heure à modifier, respectivement à l'aide des entrées %I0.2, %I0.3 et %I0.4.
- Le bouton + permet d'incrémenter l'affichage de l'heure sélectionnée, à l'aide de l'entrée %I0.0.
- Le bouton - permet de décrémenter l'affichage de l'heure sélectionnée, à l'aide de l'entrée %I0.1.

Le programme suivant lit les entrées du panneau et règle l'horloge interne :

Réseau	Instruction	Commentaire
0	LD %M0 ST %S59	—
1	LD %I0.2 ANDR %I0.0 ST %SW59:X3	Heure
2	LD %I0.2 ANDR %I0.1 ST %SW59:X11	—
3	LD %I0.3 ANDR %I0.0 ST %SW59:X2	Minute
4	LD %I0.3 ANDR %I0.1 ST %SW59:X10	—
5	LD %I0.4 ANDR %I0.0 ST %SW59:X1	Seconde
6	LD %I0.4 ANDR %I0.1 ST %SW59:X9	—

NOTE : Pour obtenir le schéma à contacts équivalent, reportez-vous à la procédure de réversibilité (voir page 16).

Annexes



Annexe A

Paramètres du PID

Contenu de ce chapitre

Ce chapitre contient les sujets suivants :

Sujet	Page
Rôle et influence des paramètres PID	270
Méthode de réglage des paramètres PID	272

Rôle et influence des paramètres PID

Introduction

Cette section décrit le rôle et l'influence des paramètres PID.

Modèle de contrôleur PID

Le contrôleur PID SoMachine Basic applique une correction PID mixte (série-parallèle). Les actions intégrale et dérivée s'exécutent indépendamment et en parallèle. L'action proportionnelle agit sur la sortie combinée des actions intégrale et dérivée.

Algorithmes de calcul

Deux algorithmes de calcul sont utilisés, selon la valeur de la constante de temps intégral (T_i) :

- Si $T_i \neq 0$, l'algorithme incrémentiel est utilisé.
- Si $T_i = 0$, un algorithme positionnel est utilisé et un décalage de +5000 est appliqué à la sortie PID.

Influence des actions

L'action proportionnelle sert à influencer la vitesse de réponse du processus. Une augmentation de l'action proportionnelle se traduit par :

- une réponse plus rapide,
- une erreur statique plus faible,
- une stabilité moindre.

L'action intégrale permet d'annuler l'erreur statique. Une augmentation de l'action d'intégration (c'est-à-dire une diminution du temps intégral T_i) se traduit par :

- une réponse plus rapide,
- une stabilité moindre.

L'action dérivée est anticipative. En pratique, elle ajoute un terme qui prend en compte la vitesse de variation de l'écart (ce qui permet d'anticiper les modifications en accélérant les temps de réponse du processus lorsque l'écart augmente et en les ralentissant lorsque l'écart diminue). Une augmentation de l'action dérivée (c'est-à-dire du temps dérivé) se traduit par :

- une réponse plus lente,
- un dépassement réduit.

NOTE : Compte tenu du temps dérivé, T_d est le temps utilisé pour anticiper la variation de l'écart. Les valeurs de T_d trop basses ou trop élevées peuvent provoquer des oscillations indésirables.

Pour chaque action, il convient de trouver un compromis acceptable entre vitesse et stabilité.

Limites de la boucle de contrôle du PID

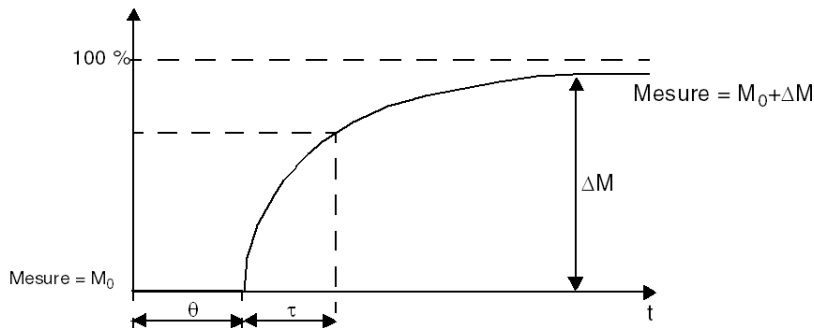
Le processus est assimilé à un premier ordre de temporisation pure avec une fonction de transfert :

$$H(p) = K \times \frac{e^{-\theta p}}{1 + \tau p}$$

où :

τ : constante de temps du modèle

θ : temporisation du modèle



Les performances de contrôle du processus dépendent du rapport $\frac{\tau}{\theta}$.

Le processus de contrôle PID approprié est atteint dans le domaine suivant : $2 < \frac{\tau}{\theta} < 20$.

Le contrôle PID convient particulièrement à la régulation des processus qui satisfont à la condition suivante :

- Pour $\frac{\tau}{\theta} < 2$, c'est-à-dire pour des boucles de commande rapides (θ faible) ou des processus à temporisation importante (τ élevé), le contrôle du processus PID n'est plus approprié. Dans ce cas, des algorithmes plus complexes doivent être utilisés.
- Pour $\frac{\tau}{\theta} > 20$, un contrôle de processus utilisant un seuil plus l'hystérésis suffit.

Méthode de réglage des paramètres PID

Introduction

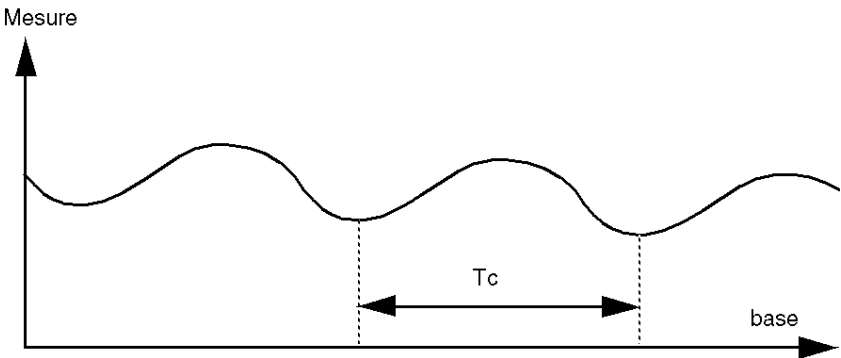
Plusieurs méthodes permettent de régler les paramètres PID. La méthode privilégiée est celle dite de Ziegler et Nichols, qui existe en deux variantes :

- réglage en boucle fermée ;
- réglage en boucle ouverte.

Avant d'appliquer l'une de ces méthodes, vous devez définir l'action (*voir page 249*) PID :

Réglage en boucle fermée

Dans ce réglage, une commande proportionnelle ($T_i = 0$, $T_d = 0$) lance le processus en augmentant un coefficient proportionnel jusqu'à ce qu'il recommence à osciller après application d'un niveau à la consigne du correcteur PID. Il suffit d'augmenter le gain proportionnel critique (K_{pc}) à l'origine de l'oscillation non amortie, et d'augmenter la période d'oscillation (T_c) pour réduire les valeurs et ainsi optimiser la régulation.



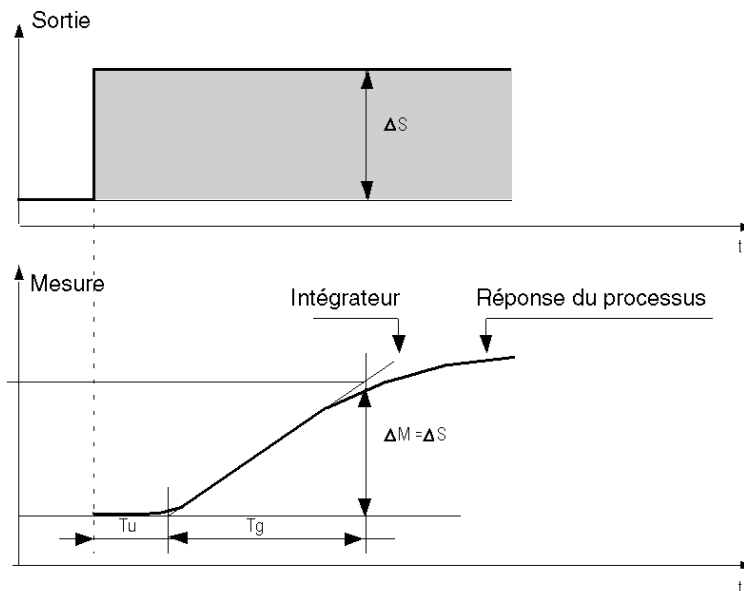
Selon le type de correcteur utilisé (PID ou PI), le réglage des coefficients est effectué avec les valeurs suivantes :

Correcteur	K_p : gain proportionnel	T_i : temps d'intégration	T_d : dérivée
PID	$K_{pc}/1,7$	$T_c/2$	$T_c/8$
PI	$K_{pc}/2,22$	$0,83 \times T_c$	—

Lorsque le PID est mis en œuvre avec Auto-Tuning, le paramètre **Correcteur AT dynamique** affecte la valeur du gain proportionnel (K_p). Le calcul du gain proportionnel dans le processus AT dépend de la vitesse sélectionnée du correcteur dynamique. L'option **Fast** offre une réponse rapide avec davantage de dépassement, tandis que l'option **Lent** génère une réponse plus lente avec moins de dépassement.

Réglage en boucle ouverte

Lorsque le régulateur est en mode manuel (*voir page 233*), vous appliquez un niveau à la sortie et vous lancez la procédure de réponse comme pour un intégrateur avec un temps de retard pur.



Le point d'intersection à droite, représentant l'intégrateur avec les axes de temps, détermine le temps T_u . Le temps T_g est ensuite défini comme le temps nécessaire pour que la variable contrôlée (mesure) ait la même taille de variation (% de l'échelle) que la sortie du régulateur.

Selon le type de correcteur utilisé (PID ou PI), le réglage des coefficients est effectué avec les valeurs suivantes :

Correcteur	K_p : gain proportionnel	T_i : temps d'intégration	T_d : dérivée
PID	$-1,2 T_g/T_u$	$2 \times T_u$	$0,5 \times T_u$
PI	$-0,9 T_g/T_u$	$3,3 \times T_u$	—

NOTE : Pour plus d'informations sur les unités des paramètres, consultez l'onglet (*voir page 245*) **PID**.

Cette méthode de réglage fournit également une commande particulièrement dynamique qui peut s'exprimer par des dépassements indésirables lors du changement des impulsions des consignes. Dans ce cas, abaissez le gain proportionnel jusqu'à obtenir le comportement requis. Cette méthode ne requiert aucune hypothèse sur la nature et l'ordre de la procédure. Vous pouvez l'appliquer aussi bien aux procédures stables qu'aux procédures d'intégration réelles. Dans le cas de procédures lentes (comme dans l'industrie du verre), l'utilisateur n'a besoin que du début de la réponse pour réguler les coefficients K_p , T_i et T_d .



0-9

%

Selon la norme IEC, % est un préfixe qui identifie les adresses mémoire internes des contrôleurs logiques pour stocker la valeur de variables de programme, de constantes, d'E/S, etc.

%KW

Selon la norme IEC, %KW représente un mot constant.

%MW

Selon la norme IEC, %MW représente un registre de mots mémoire (par exemple, un objet langage de type mot mémoire).

%Q

Selon la norme IEC, %Q représente un bit de sortie (par exemple, un objet langage de type sortie numérique).

(%PLS)

Unité de programmation qui possède 1 ou plusieurs entrées et renvoie 1 ou plusieurs sorties. Les blocs fonction (FBs) sont appelés via une instance (copie du bloc fonction avec un nom et des variables dédiés) et chaque instance a un état persistant (sorties et variables internes) d'un appel au suivant.

Exemples : temporisateurs, compteurs

E

entrée analogique

Convertit les niveaux de tension ou de courant reçus en valeurs numériques. Vous pouvez stocker et traiter ces valeurs au sein du Logic Controller.

L

langage à contacts

Représentation graphique des instructions d'un programme de contrôleur, avec des symboles pour les contacts, les bobines et les blocs dans une série de réseaux exécutés séquentiellement par un contrôleur (voir IEC 61131-3).

langage à liste d'instructions

Un programme écrit en langage à liste d'instructions (IL) consiste en une série d'instructions textuelles exécutées de manière séquentielle par le contrôleur. Chaque instruction comprend un numéro de ligne, un code d'instruction et un opérande (voir IEC 61131-3).

S

sortie analogique

Convertit des valeurs numériques stockées dans le Logic Controller et envoie des niveaux de tension ou de courant proportionnels.



Symbols

%C, 169
%DR, 179
%I, 25
%IW, 25
%IWS, 25
%KD, 33
%KF, 33
%KW, 28
%M, 23
%MD, 33
%MF, 33
%MSG, 191
%MW, 28
%Q, 25
%QW, 25
%QWS, 25
%R, 149
%S, 23
%SBR, 158
%SC, 163
%SCH, 214
%SW, 28
%TM, 141
%X, 23

A

ABS, 90
ACOS, 93
addition, 71
adressage
 format, 25
 objets d'E/S, 25
adresses d'E/S embarquées, 25
AND, 54
ANDF, 54
ANDN, 54
ANDR, 54
ASCII
 exemples, 201

ASIN, 93
ATAN, 93

B

blocs comparaison
 insertion d'expressions IL dans, 20
blocs fonction
 counter, 169
 description générale, 42
 drum register, 179
 LIFO/FIFO register, 149
 message, 191
 principes de programmation, 135
 shift bit register, 158
 step counter, 163
 timer, 141
blocs opération
 insertion d'instructions d'affectation dans, 19

C

calcul, 71
chaînes de bits, 37
COS, 93
counter
 configuration, 171
 description, 169
 exemple de programmation, 174

D

débordement
 index, 41
débordement d'index, 41
décrémentation, 71
DEG_TO_RAD, 95
DINT_TO_REAL, 96
division, 71

drum register
 configuration, 180
 description, 179
 exemple de programmation, 183

E

EQUAL_ARR, 116
EXCH, 189
EXP, 90
expression de comparaison
 insertion dans des réseaux en langage
 Schéma à contacts, 20
EXPT, 90

F

FIND_, 118
fonctions d'horloge
 horodatage, 261
 présentation, 260
 réglage de la date et de l'heure, 263
format d'adresse des entrées/sorties, 25
front descendant
 détection, 49
front montant
 détection, 48

I

incrémentation, 71
instruction NOP, 84
instructions
 arithmétiques, 71
 comparaison, 62
 conversion ASCII/entier, 101
 conversion ASCII/valeur flottante, 105
 conversion d'angle, 95
 conversion d'entier/nombre à virgule flot-

 tante, 96
 conversion entier/ASCII, 103
 conversion valeur flottante/ASCII, 107
 échange, 189
 END, 83
 NOP, 84
 pile, 109
 ROUND, 99
 saut, 85
 sous-programme, 87
 tables d'objet, 114
 trigonométriques, 93
instructions arithmétiques, 71
instructions booléennes, 47
instructions d'affectation
 chaînes de bits, 67
 insertion dans des réseaux en langage
 Schéma à contacts, 19
 mots, 69
 numériques, 66
 tables d'objets, 112
instructions d'échange
 EXCH1, 189
 EXCH2, 189
 EXCH3, 189
instructions de comparaison, 62
instructions de conversion
 BCD/binaire, 79
 mot simple/double, 81
instructions de décalage, 77
instructions de pile
 MPP, 109
 MPS, 109
 MRD, 109
instructions END, 83
instructions logiques, 75
instructions numériques
 décalage, 77
INT_TO_REAL, 96

L

LD, 50
LDF, 49, 50
LDN, 50

LDR, 48, 50
LIFO/FIFO register
 configuration, 151
 description, 149
 exemple de programmation, 155
 FIFO, 154
 LIFO, 153
LKUP, 126
LN, 90
LOG, 90

M

MAX_ARR, 120
MEAN, 131
message
 configuration, 194
 description, 191
 exemple de programmation, 199
MIN_ARR, 120
modbus
 requêtes et exemples standard, 203
multiplication, 71

N

N, 61

O

objets
 adresse directe, 40
 adresse indexée, 40
 définition, 22
 indexés, 40
 structurés, 37
 tables, 37
objets bit
 blocs fonction, 42
objets bit mémoire
 description, 23
objets flottants
 description, 33

objets mot
 bloc fonction, 43
 description, 28
objets mot double
 blocs fonction, 43
 description, 33
OCCUR_ARR, 122
opérateur NOT, 61
opérateurs
 affectation, 52
 AND, 54
 chargement, 50
 NOT, 61
 OR, 56
 XOR, 58
opérateurs AND, 54
opérateurs d'affectation, 52
opérateurs de chargement, 50
opérateurs OR, 56
opérateurs OR exclusifs, 58
opérations
 insertion dans des réseaux en langage
 Schéma à contacts, 19
OR, 56
ORF, 56
ORN, 56
ORR, 56

P

PID

- assistant de configuration, 239
- AT, onglet, 247
- auto-tuning, 222
- configuration standard, 226
- description, 252
- Entrée, onglet, 244
- états et codes d'erreur détectée, 255
- Général, onglet, 241
- modes de fonctionnement, 220
- onglet, 245
- paramètres, 270
- programmation et configuration, 254
- réglage en boucle fermée, 272
- réglage en boucle ouverte, 273
- Sortie, onglet, 249

R

- R, 52
- racine carrée, 71
- RAD_TO_DEG, 95
- REAL_TO_DINT, 96
- REAL_TO_INT, 96
- reste, 71
- ROL_ARR, 123
- ROR_ARR, 123

S

- S, 52
- schedule blocks
 - description, 214
- Schedule Blocks
 - programmation et configuration, 216
- shift bit register
 - configuration, 159
 - description, 158
 - exemple de programmation, 161
- SIN, 93
- SORT_ARR, 125
- soustraction, 71
- SQRT, 90
- ST, 52

- step counter
 - configuration, 164
 - description, 163
 - exemple de programmation, 165

- STN, 52

- SUM_ARR, 114

T

- TAN, 93

timer

- configuration, 142
- description, 141
- exemple de programmation, 147
- Type TOF, 145
- type TON, 144
- type TP, 146
- traitement numérique
 - affectation, 66
 - présentation, 65
- TRUNC, 90

V

- valeur absolue, 71

X

- XOR, 58
- XORF, 58
- XORN, 58
- XORR, 58